Practical Platform Engineering

From product and tenancy to paved roads, fleet, and isolated recovery

Birol Tilki

Version 1.0 — 16 August 2026

Contents

How to Use This Book

Who this book is for

This book is for intermediate-to-advanced DevOps, cloud, infrastructure, platform, security, and **SRE (Site Reliability Engineering)** practitioners who need to turn working delivery and security capabilities into an internal product that other teams can consume.

You should already be comfortable with Linux, Git, containers, **CI/CD (Continuous Integration and Continuous Delivery)**, cloud and Kubernetes fundamentals, infrastructure as code, networking, workload identity, observability, and the production delivery path taught in *Practical DevOps Engineering*. The book does not teach those subjects from first principles. It uses them as the surface on which platform-product decisions must operate.

You do not need to be a portal-product manager, Kubernetes operator author, public-cloud landing-zone architect, or reliability-program owner. Those disciplines contain important knowledge, but this book has a narrower promise: help you decide which capabilities belong on a platform, isolate tenants, offer a paved road with an exit, operate a shared control plane, measure finished jobs rather than vanity, run fleet lifecycle, and recover a control-plane failure without taking every tenant down.

This is not a catalog of developer-portal tools. Products change. The durable skill is being able to explain:

- who the user is and what job must finish;
- where one team's change, credential, or failure stops;
- what contract a team can rely on, and how they leave it; and
- what evidence proves the platform product is healthy—not only a tenant workload.

Relationship to earlier books

This book continues the Northwind Commerce system from *Practical DevOps Engineering* and *Practical DevSecOps Engineering*.

The DevOps book established one team's production delivery path: fast feedback, immutable artifact identity, reconciled infrastructure, a bounded Kubernetes runtime, workload identity, observable outcomes, progressive delivery, compatible data changes, safe asynchronous processing, GitOps reconciliation, cost and capacity, incident coordination, and reconstruction from durable evidence.

The DevSecOps book made security of that same path explicit: assets and harms, threat and risk decisions, attributable identity, privileged delegation, supply-chain trust, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance.

Those capabilities are prerequisites here. This book does not repeat their implementation and present it as new platform work. Instead, it asks the product questions that remain:

- Which repeated capabilities should other teams consume as a product?
- Where does tenant blast radius stop?
- What is a paved road, and what is a supported exit?
- How does a shared control plane avoid becoming shared root?
- Which measurements prove jobs finished, and which only prove the platform looked busy?

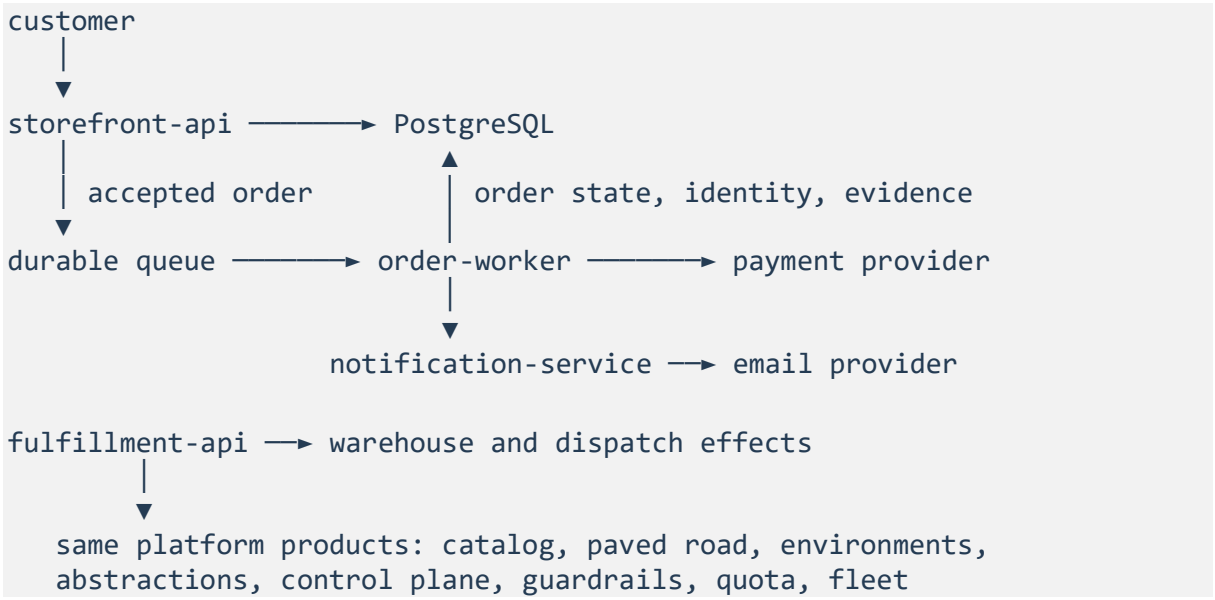
The published DevOps and DevSecOps v1.0 manuscripts remain unchanged. This book has a separate cumulative implementation under:

```
books/labs/platform/northwind/
```

The new lab does not read earlier labs' working trees at runtime. It carries reduced interface fixtures whose manifests record source paths, source checksums, local checksums, and consumers. `inherited/devops-v1.1/` is a lab-snapshot identifier, not the published DevOps book version.

The Northwind system

Northwind Commerce remains deliberately small enough to understand and large enough to require real platform decisions:



- `storefront-api` serves catalog reads and accepts orders.
- `order-worker` processes accepted orders asynchronously and can cause payment and inventory effects.
- `notification-service` sends non-critical confirmations.
- `fulfillment-api` is the second tenant's service. It must ship without inheriting cluster-admin or copying Storefront's unofficial path.
- PostgreSQL stores order and processing state.
- A durable queue separates acceptance from asynchronous completion.

The critical business outcome remains:

> A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

This book adds a critical platform outcome:

> Application teams can finish a production job on a reviewed paved road, inside an explicit tenant boundary, without inheriting shared cluster authority; the platform product has an owner, a contract, an exit, quota, and evidence that a control-plane failure does not take every tenant down.

A portal that launched is not that outcome. A green Storefront deployment is not that outcome. Shared cluster-admin that “unblocks” Fulfillment is the opposite of that outcome.

The cumulative platform path

The chapters form one dependency chain:

```
product users, jobs, promise, and refusals
  → capability intake decisions
  → tenant isolation and role boundaries
  → catalog and ownership
  → paved road with a supported exit
  → self-service environments
  → versioned infrastructure contracts
  → shared control plane as a product
  → guardrails and exception bindings
  → honest developer-experience measurement
  → tenant quota and platform unit cost
  → fleet onboard, upgrade, and deprecation
  → support and safe platform change
  → bounded control-plane recovery
```

Later chapters consume earlier decisions. The product brief determines which jobs intake may accept. Tenancy determines what an environment product may share. The paved road determines which defaults guardrails may bind. Quota and fleet change are meaningless until those contracts exist.

The cumulative product failure is shared authority: a platform that is really a ticket queue plus a cluster everyone can break. It develops across tenancy, environments, the control plane, fleet upgrades, and recovery.

Not every platform failure comes from that story. Exercises are labeled according to their relationship with the main narrative:

- **Cumulative product failure:** advances shared-authority and ticket-queue harm.
- **Connected consequence:** demonstrates harm enabled by an earlier platform decision.
- **Independent control failure:** proves that the control matters even without the cumulative story.

Four chapter forms

The book does not force every learning objective into the same structure.

Concept-led chapters

A concept-led chapter develops a mental model required for later production decisions. The reader may define users and jobs, model tenancy, or reject vanity measurement. Its durable output can be a reviewed reasoning artifact rather than executable code.

Concept-led does not mean detached theory. Every substantial conceptual section must affect a chapter decision, durable artifact, interpretation of evidence, failure diagnosis, or later implementation dependency.

Decision-led chapters

A decision-led chapter compares approaches under explicit user jobs and operational constraints. It records the recommendation, trade-offs, owner, and review trigger.

Adoption percentage, survey scores, and ticket volume are inputs—not the decision itself.

Implementation-led chapters

An implementation-led chapter starts from a red capability, establishes the necessary mental model, guides a contract or system change, exercises a controlled failure, diagnoses the evidence, and verifies containment or recovery where required.

Hybrid chapters

A hybrid chapter uses a concept or decision immediately to govern implementation. It still answers one primary production question.

There is no theory-to-practice percentage. **Decision 001** in `SERIES-DECISIONS.md` forbids adding an exercise merely to satisfy one. The practical requirement is consequential reader reasoning, meaningful change where appropriate, trustworthy evidence, and explicit limits on what has been proved.

Four kinds of evidence

Platform mechanisms often produce impressive output while leaving the important claim unanswered. This book separates four evidence categories:

1. **Mechanism evidence** proves that a catalog, portal, provisioner, controller, or scorecard operated.
2. **Decision evidence** proves that an owner evaluated user jobs, isolation, and trade-offs.
3. **Outcome evidence** proves that a team finished a production job inside the contract.
4. **Recovery evidence** proves that tenant blast radius was restored—not merely that a ticket closed.

These categories are complementary. A published portal does not prove jobs finish. A healthy Storefront order metric does not prove Fulfillment is isolated. A restored control-plane process does not prove every tenant is healthy.

Do not call tenant isolation recovery **Evidence of restored trust**. That phrase belongs to DevSecOps compromise recovery. Use **Evidence of restored isolation** or **Evidence of bounded platform-product recovery**.

Do not call platform-product indicators portfolio **SLOs (Service Level Objectives)**. Those belong to SRE. This book may use platform-product **SLIs (Service Level Indicators)** such as time-to-first-environment. Call unreliability against those indicators a **job-time budget**. Reserve **error budget** for SRE portfolio governance.

Lab states and commands

Use Python 3.13. From the Platform lab root:

```
python3 -m venv .venv
source .venv/bin/activate
make bootstrap
make test
make lint
make audit
```

The lab uses these states when they support the chapter's learning objective:

- **Start:** cumulative prerequisites exist, but the chapter capability is incomplete.
- **Baseline:** the evaluator runs successfully and proves the declared weakness is present.
- **Complete:** the chapter decision or implementation satisfies independent expectations.
- **Challenge:** a concept or decision scenario exposes incomplete reasoning. Correction happens inside that write-up; there is no `corrected` lab state.
- **Failure:** an inert local input exercises isolation, quota, upgrade, or authority failure.
- **Contained:** modeled harm and authority are bounded, but product health is not yet assumed restored.
- **Recovered:** required state is reconciled and platform-product plus tenant isolation evidence pass.

A successful baseline command means the evaluator correctly found the expected unsafe state. It does not mean the product is already healthy.

Chapter commands follow a stable pattern where applicable:

```
make chapter-NN-baseline
make chapter-NN-checkpoint
make chapter-NN-challenge
make chapter-NN-failure
make chapter-NN-contain
make chapter-NN-recover
make chapter-NN-verify-recovery
```

Not every chapter uses every command. Concept-led and decision-led chapters do not manufacture a failure or recovery sequence when classification, modeling, or decision correction is the real work.

Chapter snapshots

Versioned start and complete tags are the exercise snapshots:

```
v1.0-chapter-NN-start
v1.0-chapter-NN-complete
chapter-NN-start
chapter-NN-complete
```

The versioned tags are the immutable release identity. The reader-facing aliases point to the same commits in this freeze and may advance in a later release. Tags are curated exercise snapshots, not merge milestones. Do not infer the teaching contract from Git ancestry.

Failure state is generated from the complete snapshot. An unofficial plane patch, mixed backup, or starved quota must never be preserved as if it were a trusted completed reference.

Create a working branch from the start snapshot and compare against the complete reference without merging it as a shortcut:

```
git switch -c my-chapter-NN chapter-NN-start
git diff chapter-NN-start chapter-NN-complete
```

Working-tree procedure

From the Platform lab root:

1. from the Platform lab root, create the Python virtual environment and run `make bootstrap`, `make test`, `make lint`, and `make audit`;
2. start from `chapter-NN-start` (or work in the completed tree if you already have later chapters);
3. run the chapter baseline and confirm that it detects the documented red state;
4. follow the chapter's guided work and preserve its decision or implementation evidence;
5. run the completed checkpoint;
6. run the declared challenge or failure where applicable;
7. verify correction, containment, or recovery where the chapter requires it; and
8. retain the chapter's durable outputs.

A concept-led chapter may change only reviewed YAML artifacts. The absence of application code does not make the product brief disposable.

Safe failure simulations

The lab's failures are deterministic, local, inert, and non-destructive:

- cross-tenant access uses synthetic identifiers;
- quota and noisy-neighbor cases are numeric fixtures, not live load tests;
- control-plane restore mutates generated lab state only; and
- no real credential, cluster mutation, billing API, or destructive external action is included.

What local verification proves

The local evaluators verify structure, relationships, isolation, compatibility, expiry, vanity rejection, quota, fleet windows, and bounded restore within the declared model.

They do not prove that a real portal, identity provider, cluster fleet, cost system, or ticketing system enforces the same behavior.

Chapter 14 can prove that inventoried plane evidence was restored and tenant isolation held in the model. It cannot prove regional loss or a portfolio recovery-time program. Those remain SRE.

Five principles and four platform questions

The five principles established for the series remain active:

1. **Blast-radius control:** expose the smallest defensible scope to uncertain change, attack, or failure.
2. **Explicit contracts:** make identity, authority, trust, state, policy, outcomes, and failure behavior reviewable.
3. **Trustworthy evidence:** separate observations and expectations so a mechanism cannot approve itself.
4. **Reconciliation:** compare desired, recorded, external, and actual state, then make disagreement visible and owned.
5. **Recovery:** distinguish corrective action from proof that the protected outcome and required trust are healthy again.

Platform work repeatedly asks four platform questions:

1. Who is the user, and what job must finish?
2. What isolation boundary limits tenant blast radius?
3. What contract can a team rely on, and how do they leave it?
4. What evidence proves the platform product is healthy—not only a tenant workload?

Dedicated failures name only the principles and questions that materially apply.

Best Practice and Production Practice

The book retains two labels from the series:

- **Best Practice:** a strong default that is broadly useful.
- **Production Practice:** how that default must be validated or adapted for the actual users, jobs, isolation, failure modes, evidence quality, and recovery obligations.

For example, a paved road is a strong default. In production, its value depends on a finished job, remaining guardrails on exit, an owner, and evidence that unofficial forks fail conformance.

How to judge your work

Do not ask only whether the portal shipped. Ask:

> Who is the user, what job must finish, where does tenant blast radius stop, what contract can they rely on and leave, and what evidence proves the platform product—not only one workload—is healthy?

That question is the reading contract for the chapters ahead.

1. Define the Platform as a Product

Northwind already has a delivery path and a security path. Storefront can promote a digest, bind workload identity, and stop a bad release. Those mechanisms do not answer the first platform question:

> Who is the platform for, what jobs must it finish, and what does it refuse to own?

Without that answer, platform work becomes a queue of tickets and a portal launch. A team asks for cluster access. Another team copies YAML. Leadership asks when the developer portal will be ready. Each request may be reasonable, but the group cannot compare them until it knows which users, finished jobs, and refusals define the product.

This chapter establishes that foundation. You will name the users, the jobs they must finish, the promise the platform makes, the work it refuses, and the evidence that would prove the product works. The result is not a slide. Later chapters use it to decide which capabilities to productize, where tenants are isolated, what a paved road may contain, and which measurements are allowed to count as success.

1. An unsafe product definition

The inherited DevOps and DevSecOps systems protect a critical business outcome:

> A valid order is durably accepted and reaches a correct terminal state without duplicate charge, invalid inventory state, or permanent disappearance.

That statement is strong for one team operating one path. It does not describe a product other teams can consume.

Consider four examples:

- Fulfillment files a ticket for an environment and waits while Storefront already has one.
- A portal ships with a catalog homepage, and leadership counts the launch as success.
- A platform engineer grants cluster-admin “temporarily” so Fulfillment can ship this week.
- Time-to-first-environment and paved-road completion have no owner.

The Storefront order outcome can remain green through all four. Northwind still does not have a platform product.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 1 baseline:

```
make chapter-01-baseline
```

The command succeeds when it detects the intended unsafe definition:

```
chapter 01 baseline: portal-launch success correctly detected
```

The fixture treats a portal launch as success, leaves `obtain-bounded-environment` without an accountable owner, and uses `portal-launch` as the later proof that a service shipped. A homepage can exist while no team can finish a production job.

That distinction drives the chapter.

2. The production model: users, jobs, promise, and refusals

> *Theory — Internal product model*
>
> This model enables Northwind to select platform work according to finished user jobs rather than according to tickets, portals, or whoever shouts.

A platform user is someone who must finish a job

In this book, a platform user is a team or role that needs a finished production outcome from the platform. Storefront and Fulfillment are application-team users. The platform team is also a user of its own product: it must publish a path other teams can find.

A user is not “everyone with a laptop.” A user is not “the company.” Naming a user without a job produces a portal for nobody in particular.

A job is a finished outcome, not a request type

A job is complete when a stated production outcome exists. “Open a ticket” is not a job. “Browse the catalog” is not a job. “Obtain a tenant-scoped environment with a lease and no shared cluster-admin” is a job.

The job must name:

- who needs it;
- what finished looks like;
- who owns the product side of it; and
- what later proof would show it keeps working.

If the later proof is a portal launch, a satisfaction survey, ticket volume, or adoption percentage, the job has been replaced by vanity. Chapter 10 will make that measurement contract explicit. Chapter 1 must not encode vanity as the product’s definition of success.

A promise is what the product commits to finish

The promise is the product’s public contract. For Northwind:

> Application teams finish production jobs on a reviewed paved road, inside an explicit tenant boundary, without inheriting shared cluster authority.

The promise is not “we will build an internal developer platform.” That sentence names a category. It does not name a finished job or a blast-radius boundary.

A non-goal is work the platform refuses, with a remaining owner

A platform that will not refuse work becomes a ticket queue with extra YAML. Every refusal must name the team that still owns the work. Otherwise the refusal is a vacuum, and the ticket returns.

This book's first refusals are structural, not taste:

- custom order-pricing logic stays with the application team that owns the commercial decision;
- threat modeling and restored-trust claims remain DevSecOps outcomes the platform may expose, not rebuild;
- portfolio service-level objectives and regional-loss programs remain SRE.

Best Practice: Write non-goals in the same register as jobs, not in a slide appendix.

Production Practice: A non-goal is only real when a later intake request for that work is declined with the recorded remaining owner. Chapter 2 will test that.

Platform-product evidence is not tenant-workload evidence

Storefront's order-success rate can be healthy while Fulfillment cannot obtain an environment. A platform-product indicator must describe a user job the platform owns: time-to-first-environment, paved-road completion, catalog freshness. It is not a portfolio **SLO (Service Level Objective)**. Those belong to SRE.

3. Build Northwind's product foundation

The completed Chapter 1 model uses four files:

```
product/brief.yaml
product/users.yaml
product/jobs.yaml
product/non-goals.yaml
```

The separation is deliberate. Users describe who must finish work. Jobs describe finished outcomes and later proof. The brief binds promise, owner, and success evidence. Non-goals name refused work and remaining owners.

> **Practice — Name the users who must finish jobs**
>
> Identify who the platform is for before choosing capabilities or tools.

Open `product/users.yaml`. The register begins with three users:

```
users:
  - id: storefront-team
    role: application-team
    jobs: [obtain-bounded-environment, ship-on-paved-road]
  - id: fulfillment-team
    role: application-team
    jobs: [obtain-bounded-environment, ship-on-paved-road]
  - id: platform-team
    role: platform-product-owner
    jobs: [publish-owned-path]
```

Storefront already has an unofficial path. Fulfillment does not. If the product names only Storefront, Fulfillment will inherit cluster-admin to catch up. If it names “all engineers,” no job has an owner.

> **Practice — State falsifiable jobs**
>
> Express finished outcomes the platform can later prove without counting a portal launch.

Open `product/jobs.yaml`:

```
- id: obtain-bounded-environment
  user: fulfillment-team
  finished_outcome: A tenant-scoped environment exists with a lease and no
shared cluster-admin.
  owner: platform-team
  later_proof: time-to-first-environment
```

Inspect each job with three questions:

1. Can you tell when the job is finished without looking at a homepage?
2. Would the job remain important if Northwind replaced the portal product?
3. Is the named owner authorized to change the path that finishes the job?

Do not expand the register into every possible ticket type. Later chapters need a reviewable set of jobs, not a service-catalog dump.

> **Practice — Bind promise, owner, and job-completion evidence**
>
> Make success evidence match the jobs, and make ownership reciprocal.

Open `product/brief.yaml` and `product/non-goals.yaml`. The brief’s `success_evidence` values must be job proofs, not `portal-launch`. Each non-goal names a `remaining_owner`. The checkpoint verifies that users exist, jobs have owners, vanity proofs are rejected, and required refusals are present.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-01-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 01 checkpoint: product users, jobs, refusals, and job-completion
evidence verified
```

The audit validates the four Chapter 1 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- all independently required users, jobs, and non-goals exist;
- the brief names an accountable owner;
- success evidence and later proofs are not vanity labels;
- every job names a known user, owner, finished outcome, and later proof; and
- every non-goal names a remaining owner.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that three jobs are sufficient for a real company. It does not prove that the platform team has organizational authority, or that Fulfillment's warehouse effects are the right second tenant. Those are judgment claims.

This distinction is the chapter's evidence model:

Evidence category	Chapter 1 evidence
Mechanism evidence	Schemas and the relationship evaluator operated successfully.
Decision evidence	Users, jobs, promise, refusals, owners, and later proofs are explicit and reviewed.
Outcome evidence	Later environment, paved-road, and measurement chapters can be evaluated against these jobs.
Recovery evidence	Not yet produced; later chapters must prove tenant isolation and bounded platform-product recovery in the model.

Chapter 1 primarily creates decision evidence. Pretending that the local checkpoint proves teams can already finish jobs would weaken every later chapter.

4. Test the model under failure

Independent control failure — Portal launch counted as success

> **Practice — Diagnose vanity success**
>
> Replace portal-launch evidence with owned job-completion evidence.

The baseline fixture contains this model:

```
promise: Ship a developer portal.
owner: unassigned
success_evidence:
  - portal-launch
```

The problem is not merely missing fields. The model classifies a launch as the product.

Severity: high; every later measurement, intake, and support conversation will optimize the homepage instead of finished jobs.
Plausible harm: Fulfillment still waits for environments; unofficial cluster-admin spreads; the platform cannot tell whether it works.
Potential blast radius: every team asked to “adopt the platform”; Storefront’s unofficial path remains the real path.
Bounded by: later measurement contracts, intake refusals, and tenant isolation. None repairs a product definition that counts launches.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Platform questions

- **User and job:** The users are application teams that must obtain environments and ship on a paved road; a portal launch is not that job.
- **Isolation:** Not yet applicable as a tenancy decision. Chapter-local implication: an unowned product invites shared cluster-admin as the unofficial path.
- **Contract and exit:** There is no contract yet, only a homepage. Teams cannot rely on or leave a product that has not named a job.
- **Platform-product evidence:** `portal-launch` is vanity. Required proofs are time-to-first-environment and paved-road completion.

Diagnosis

Calling the product “a portal” encourages portal controls: pick a catalog UI, publish markdown, count unique visitors. Those may reduce orientation tickets, but they do not answer who finishes which job, who owns the wait, or what evidence would falsify success.

The missing owner makes intake and support ambiguous. The missing non-goals make every request in-scope. The missing job proof leaves Chapter 10 nothing honest to measure.

Correction

The completed model does not elevate the portal into the product. It defines users, jobs, a promise about tenant-bounded paved-road work, and success evidence that later chapters can collect without a homepage.

That correction changes later decisions:

- Chapter 2 must intake capabilities against these jobs, not against whoever asked twice.
- Chapter 3 must isolate Fulfillment so finishing a job does not require cluster-admin.
- Chapter 5 must make `ship-on-paved-road` a path with an exit, not a cage.
- Chapter 6 must make `obtain-bounded-environment` a product with a lease.
- Chapter 10 must treat time-to-first-environment as product evidence and portal-launch as a non-metric.

The concept is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common modeling errors

Naming the portal as the product

A portal is a distribution surface. If it disappeared, the jobs should still be nameable. If they are not, there was no product.

Treating Storefront's working path as the platform

One team's tribal YAML is not a product. Fulfillment's inability to reuse it is the demand signal.

Refusing nothing

A platform that accepts custom pricing, threat-model rebuilds, and portfolio SLO programs will not have time to offer environments. Record remaining owners.

Using tenant-workload health as platform success

Green order metrics can hide a ticket queue. Platform-product evidence must describe platform jobs.

Assigning ownership without authority

A platform team cannot own time-to-first-environment if it cannot change provisioning. Record the real decision path.

6. What changed

Before	After
Success was a portal launch.	**Success evidence is time-to-first-environment and paved-road completion.**
Users were “engineers.”	**Storefront, Fulfillment, and the platform team are named users with jobs.**
Jobs were ticket types.	**Jobs are finished outcomes with owners and later proofs.**
Refusals lived in conversation.	**Non-goals name remaining owners for pricing, DevSecOps rebuilds, and SRE programs.**
A valid schema could appear to prove a sound product.	**Structural, decision, outcome, and recovery evidence remain distinct.**
Later chapters had no stable user-job reference.	**Intake, tenancy, paved road, environments, and measurement inherit this brief.**

What changed was not merely four YAML files. Northwind now has a falsifiable product contract that later chapters can intake, isolate, pave, measure, and recover.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Platform product brief	product/brief.yaml	It retains the promise, owner, and job-completion evidence later measurement must not replace with vanity.
Job-and-refusal register	product/jobs.yaml and product/non-goals.yaml	They retain the finished outcomes and the work the platform will not absorb.

These artifacts should change when Northwind’s users, jobs, or organizational authority materially change—not whenever a portal theme is redesigned.

What You Learned

A platform product is defined by users, finished jobs, a promise, and explicit refusals. A portal, catalog, or cluster is a resource that may serve those jobs. Vanity evidence cannot prove the product. Schema checks can prove structural completeness within declared scope. They cannot prove that human judgment is correct. A concept earns its place when it changes later production decisions, evidence, diagnosis, or recovery.

Prove It

> **Independent Practice — Model a third user without copying Fulfillment**
>
> Decide how Northwind would admit a data-analytics team that needs a bounded read environment but must not ship on the order paved road.

Extend the Chapter 1 model without adding implementation policy yet:

1. Decide whether the analytics team is a new user or a bounded use of an existing user.
2. Name one job that is not `ship-on-paved-road`, including a finished outcome that is not a dashboard screenshot.
3. Choose later proof that is not `portal-launch`, `csat`, or `adoption-percentage`.
4. Assign an accountable owner and a non-goal if the team asks for production write access.
5. Identify one observation that would falsify the job.
6. Explain which material change would trigger review of your decision.

Do not copy the Fulfillment entry and rename it. Read-only analytics has different isolation, support, and exit consequences. Your durable output is the decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 1 capability when you can explain why a portal launch is not a finished job, trace every job to a known user and owner, describe evidence that would falsify the promise, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now knows who the platform is for and what it refuses. It still does not know which repeated capabilities should become products, which should stay with application teams, and which should be declined even when two teams ask.

Chapter 2 turns the product brief into an intake method with productize, leave, and decline decisions.

2. Decide Which Capabilities Become Platform Products

Chapter 1 named the users, jobs, promise, and refusals. That brief does not decide which painful YAML copy, security default, or dashboard widget becomes a platform product.

The production question is now:

> Which repeated capabilities should the platform own, which should stay with application teams, and which should be declined?

Without an intake method, every request looks like platform work. Fulfillment waits for an environment. Storefront asks the platform to host custom order-pricing rules because another team asked too. A portal backlog fills with service-specific widgets. Centralizing everything creates a golden cage. Productizing nothing leaves every team to rebuild the path.

This chapter records productize, leave, and decline decisions against the Chapter 1 jobs. Later chapters implement the accepted capabilities; they must not reopen the pricing refusal because two teams asked again.

1. An unsafe intake decision

A weak record says:

```
candidate: order-pricing-logic
treatment: productize
demand: two-teams-asked
```

It does not identify a user job from the product brief, repetition beyond ticket count, isolation impact, support cost, remaining owner, or review trigger. “Two teams asked” may justify a conversation. It cannot complete a productization decision.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 2 baseline:

```
make chapter-02-baseline
```

The command succeeds when it detects the intended unsafe intake:

```
chapter 02 baseline: two-teams-asked productization correctly detected
```

The fixture productizes custom order-pricing logic because two teams asked, leaves environment provisioning as a Storefront tribal path, and still claims artifact promotion is a product. Demand has been treated as a vote. A Chapter 1 non-goal has been absorbed. Fulfillment still waits.

That inversion drives the chapter.

2. The production model: demand, differentiation, and support cost

> *Theory — Capability intake*
>
> This model enables Northwind to select platform work according to repeated user jobs and support cost rather than according to whoever asked twice.

Demand is unfinished work that repeats, not a vote

Demand exists when more than one team must finish the same production job and currently cannot do so without a ticket, a copy, or inherited cluster authority. Fulfillment waiting for an environment is demand. Storefront and Fulfillment each rebuilding digest promotion is demand.

Two tickets for custom pricing are not that signal. They are two application teams asking the platform to host differentiating commercial logic. Ticket volume, the loudest requester, and “the portal needs it” are forbidden justifications. Chapter 10 will refuse those labels as metrics. Chapter 2 must not encode them as intake.

Differentiation stays with the team that owns the commercial decision

A capability is differentiating when changing it changes how a team competes, prices, or presents its own product. Order-pricing rules are Storefront’s commercial decision. Application dashboard widgets that only Storefront understands are Storefront’s presentation decision.

A capability is a candidate for the platform when the user job is the same across teams and the variation is configuration, not a new product. Environment leases, digest-pinned promotion, and an owned software catalog are that kind of work. They do not make Storefront’s prices better. They make Fulfillment able to ship without becoming a second unofficial platform.

Support cost and cognitive load decide thickness

Every productized capability becomes a support surface. If the platform hosts pricing, every commercial change is a platform ticket. If it themes every service dashboard, every widget is a platform ticket. Cognitive load moves from application teams to a queue that cannot finish `obtain-bounded-environment`.

A **thin platform** productizes the smallest set of capabilities that finish the named jobs. A **thick platform** absorbs adjacent requests until the product is a cage: teams cannot leave, and the platform team cannot change the path because every exception is unique.

Thickness is not a moral failing. Some organizations need a thicker paved road later. The intake record must say why, who pays the support cost, and what trigger would reverse the extraction. Reversible extraction is the default: start with the job, not with a portal catalog of everything Northwind might one day want.

Treatment is productize, leave, or decline

Northwind can:

- **Productize** a repeated capability that finishes a named user job, with an owner and a review trigger;
- **Leave** a capability with the application team that uniquely needs it, with a remaining owner and a trigger that would reopen extraction; or
- **Decline** work the product brief already refused, or work that would make the platform own differentiating logic, naming the remaining owner from the non-goal register.

“Backlog,” “maybe later,” and “the portal could do it” are not treatments.

Best Practice: Require the same fields on every treatment: demand, repetition, isolation impact, support cost, owner, and review trigger.

Production Practice: A decline is only real when it cites the remaining owner already recorded in `product/non-goals.yaml`. Chapter 1’s pricing refusal is tested here.

Isolation and contracts are not yet the product

Intake does not create tenant isolation. It decides which capabilities would expand or contain blast radius if left as tickets. Shared cluster-admin as the unofficial environment path is an isolation *impact*, not a tenancy model. Chapter 3 owns that model.

Intake is also not yet a paved-road contract. Teams cannot rely on or leave a path that has not been productized. Chapter 5 owns that contract. The decision record is the first reviewable promise about what the platform will and will not own.

3. Make the intake decisions

The completed Chapter 2 model uses three files:

```
intake/method.yaml
intake/candidates.yaml
intake/decisions.yaml
```

The separation is deliberate. The method names allowed treatments and forbidden demand labels. Candidates are proposed capabilities, not decisions. Decisions bind treatment to a user job or remaining owner.

> **Practice — Adopt a method that cannot treat a vote as demand**
>
> Name the three treatments and the justifications that must never productize a capability.

Open `intake/method.yaml`. The method forbids `two-teams-asked`, `loudest-ticket`, and `portal-needed`. It requires repetition, isolation impact, support cost, owner, and review trigger on every row. If those fields are missing, later chapters inherit a slogan instead of a decision.

> **Practice — Productize the jobs Fulfillment cannot finish**
>
> Keep environment provisioning and artifact promotion on the intake path, and bind each to a Chapter 1 job.

Open `intake/decisions.yaml`. The required productization decisions are:

```yaml
- id: decide-environment-provisioning
  candidate: environment-provisioning
  treatment: productize
  user_job: obtain-bounded-environment
  demand: both-tenants-wait-on-tickets
  repetition: storefront-and-fulfillment-copy-environment-manifests
  isolation_impact: shared-cluster-admin-if-left-as-a-ticket
  support_cost: one-ticket-per-environment-with-no-lease
  owner: platform-team
  review_trigger: new-tenant-or-environment-class
- id: decide-artifact-promotion
  candidate: artifact-promotion
  treatment: productize
  user_job: ship-on-paved-road
  demand: fulfillment-cannot-reuse-storefront-promotion
  repetition: each-team-rebuilds-digest-promotion
  isolation_impact: unofficial-latest-tags-if-left-tribal
  support_cost: platform-debugs-every-team-pipeline
  owner: platform-team
  review_trigger: new-artifact-type-or-registry
```

Inspect each productize row with three questions:

1. Does the user job exist in `product/jobs.yaml`, or was a new job invented to justify the request?
2. Would the capability still matter if Northwind replaced the portal product?
3. Is the named owner authorized to change the path that finishes the job?

The catalog decision productizes `publish-owned-path` for the same reason: owners currently live in chat. Chapter 4 will implement the catalog. Chapter 2 only decides that ownership discovery is platform work, not a wiki someone might update.

> **Practice — Decline differentiating logic and leave one-team presentation**
>
> Refuse work the brief already refused, and leave capabilities that only one team understands.

The pricing row must decline, not defer:

```yaml
- id: decide-order-pricing-logic
  candidate: order-pricing-logic
  treatment: decline
  remaining_owner: storefront-team
  demand: two-application-teams-requested-hosting
  repetition: commercial-rules-differ-per-team
  isolation_impact: platform-would-own-differentiating-business-logic
  support_cost: perpetual-pricing-change-tickets
  owner: platform-team
  review_trigger: pricing-becomes-a-shared-regulated-capability
```

`remaining_owner: storefront-team` must match the Chapter 1 non-goal. “Two application teams requested hosting” is recorded as the request that was refused, not as demand that productizes the capability. The forbidden label is `two-teams-asked` used as a productize justification.

Application dashboard layout is left with Storefront. One team today is not repetition. The review trigger is explicit: three or more teams sharing the same widget contract would reopen extraction. That is reversible, not a permanent exile.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-02-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 02 checkpoint: productize, leave, and decline decisions verified
```

The audit validates the three Chapter 2 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the method names productize, leave, and decline;
- environment provisioning and artifact promotion are productized against known jobs;
- order-pricing logic is declined with the non-goal’s remaining owner;
- productize rows do not use forbidden demand labels;
- every decision names repetition, isolation impact, support cost, owner, and review trigger; and
- leave and decline rows name a remaining owner.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that these five candidates are the right set for a real company. It does not prove that Northwind should remain a thin platform, or that pricing will never become a regulated shared capability. Those are judgment claims. The review triggers exist so the judgments can be reopened without pretending they were never made.

This distinction is the chapter's evidence model:

Evidence category	Chapter 2 evidence
Mechanism evidence	Schemas and the intake evaluator operated successfully.
Decision evidence	Productize, leave, and decline records trace to jobs, refusals, repetition, isolation impact, support cost, owners, and review triggers.
Outcome evidence	Later paved-road, environment, and catalog chapters can be evaluated against these accepted capabilities.
Recovery evidence	Not yet produced; later chapters must prove tenant isolation and bounded platform-product recovery in the model.

Chapter 2 primarily creates decision evidence. Pretending that the local checkpoint proves teams can already obtain environments or promote artifacts would weaken every later chapter.

4. Test the decision under failure

Independent control failure — Custom order-pricing logic is accepted as a platform capability because two teams asked

> **Practice — Decline differentiating logic that arrived as a vote**
>
> Reverse a two-teams-asked productization and keep environment provisioning and artifact promotion on the intake path.

The baseline fixture contains this model:

```
- id: decide-order-pricing-logic
  candidate: order-pricing-logic
  treatment: productize
  user_job: ship-on-paved-road
  demand: two-teams-asked
- id: decide-environment-provisioning
  candidate: environment-provisioning
  treatment: leave
  remaining_owner: storefront-team
```

The problem is not merely missing fields. The model classifies a vote as demand and a Chapter 1 non-goal as a platform job. Environment provisioning, the job Fulfillment actually cannot finish, is left as Storefront tribal knowledge.

Severity: high; every later paved-road, environment, and support conversation will optimize unique commercial requests instead of repeated jobs.
Plausible harm: the platform owns pricing change tickets; Fulfillment still waits; unofficial cluster-admin remains the environment path.
Potential blast radius: every application team asked to “put it on the platform”; Storefront’s unofficial path remains the real path.
Bounded by: Chapter 1 non-goals, later paved-road contracts, and tenant isolation. None repairs an intake method that productizes whoever asked twice.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Platform questions

- **User and job:** The users are application teams that must obtain environments and ship on a paved road. Hosting order-pricing rules is not that job; it is Storefront’s commercial work, already refused.
- **Isolation:** Not yet applicable as a tenancy decision. Chapter-local implication: leaving environment provisioning as a ticket preserves shared cluster-admin as the unofficial path, and productizing pricing would expand the platform’s blast radius into business logic it cannot isolate.
- **Contract and exit:** There is no paved-road contract yet. Teams cannot rely on or leave a product that absorbs differentiating logic and declines the repeated environment job.
- **Platform-product evidence:** `two-teams-asked` is a vote, not proof the product works. Required proofs remain time-to-first-environment and paved-road completion. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling demand “two teams asked” encourages intake by volume: accept pricing, accept widgets, accept the next unique pipeline. Those requests may be sincere, but they do not answer which user job finishes, who pays support cost, or what isolation impact follows if the capability stays a ticket.

The missing decline makes Chapter 1’s non-goal ornamental. The leave on environment provisioning makes Chapter 6’s lease product a surprise rather than an inherited decision. Mapping pricing onto `ship-on-paved-road` invents a job the brief never named.

Correction

The completed model does not productize pricing. It declines it with `storefront-team` as remaining owner, productizes environment provisioning and artifact promotion against the Chapter 1 jobs, leaves one-team dashboard layout with a review trigger, and forbids vote labels as productize demand.

That correction changes later decisions:

- Chapter 3 must isolate Fulfillment so the environment product does not require cluster-admin.
- Chapter 4 may implement the catalog because ownership discovery was productized, not because a portal needed a homepage.
- Chapter 5 must pave artifact promotion as a path with an exit, not as a unique Storefront pipeline.
- Chapter 6 must make `obtain-bounded-environment` a lease product because intake already accepted that job.
- Chapter 9 must not grow a golden cage of pricing exceptions; pricing was never a platform default.
- Chapter 13 must support environment and promotion failures as product incidents, not as Storefront tribal knowledge.

The decision is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common intake errors

Treating a second ticket as demand

Two teams can share a coincidence. Demand requires the same unfinished job, not the same complaint category.

Productizing to keep the peace

Accepting pricing “for now” creates a support surface that never expires. Decline with a remaining owner and a review trigger instead.

Leaving the painful repeated job because one team already copes

Storefront’s copied environment YAML is not a product. Fulfillment’s wait is the signal. Leave is for one-team differentiation, not for unofficial paths.

Inventing a user job to justify a request

If the job is not in `product/jobs.yaml`, either the brief is wrong or the request is out of scope. Do not mint `host-pricing-rules` to make a productize row schema-valid.

Measuring intake success as adoption percentage

A thick catalog of unused capabilities is not a thin platform. Later measurement must count finished jobs, not accepted candidates.

Assigning ownership without authority

The platform team cannot own environment provisioning if it cannot change tickets into leases. Record the real decision path.

6. What changed

Before	After
Custom pricing was accepted because two teams asked.	**Order-pricing logic is declined with Storefront as remaining owner.**
Environment provisioning stayed a Storefront ticket path.	**Environment provisioning is productized against obtain-bounded-environment.**
Artifact promotion was tribal YAML.	**Artifact promotion is productized against ship-on-paved-road.**
Demand meant ticket volume.	**Demand means a repeated unfinished job; vote labels cannot productize.**
Leave, decline, and backlog were interchangeable.	**Leave and decline name remaining owners and review triggers.**
A valid schema could appear to prove a sound productization.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has a reviewable intake contract that later chapters can pave, isolate, measure, and support without absorbing differentiating work.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Capability intake method	intake/method.yaml	It retains the treatments and forbidden demand labels later chapters must not reopen as votes.
First productization decisions	intake/decisions.yaml	They retain which capabilities the platform owns, leaves, or declines, with owners and review triggers.

These artifacts should change when Northwind’s jobs, repetition, or organizational authority materially change—not whenever a second team files a ticket.

What You Learned

A platform productizes repeated capabilities that finish named user jobs. It leaves one-team presentation with the team that owns it. It declines differentiating logic even when two teams ask. Demand is unfinished repeated work, not a vote. Schema checks can prove structural completeness within declared scope. They cannot prove that centralization is objectively correct. A decision earns its place when it changes later production implementation, evidence, diagnosis, or recovery.

Prove It

> **Independent Practice — Decide a third-party settlement calendar without copying the pricing row**
>
> A payments team asks the platform to host a settlement calendar because two other teams “might need it next quarter.”

Extend the Chapter 2 model without adding implementation policy yet:

1. Decide whether the calendar is a repeated unfinished job or a differentiating commercial schedule.
2. Choose productize, leave, or decline without using `two-teams-asked`, `loudest-ticket`, or `portal-needed` as demand.
3. Name the user job from Chapter 1 or the remaining owner if the work is refused.
4. State repetition, isolation impact, and support cost as they would exist if the platform hosted the calendar.
5. Identify one observation that would falsify your treatment.
6. Explain which material change would trigger review of your decision.

Do not copy the pricing decline and rename it. A shared regulated calendar has different isolation and review consequences from Storefront-only prices. Your durable output is the decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 2 capability when you can explain why two teams asking is not demand, trace every productize row to a known user job, decline a Chapter 1 non-goal with the recorded remaining owner, describe evidence that would falsify the treatment, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Northwind now knows which capabilities belong on the product and which do not. Tenant boundaries are still labels on a shared cluster. Fulfillment can still inherit cluster-admin “temporarily” to ship.

Chapter 3 models tenants, teams, and isolation boundaries so finishing a productized job does not require shared authority.

3. Model Tenants, Teams, and Isolation Boundaries

Chapter 2 decided which capabilities the platform owns. Environment provisioning and artifact promotion are on the intake path. Custom pricing is not. Those decisions do not answer the second platform question:

> Where does one team's change, credential, or failure stop?

Without that answer, Storefront and Fulfillment share cluster-admin patterns and a namespace-as-name convention. A debug RoleBinding or a noisy workload can become every team's incident. Fulfillment can still inherit cluster-admin "temporarily" to ship `fulfillment-api`. The product brief forbids shared cluster authority. The cluster still offers it.

This chapter records tenants, isolation dimensions, allowed sharing, and the authority each role may never inherit. Later chapters provision environments, operate the control plane, allocate quota, and recover a plane failure against this model. They must not treat a namespace label as isolation.

1. An unsafe isolation model

A weak record says:

```
id: fulfillment
namespace: fulfillment
role: cluster-admin
justification: temporary-to-ship-faster
```

It does not identify a tenant owner, isolation dimensions, prohibited inherited roles, or a blast-radius statement that stops at Fulfillment. "Temporarily" may justify a conversation. It cannot complete an isolation decision. A namespace name is a label. It is not a boundary.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 3 baseline:

```
make chapter-03-baseline
```

The command succeeds when it detects the intended unsafe model:

```
chapter 03 baseline: temporary cluster-admin inheritance correctly
detected
```

The fixture grants Fulfillment cluster-admin to ship this week, leaves Storefront on the same unofficial path, treats `namespace-label` as the only isolation dimension, and does not deny cluster-admin as shared authority. Isolation has collapsed before the catalog exists. That is this book's cumulative product failure.

That collapse drives the chapter.

2. The production model: tenants, dimensions, and prohibited inheritance

> *Theory — Tenant isolation*
>
> This model enables Northwind to finish productized jobs inside an explicit blast-radius boundary rather than by sharing cluster-admin.

A tenant is an isolation unit, not a namespace name

A tenant is a blast-radius boundary with an owner. Storefront is a tenant. Fulfillment is a tenant. The platform team is a user of the product, not a third application tenant. It operates the shared control plane; it does not live in `namespace: platform` as if that were isolation.

A team is the human owner of a tenant. An environment is a time-bounded instance of that tenant—non-production or production—that Chapter 6 will turn into a lease. A namespace, project, or account name may implement a tenant. It does not define one.

If isolation is “the namespace is called fulfillment,” a RoleBinding in that namespace, a credential copied from Storefront, or a debug grant on the cluster will ignore the name.

Isolation is a set of dimensions, not a single wall

Northwind isolates on five dimensions. Each dimension names what may be shared, what may never be inherited, and where blast radius stops:

Dimension	Stops	Must not be inherited
Identity	Who a credential can act as	Cross-tenant identity, cluster-admin
Network	Which services a workload can reach	Peer-tenant workload network
Quota	How much one tenant can consume	Unlimited burst into another tenant’s floor
Secrets	Which credentials a tenant can read	Peer-tenant secret read
Change authority	What a tenant change can mutate	Cluster-admin, platform-operator, peer-tenant change

A noisy neighbor is a quota and network failure: one tenant’s load or chatter becomes another tenant’s incident. Chapter 11 will allocate floors and ceilings. Chapter 3 must already deny unlimited burst and default-deny peer-tenant network. Otherwise later quota numbers decorate a shared cluster.

Best Practice: Write a blast-radius statement per tenant and per dimension. “The cluster” is not a statement. It is the failure.

Production Practice: A prohibited role is only real when a later binding that grants it fails. The Fulfillment cluster-admin grant is that test.

Shared-nothing and shared-control-plane are different products

A **shared-nothing** platform gives each tenant its own cluster or account. Blast radius is cheap to explain and expensive to operate. Fleet upgrade, quota, and control-plane recovery become many planes.

A **shared-control-plane** platform lets tenants consume one plane as a product and denies them plane-admin. Blast radius depends on the dimensions above. Fleet upgrade, quota, and recovery become one plane that must not take every tenant down.

Northwind chooses shared-control-plane. That is why Chapter 8 can operate the plane as a product and Chapter 14 can recover it without taking Storefront and Fulfillment with it. Shared cluster-admin is not shared-control-plane. It is shared-nothing's opposite: one plane, no tenants.

The control-plane API may be shared. Cluster-admin may not. Tenant secrets may not. Peer-tenant change authority may not.

Temporary inheritance is still inheritance

Fulfillment cannot finish `obtain-bounded-environment` today. Granting cluster-admin "until the environment product exists" makes the unofficial path the real path. Every later environment lease, paved-road default, and plane subject will have to unwind that grant. The cumulative product failure starts here: shared authority used as a shipping shortcut, collapsing isolation before the catalog exists.

The inherited DevSecOps authorization interface already forbids self-approval. It does not decide tenant blast radius. Platform tenancy must not treat a valid Storefront identity as authority to change Fulfillment, or a valid Fulfillment identity as cluster-admin.

3. Build Northwind's isolation model

The completed Chapter 3 model uses four files:

```
tenancy/tenants.yaml
tenancy/isolation.yaml
tenancy/roles.yaml
tenancy/sharing.yaml
```

The separation is deliberate. Tenants name owners, environments, dimensions, prohibited roles, and blast radius. Isolation dimensions name allowed sharing and denied inheritance. Roles name what a principal may never inherit. Sharing names the plane and quota pool tenants may consume, and the authority they may not.

> **Practice — Name tenants as isolation units**
>
> Bind each tenant to a Chapter 1 owner and a blast-radius statement that does not mean the cluster.

Open `tenancy/tenants.yaml`. Fulfillment is recorded as:

```
- id: fulfillment
  owner: fulfillment-team
  environments: [fulfillment-nonprod, fulfillment-prod]
  isolation_dimensions: [identity, network, quota, secrets, change-
authority]
  prohibited_inherited_roles: [cluster-admin, platform-operator]
  blast_radius: fulfillment-workloads-and-warehouse-effects-only
```

Inspect each tenant with three questions:

1. Would a credential stolen in this tenant be able to act as Storefront?
2. Would a debug RoleBinding here be able to change the control plane?
3. Is the named owner a Chapter 1 user, or a namespace string?

Do not add a `platform` tenant to make the platform team look symmetric. The platform team operates the shared plane. Treating it as a peer tenant invites cluster-admin "for the platform namespace."

> **Practice — Declare dimensions and denied inheritance**
>
> Make change authority deny cluster-admin, and make quota deny a noisy neighbor's unlimited burst.

Open `tenancy/isolation.yaml`. Change authority is the dimension the baseline collapsed:

```
- id: change-authority
  allowed_sharing: none
  denied_inheritance: [cluster-admin, platform-operator, peer-tenant-
change]
  blast_radius: a-tenant-change-cannot-mutate-another-tenant-or-the-
control-plane
```

Network may share `kubernetes-control-plane`. Quota may share `cluster-capacity-pool`. Identity, secrets, and change authority share nothing. If a dimension's `allowed_sharing` names a surface that is not in `tenancy/sharing.yaml`, the graph is incomplete.

> **Practice — Bind tenant-scoped roles and deny cluster-admin as sharing**
>
> Replace inherited cluster-admin with tenant-operator, and record cluster-admin as denied sharing.

Open `tenancy/roles.yaml` and `tenancy/sharing.yaml`. Tenant-developer and tenant-operator both list `cluster-admin` under `never_inherit`. Bindings grant Storefront and Fulfillment `tenant-operator` inside their own tenant. Sharing records the Kubernetes control plane as `shared-control-plane` and lists `cluster-admin` under `denied`.

A justification of `temporary-to-ship-faster` does not change the graph. The evaluator rejects the grant, not the adverb.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-03-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 03 checkpoint: tenant isolation and prohibited inheritance
verified
```

The audit validates the four Chapter 3 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- Storefront and Fulfillment exist with Chapter 1 owners;
- each tenant names the five isolation dimensions and prohibits cluster-admin;
- blast-radius statements are not aliases for the shared cluster;
- tenant-scoped roles may not inherit cluster-admin;
- no tenant binding grants cluster-admin;
- change authority denies cluster-admin;
- the control plane is shared as a product, not as cluster-admin; and
- sharing denies cluster-admin.

The expected identifiers live in a separate checkpoint file. The model under test does not emit its own passing expectations.

The checkpoint does not prove that a real cluster, identity provider, or network policy enforces these boundaries. It cannot discover unknown sharing outside the declared model. Those are later mechanism claims, and some of them remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 3 evidence
Mechanism evidence	Schemas and the isolation-graph evaluator operated successfully.
Decision evidence	Tenants, dimensions, prohibited roles, bindings, and denied sharing are explicit and reviewed.
Outcome evidence	Later environment, control-plane, quota, and recovery chapters can be evaluated against these boundaries.
Recovery evidence	Not yet produced; later chapters must prove tenant isolation held after a real modeled failure.

Chapter 3 primarily creates decision evidence. Pretending that the local checkpoint proves NetworkPolicies, quotas, or plane recovery already work would weaken every later chapter. Do not call this correction **Evidence of restored isolation**. Nothing runtime was restored. The model no longer grants the shared authority this book interrupts.

4. Test the model under failure

Cumulative product failure — Fulfillment inherits cluster-admin “temporarily” to ship faster, collapsing isolation before the catalog exists

> **Practice — Replace inherited cluster-admin with tenant-scoped roles**
>
> Interrupt shared authority at the binding, not after Fulfillment has shipped on the unofficial path.

The baseline fixture contains this model:

```
- id: fulfillment
  owner: fulfillment-team
  isolation_dimensions: [namespace-label]
  prohibited_inherited_roles: []
  blast_radius: the-cluster
bindings:
  - principal: fulfillment-team
    tenant: fulfillment
    role: cluster-admin
    justification: temporary-to-ship-faster
```

The problem is not merely missing fields. The model classifies a namespace label as isolation and a temporary grant as a shipping strategy. Storefront remains on the same unofficial path. Catalog, environments, and the control plane have nothing left to isolate.

Severity: high; every later environment, paved-road, quota, and recovery conversation will optimize a shared cluster instead of a tenant boundary.
Plausible harm: a Fulfillment debug RoleBinding changes Storefront; a noisy workload becomes every team’s incident; cluster-admin spreads as the way to finish `obtain-bounded-environment`.
Potential blast radius: both application tenants and the shared control plane; Storefront’s order path is no longer a separate failure domain.
Bounded by: later environment leases, plane subjects, quota floors, and control-plane recovery. None repairs a tenancy model that already granted cluster-admin.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must obtain a bounded environment and ship on a paved road. Cluster-admin is not that job; it is shared authority substituting for the environment product Chapter 2 already accepted.
- **Isolation:** The boundary is the five dimensions and the prohibited inheritance of cluster-admin. A namespace label and a temporary grant are not that boundary. Fulfillment's blast radius must stop at Fulfillment workloads and warehouse effects.
- **Contract and exit:** Not yet applicable as a paved-road contract. Chapter-local implication: the isolation model is the authority contract a team can rely on. Teams cannot leave a path that has not been paved; they also cannot rely on a tenant product that still offers cluster-admin.
- **Platform-product evidence:** A successful grant is not proof the platform works. Required later proofs remain time-to-first-environment and paved-road completion inside this boundary. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling isolation a namespace name encourages namespace controls: create `fulfillment`, copy Storefront's RoleBindings, grant cluster-admin so the team is not blocked. Those steps can ship `fulfillment-api` this week. They make Storefront, Fulfillment, and the control plane one failure domain.

The missing prohibited role makes Chapter 1's promise ornamental. The missing change-authority dimension makes Chapter 6's lease a label on the same cluster. The missing denial of cluster-admin in sharing makes Chapter 8's plane subjects inherit the unofficial path. Shared authority is no longer a risk on a slide. It is the recorded way Fulfillment ships.

Correction

The completed model does not grant Fulfillment cluster-admin. It defines Storefront and Fulfillment as tenants with Chapter 1 owners, five isolation dimensions, prohibited inheritance of cluster-admin and platform-operator, tenant-scoped role bindings, shared-control-plane consumption without plane-admin, and a blast-radius statement that is not the cluster.

Record this grant-and-reversal as the cumulative product failure the rest of the book interrupts. Later chapters inherit the corrected boundary. They must not reintroduce cluster-admin as a shipping shortcut.

That correction changes later decisions:

- Chapter 4 may publish catalog owners because tenants now exist as isolation units, not because a namespace list looked like a catalog.
- Chapter 6 must lease environments inside these tenants; a lease that implies cluster-admin has left the model.
- Chapter 8 must bind plane subjects that tenants may not inherit.
- Chapter 11 must allocate quota against the noisy-neighbor denial already recorded here.
- Chapter 14 must recover the shared plane without replaying one tenant's authority into another.

The concept is practical because it changes the production contract across the rest of the book. Adding an arbitrary command would not make it more practical.

5. Production reality

Common isolation errors

Treating the namespace as the tenant

A name is how humans find an object. Isolation is what a stolen credential, a RoleBinding, or a noisy process cannot cross.

Granting cluster-admin to unblock a second team

The second tenant is the teaching thread that makes tenancy real. If Fulfillment ships by inheriting Storefront's unofficial path, Northwind still has one tenant.

Making the platform team a peer tenant

Plane operators need plane-scoped authority in Chapter 8. They do not need a `platform` namespace that inherits cluster-admin "because it is the platform."

Isolating network and ignoring change authority

Default-deny east-west traffic does not stop a RoleBinding. Change authority is the dimension this chapter's failure actually collapsed.

Declaring shared-control-plane while sharing cluster-admin

If tenants can administer the plane, you have a shared cluster, not a plane product.

Using Storefront's order metrics as isolation evidence

A green order path can hide Fulfillment cluster-admin. Platform-product evidence must describe tenant boundaries and later job completion inside them.

6. What changed

Before	After
Fulfillment inherited cluster-admin to ship faster.	**Fulfillment is bound to tenant-operator; cluster-admin is prohibited and denied.**
Isolation was a namespace label.	**Isolation is identity, network, quota, secrets, and change authority.**
Blast radius was the cluster.	**Each tenant's blast radius stops at its workloads and effects.**
The plane was a shared cluster-admin path.	**The plane is a shared-control-plane product tenants may not administer.**
Temporary grants were a shipping strategy.	**Temporary inheritance is still inheritance and fails the graph.**
A valid schema could appear to prove isolation.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a reviewable isolation contract that later chapters can lease, pave, measure, and recover without treating shared authority as the path.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Tenant-and-isolation model	tenancy/tenants.yaml and tenancy/isolation.yaml	They retain owners, dimensions, prohibited roles, and blast-radius statements later environments and recovery must not replace with namespace labels.
Role boundary map	tenancy/roles.yaml and tenancy/sharing.yaml	They retain tenant-scoped bindings and the denial of cluster-admin as shared authority.

These artifacts should change when Northwind's tenants, isolation dimensions, or organizational authority materially change—not whenever a namespace is renamed.

What You Learned

A tenant is an isolation unit with an owner, dimensions, prohibited inheritance, and a blast-radius statement. A namespace name is not that unit. Shared-control-plane is a product tenants consume without inheriting cluster-admin. Temporary cluster-admin is the cumulative product failure this book interrupts. Schema checks can prove structural completeness within declared scope. They cannot discover unknown real cluster sharing. A concept earns its place when it changes later production decisions, evidence, diagnosis, or recovery.

Prove It

> **Independent Practice — Model a read-only analytics tenant without copying Fulfillment**
>
> A data-analytics team needs a bounded read environment for order events and must not ship on the order paved road.

Extend the Chapter 3 model without adding implementation policy yet:

1. Decide whether analytics is a new tenant or a bounded use of Storefront.
2. Name isolation dimensions that are not a copy of Fulfillment's warehouse-effects blast radius.
3. State what read sharing is allowed, and deny change authority on Storefront and cluster-admin.
4. Assign an owner from Chapter 1 or justify a new Chapter 1 user first.
5. Identify one observation that would falsify the boundary—for example a RoleBinding that can restart `storefront-api`.
6. Explain which material change would trigger review of your tenant record.

Do not copy the Fulfillment entry and rename it. Read-only analytics has different network, secret, and change-authority consequences. Your durable output is the isolation decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 3 capability when you can explain why a namespace label is not a tenant, trace every tenant to a known owner and prohibited cluster-admin inheritance, describe evidence that would falsify the blast-radius statement, distinguish structural validation from decision and outcome evidence, and explain what the baseline and completed checkpoint do and do not prove.

Next

Isolation is defined. Teams still cannot discover owners, dependencies, or the paved road without asking the platform group. Ownership lives in chat history and stale READMEs.

Chapter 4 publishes a software catalog and ownership map so a tenant can find the path without inheriting a ticket queue.

4. Publish a Software Catalog and Ownership Map

Chapter 3 defined tenants and prohibited cluster-admin. Isolation exists on paper. It does not answer the production question:

> How do teams find the service, owner, dependencies, and support path without asking the platform group?

Ownership still lives in chat history and stale READMEs. An incident or a dependency change cannot find an owner. Orphan services accumulate. Chapter 2 already productized `software-catalog` against `publish-owned-path`, with review trigger `team-rename-or-new-system`. A wiki that still reports green after a rename is not that product.

This chapter publishes catalog entries for Storefront, Fulfillment, and the platform products those teams consume. The catalog is a contract: every runnable system has a living owner, an escalation contact, a dependency list, and a failed check when metadata is stale. Later chapters will pave a path and lease environments against these owners. They must not treat a homepage as the map.

1. A green catalog with a deleted owner

A weak record says:

```yaml
system: fulfillment-api
owner: fulfillment-legacy-group
escalation: chat-history
last_reviewed_at: "2025-12-01T00:00:00Z"
reported_status: green
```

It does not identify a living Chapter 1 user, a Chapter 3 tenant, an escalation path that can be paged, or a review timestamp inside the freshness window. “Green” may describe a portal page. It cannot complete an ownership decision. A renamed team that leaves a deleted group in the catalog is an orphan with a badge.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 4 baseline:

```
make chapter-04-baseline
```

The command succeeds when it detects the intended unsafe catalog:

```
chapter 04 baseline: deleted-group green catalog correctly detected
```

The fixture keeps Storefront owned and current, leaves `fulfillment-api` owned by `fulfillment-legacy-group`, stamps that ownership in the previous year, and still reports green. The catalog has operated. The job has not.

That substitution drives the chapter.

2. The production model: catalog as contract, not wiki

> *Theory — Software catalog*
>
> This model enables Northwind to find a living owner and a dependency path without opening an orientation ticket, and to fail the catalog when those facts go stale.

A catalog is a system of record, not a page that launched

A software catalog is the contract for who owns a runnable system, how to escalate, what it depends on, and whether that metadata is still true. It is not a developer-portal homepage. Chapter 1 already refused `portal-launch` as success evidence. `publish-owned-path` names `catalog-freshness` as later proof. A published page without freshness is the same vanity in a different coat.

Mechanism evidence is that an entry exists. Decision evidence is that a living owner, tenant, and review timestamp were assigned. Outcome evidence would be that a team finished a job using that map. This chapter produces the first two. It does not prove a real portal search or HR directory.

An owner is living, or the entry is incomplete

A living owner is a Chapter 1 user who still exists. `fulfillment-team` is living. `fulfillment-legacy-group` is not. Escalation is a contact that can be reached when the owner is not at the keyboard. Lifecycle status (`active` or `deprecated`) is not a substitute for ownership. A deprecated system still needs someone who can answer.

The catalog must not emit its own passing grade. Completeness is computed from living ownership, escalation, dependencies, tenant binding, and `last_reviewed_at` against an independent `stale_before` expectation. `reported_status: green` is a claim. If the owner is dead or the timestamp is stale, that claim fails.

Best Practice: Bind each runnable system to a Chapter 3 tenant whose owner matches the catalog owner.

Production Practice: A rename is a catalog event. Chapter 2's review trigger `team-rename-or-new-system` is tested here: the old group must fail, not remain green.

Dependencies are how blast radius is discovered

An incident that cannot name dependencies cannot name who else to call. `storefront-api` depends on `order-worker` and PostgreSQL. `fulfillment-api` depends on warehouse dispatch. Platform products may have an empty list when they do not ride a tenant data path; they still need the field. `environment-provisioning` depends on `kubernetes-control-plane`, the shared surface Chapter 3 already named. Do not invent a second identifier for the same plane.

A missing dependency list is an incomplete entry. An invented "the platform" dependency is not a list.

Stale metadata is a failed contract, not a warning

If `last_reviewed_at` is older than the independent freshness window, the entry is stale. If the owner is not living, the entry is incomplete even when the timestamp is current. A team rename can produce exactly that: someone “reviewed” the page yesterday and left the deleted group in place. Freshness without a living owner is still an orphan.

This failure is a catalog-contract failure. It is not a runtime isolation event that needs containment and plane recovery. The control is that the entry cannot be complete. Do not call a failed check **Evidence of restored isolation**. Nothing tenant-runtime was restored. The catalog stopped lying.

3. Publish Northwind’s catalog

The completed Chapter 4 model uses three files:

```
catalog/systems.yaml
catalog/ownership.yaml
catalog/dependencies.yaml
```

The separation is deliberate. Systems name kind, lifecycle, and tenant. Ownership names living owner, escalation, and review time. Dependencies name what an incident must also find. The evaluator joins them. No file may declare itself green.

> **Practice — Register runnable systems against tenants**
>
> Put Storefront and Fulfillment services in the catalog with Chapter 3 tenant ids, and register the platform products Chapter 2 accepted.

Open `catalog/systems.yaml`. Fulfillment’s service is a tenant-scoped runnable system:

```yaml
- id: fulfillment-api
  kind: runnable-service
  lifecycle: active
  tenant: fulfillment
```

`software-catalog`, `environment-provisioning`, and `artifact-promotion` are `platform-product` entries. They have no tenant field. Chapter 3 refused to make the platform team a peer tenant; the catalog must not smuggle that tenant back in.

Inspect each runnable system with three questions:

1. Does its tenant exist in `tenancy/tenants.yaml`?
2. Would deleting this entry make an incident unable to find the service?
3. Is this a Northwind system, or a wiki section that names a category?

> **Practice — Assign living owners and escalation**
>
> Require a Chapter 1 user and a contact that is not chat history.

Open `catalog/ownership.yaml`:

```
- system: fulfillment-api
  owner: fulfillment-team
  escalation: fulfillment-oncall
  last_reviewed_at: "2026-08-16T00:00:00Z"
```

There is no `reported_status`. The checkpoint computes whether the owner is living and whether `last_reviewed_at` is after `stale_before`. `fulfillment-team` must match the Chapter 3 owner of tenant `fulfillment`. A catalog that says Storefront owns `fulfillment-api` has drifted from isolation even if both teams are living.

> **Practice — Record dependencies the incident will need**
>
> Name what a service reaches, including the shared plane where the platform product consumes it.

Open `catalog/dependencies.yaml`. Runnable systems have a non-empty `depends_on` list. The environment product names `kubernetes-control-plane` rather than a new string for the same shared surface.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-04-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 04 checkpoint: living owners, dependencies, and freshness verified
```

The audit validates the three Chapter 4 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- required systems `storefront-api`, `fulfillment-api`, and `software-catalog` exist;
- every system has ownership, escalation, and a dependency list;
- owners are living Chapter 1 users;
- runnable systems bind a known tenant whose owner matches;
- `last_reviewed_at` is not before the independent freshness window; and
- a green claim cannot survive a dead owner or a stale timestamp.

The expected identifiers and `stale_before` live in a separate checkpoint file. The catalog under test does not emit its own passing expectations.

The checkpoint does not prove that a real developer portal, identity provider, or HR system would return these owners. It cannot discover services that were never registered. Those remain later mechanism claims, and some of them a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 4 evidence
Mechanism evidence	Schemas and the catalog freshness evaluator operated successfully.
Decision evidence	Living owners, tenant bindings, escalation, dependencies, and review timestamps are explicit.
Outcome evidence	Later paved-road, support, and fleet chapters can be evaluated against these owners. Finding an entry is not yet proof a team finished ship-on-paved-road.
Recovery evidence	Not produced. A failed freshness check is a contract failure, not restored tenant isolation.

Chapter 4 primarily creates mechanism and decision evidence for `publish-owned-path`. Pretending that the local checkpoint proves portal search, on-call routing, or job completion would weaken Chapters 5 and 13.

4. Test the design under failure

Independent control failure — A renamed team leaves fulfillment-api owned by a deleted group while the catalog still reports green

> **Practice — Fail freshness and ownership before the entry can be complete**
>
> Inject a deleted-group owner onto a current catalog and refuse the green claim.

The completed catalog is healthy. The failure command does not rewrite it. It injects the renamed-team case against that snapshot:

```
make chapter-04-failure
```

Expected output:

```
chapter 04 failure: catalog correctly rejected a deleted-group owner
```

The injected record is:

```
system: fulfillment-api
owner: fulfillment-legacy-group
escalation: fulfillment-oncall
last_reviewed_at: "2026-08-16T00:00:00Z"
reported_status: green
```

The review timestamp is current. The baseline's stale date is not required for this failure. A rename can happen on a page someone touched today. Living owner and green claim are independent checks. Both must fail here.

Severity: high; every later incident, paved-road, and support conversation will page a group that does not exist.
Plausible harm: `fulfillment-api` becomes an orphan; Storefront cannot find a dependency owner; the platform remains the phone book.
Potential blast radius: every consumer of `fulfillment-api` and every platform product that trusts this catalog as a system of record.
Bounded by: Chapter 1 living users, Chapter 3 tenant owners, and the independent freshness window. None of those repair a catalog that is allowed to stay green.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Platform questions

- **User and job:** The user is the platform team finishing `publish-owned-path`, and the application teams who must find an owner without a ticket. A green page owned by `fulfillment-legacy-group` is not that job.
- **Isolation:** Catalog metadata is not a network policy. Chapter-local implication: `fulfillment-api` is a Fulfillment tenant system; its owner must be the living tenant owner. A deleted group cannot be the blast-radius owner Chapter 3 recorded.
- **Contract and exit:** The catalog contract is a living owner, escalation, dependencies, and computed freshness. A paved-road exit is not yet applicable. Teams cannot leave a path that has not been paved; they also cannot rely on a catalog that remains green after a rename.
- **Platform-product evidence:** `reported_status: green` is a self-emitted claim. Required proof is `catalog-freshness` computed against living owners. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the catalog green because the portal rendered encourages wiki controls: update markdown, keep the badge, skip the identity of the owner. After a rename, `fulfillment-legacy-group` still looks complete. Chat history becomes the real escalation path. Chapter 2's review trigger never fires because the page did not fail.

The missing living-owner check makes Chapter 1's user register ornamental. The missing tenant match makes Chapter 3's isolation owner a second, disagreeing record. The self-emitted green status makes the evaluator a display function.

Correction

The completed model does not let `fulfillment-api` report green under a deleted group. It requires a living Chapter 1 owner, a matching Chapter 3 tenant owner, escalation, dependencies, and a review timestamp inside the freshness window. Completeness is computed. The failure command proves the check still fails when only the owner is swapped and the timestamp stays current.

That correction changes later decisions:

- Chapter 5 must pave a path that names these system ids, not unofficial YAML copies with no owner.
- Chapter 12 must treat a team rename as a catalog freshness failure, not a fleet footnote.
- Chapter 13 must escalate to the living contact, not to chat history.
- Support and fleet change may not treat a green badge as proof the product is healthy.

The design is practical because the failed check is the product. Adding an arbitrary portal command would not make it more practical.

5. Production reality

Common catalog errors

Treating the portal as the catalog

If the homepage disappeared, owners and dependencies should still be nameable. If they are not, there was no catalog.

Allowing the entry to emit green

A system of record that scores itself cannot be failed by a rename. Keep observations (`last_reviewed_at`, owner id) separate from expectations (`stale_before`, living users).

Copying Storefront ownership onto Fulfillment

A living owner on the wrong tenant is still an isolation error. Match `tenancy/tenants.yaml`.

Leaving platform products out because they are not services

`publish-owned-path` is a platform job. If the catalog product has no owner, the platform is again a ticket queue.

Using Storefront’s order metrics as catalog health

Green orders can hide an orphan `fulfillment-api`. Platform-product evidence is freshness and living ownership.

Skipping dependencies because “everyone knows”

The first incident is when everyone does not know. Record the list before it is needed.

6. What changed

Before	After
fulfillment-api was owned by a deleted group and still green.	**Ownership requires a living Chapter 1 user; a green claim fails without one.**
Ownership lived in chat and stale READMEs.	**Owners, escalation, and review timestamps are catalog records.**
A namespace list stood in for a catalog.	**Runnable systems bind Chapter 3 tenants; platform products do not fake a platform tenant.**
Dependencies were tribal knowledge.	**Each runnable system names what an incident must also find.**
Freshness was a badge the page displayed.	**Freshness is computed against an independent stale_before expectation.**
A valid schema could appear to prove a living map.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has a catalog contract that later chapters can pave, support, and rename against without treating a green homepage as ownership.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Catalog ownership map	catalog/systems.yaml and catalog/ownership.yaml	They retain living owners, tenant bindings, and escalation later support and fleet rename must not replace with a wiki.
Stale-entry failure contract	catalog/ownership.yaml plus the Chapter 4 evaluator	They retain the rule that a deleted group or a stale timestamp cannot remain complete.

These artifacts should change when Northwind's systems, owners, or team names materially change—not whenever a portal theme is redesigned.

What You Learned

A software catalog is a contract for living owners, escalation, dependencies, and computed freshness. A green page owned by a deleted group is an orphan. Schema checks can prove structural completeness within declared scope. They cannot prove a real portal or HR system. A design earns its place when a rename fails the catalog instead of leaving it green.

Prove It

> **Independent Practice — Handle a Storefront team rename without copying the Fulfillment row**
>
> `storefront-team` is renamed to `storefront-commerce`. The portal still shows green.

Extend the Chapter 4 model without adding a paved-road policy yet:

1. Decide which catalog fields must change besides the owner string.
2. State whether `storefront-api`, `order-worker`, and `notification-service` fail together or can drift independently.
3. Name the Chapter 1 and Chapter 3 records that must move with the rename.
4. Identify one observation that would falsify “the rename is complete”—for example a remaining `storefront-team` owner on `order-worker`.
5. Choose whether `reported_status: green` may exist at all during the rename.
6. Explain which material change would trigger review of the catalog contract itself.

Do not copy the Fulfillment deleted-group row and rename it. Storefront owns three runnable systems on one tenant; a partial rename is a different failure than one orphan service. Your durable output is the freshness decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 4 capability when you can explain why a green catalog is not a living owner, trace every required system to a known user and tenant, describe evidence that would falsify freshness, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Teams can find systems, owners, and dependencies. Each still copies a private path to production. Storefront’s working path is tribal knowledge; Fulfillment copies YAML.

Chapter 5 builds a paved road teams can leave, so shipping does not require an unofficial fork or a golden cage.

5. Build a Paved Road Teams Can Leave

Chapter 4 published owners and dependencies. Teams can find `fulfillment-api`. They still cannot ship it on a shared path. Storefront's working path is tribal knowledge. Fulfillment copies YAML. Chapter 2 already productized artifact promotion against `ship-on-paved-road`. A catalog without a path leaves that job unfinished.

The production question is now:

> What supported path gets a team to production, and how do they leave it without becoming unsupported?

A mandatory golden path becomes a cage. An undocumented escape becomes shadow infrastructure. This chapter records a versioned paved-road contract, a scaffold that implements it, computed conformance, and a supported exit. The inherited DevOps release interface already requires promotion by `artifact_digest`. The unofficial fork that skips a slow template and promotes `latest` is not an exit. It is the connected consequence of offering a path without a way to leave it.

1. An unofficial fork of a tribal path

A weak record says:

```
system: fulfillment-api
path: unofficial
defaults_present: [latest-tag]
```

It does not identify the Chapter 1 job, the inherited digest-promotion default, workload-identity claims, a scaffold that implements the path, or a registered exit. "We skipped the slow template" may justify a conversation. It cannot complete `ship-on-paved-road`. Identity and artifact defaults are gone. Support has nothing left to support.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 5 baseline:

```
make chapter-05-baseline
```

The command succeeds when it detects the intended unsafe path:

```
chapter 05 baseline: unofficial fork of the paved road correctly detected
```

The fixture drops inherited artifact-digest promotion from the contract, leaves the scaffold covering only a catalog entry, grants Fulfillment an unofficial `latest-tag` fork, and registers no exit. Storefront's tribal YAML has become Fulfillment's shadow path. That is a connected consequence of productizing a road without paving it, and of paving it without an exit.

That fork drives the chapter.

2. The production model: a contract you can complete and leave

> *Theory — Paved road and supported exit*
>
> This model enables Northwind to finish `ship-on-paved-road` with inherited identity and artifact defaults intact, and to leave the scaffold without becoming unofficial.

A paved road is a product contract, not a mandate

A paved road is the default supported path to production. It names steps, defaults, a version, an owner, and a support level. It is not a golden cage. Every golden path needs a supported exit, an owner, and a review trigger.

The Northwind path is the job `ship-on-paved-road`. Its steps are:

1. a catalog entry from Chapter 4;
2. a digest-pinned artifact from the inherited DevOps release interface;
3. workload identity with the inherited required claims;
4. reviewed promotion of that digest.

Its defaults are `artifact-digest`, `workload-identity-claims`, and `no-cluster-admin`. Those defaults are not taste. `artifact-digest` is the inherited `promotion_identity`. Workload identity is the inherited short-lived federated model. `no-cluster-admin` is Chapter 3's prohibited inheritance. A path that promotes `latest` has left the inherited delivery contract.

A scaffold implements the steps so a new service can complete the path. The lab does not generate a live repository. Completeness is whether the recorded path kept the defaults, not whether a template ran.

An unofficial fork is not an exit

An exit is a reviewed, owned departure from the scaffold that still names lost defaults and remaining guardrails. An unofficial fork is an unreviewed departure that loses defaults and still asks the platform to debug the result.

Treatment	What the team gets	What must remain
Paved	Scaffold, defaults, full path support	All contract defaults
Exited	Support without the lost scaffold defaults	Remaining guardrails
Unofficial	No product support	Nothing the platform can honestly promise

You may leave the slow golden template. You may not leave digest promotion, workload identity, or the cluster-admin prohibition. Those remain guardrails on every path, including exits. Chapter 9 will bind them as defaults and exceptions. Chapter 5 must already refuse to treat them as optional template chrome.

Best Practice: Version the path. A team must be able to say which contract they completed or exited.

Production Practice: Completeness is computed. Conformance entries cannot emit a passing grade. `path: unofficial` always fails.

The connected consequence is enabled by the earlier product

Fulfillment can now find the path in the catalog. Chapter 2 told them artifact promotion is a platform product. When the template is slow and there is no exit, they fork unofficially to ship `fulfillment-api`. That fork is not an independent accident. It is what a productized road without an exit produces: shadow infrastructure that drops the defaults the inherited delivery path required.

3. Pave the road and register an exit

The completed Chapter 5 model uses four files:

```
paved-road/contract.yaml
paved-road/scaffold.yaml
paved-road/conformance.yaml
paved-road/exits.yaml
```

The separation is deliberate. The contract names the job, steps, and defaults. The scaffold names which steps it implements. Conformance records how each catalog system took the path. Exits name how a system left the scaffold without going unofficial.

> **Practice — Bind the path to the job and the inherited defaults**
>
> Make `ship-on-paved-road` consume digest promotion and workload identity rather than inventing a new identity for the same claims.

Open `paved-road/contract.yaml`:

```yaml
job: ship-on-paved-road
version: "1.0"
support_level: paved
steps:
  - id: catalog-entry
  - id: digest-pinned-artifact
  - id: workload-identity
  - id: reviewed-promotion
defaults:
  - artifact-digest
  - workload-identity-claims
  - no-cluster-admin
```

The evaluator loads `inherited/devops-v1.1/release/interface.yaml` and fails the contract if `artifact-digest` is missing while inherited promotion is by digest. It loads the identity interface and fails if workload-identity claims are missing. Do not rename those defaults when later chapters bind guardrails.

The path does not pretend environments are already a lease product. Chapter 6 owns that. A paved road that required a self-service environment this chapter does not have would fake outcome evidence.

> **Practice — Implement the steps, then compute conformance**
>
> Cover every contract step in the scaffold, and record whether each runnable system is paved, exited, or unofficial.

Open `paved-road/scaffold.yaml` and `paved-road/conformance.yaml`. The scaffold's `implements` list must include every contract step. `fulfillment-api` completes the paved path with all three defaults present. There is no `reported_status`. Unofficial is a failure, not a third success mode.

> **Practice — Register an exit that keeps remaining guardrails**
>
> Name owner, lost defaults, remaining guardrails, and a review date. Do not put digest promotion in `lost_defaults`.

Open `paved-road/exits.yaml`. `notification-service` leaves the slow template:

```
- id: notification-skip-slow-template
  system: notification-service
  owner: storefront-team
  lost_defaults: [golden-template-scaffold]
  remaining_guardrails: [artifact-digest, workload-identity-claims, no-
cluster-admin]
  review_at: "2026-11-16T00:00:00Z"
```

Non-critical confirmations do not need the full template. They still promote a digest, still bind workload identity, and still must not inherit cluster-admin. That is a supported exit. Fulfillment skipping the template *and* promoting `latest` is not.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-05-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 05 checkpoint: paved-road completion and supported exit verified
```

The audit validates the four Chapter 5 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the contract names `ship-on-paved-road` and a known owner;
- required defaults include inherited digest promotion, workload identity, and no cluster-admin;
- the scaffold implements every contract step;
- `storefront-api` and `fulfillment-api` have conformance records;
- paved systems keep required defaults;
- at least one supported exit exists;
- exits keep remaining guardrails and do not list them as lost; and
- unofficial forks fail.

The expected identifiers live in a separate checkpoint file. Conformance under test does not emit its own passing expectations.

The checkpoint does not prove that a live repository was scaffolded, that a real registry denied `latest`, or that a real identity provider issued the claims. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 5 evidence
Mechanism evidence	Schemas and the conformance evaluator operated successfully.
Decision evidence	The path, defaults, support difference, and exit trade-offs are explicit and reviewed.
Outcome evidence	fulfillment-api has a computed paved completion with identity and artifact defaults intact. That is not proof a production cluster admitted the digest.
Recovery evidence	Not produced. A failed unofficial fork is a path-contract failure, not restored tenant isolation.

Chapter 5 produces mechanism, decision, and limited outcome evidence for `ship-on-paved-road`. Pretending the local checkpoint proves a live pipeline would weaken Chapters 6 and 9.

4. Test the design under failure

Connected consequence — Fulfillment forks the path unofficially to skip a slow template, losing identity and artifact defaults

> **Practice — Reject the unofficial fork or register an exit**
>
> Inject a `latest-tag` fork onto the completed `fulfillment-api` path and refuse conformance.

The completed road is healthy. The failure command does not rewrite it. It injects the connected fork against that snapshot:

```
make chapter-05-failure
```

Expected output:

```
chapter 05 failure: unofficial fork correctly rejected
```

The injected record is:

```
system: fulfillment-api
path: unofficial
defaults_present: [latest-tag]
```

The contract, scaffold, and `notification-service` exit stay in place. Fulfillment could have returned to the paved road or registered an exit that kept remaining guardrails. They did neither. They skipped the template and lost the inherited defaults. The evaluator rejects the fork, the missing digest, the missing identity claims, and `latest-tag` as a forbidden default.

Severity: high; every later environment, guardrail, and support conversation will debug a shadow path that the product does not own.
Plausible harm: `latest` replaces digest identity; workload identity is dropped; cluster-admin returns as the unofficial way to ship.
Potential blast radius: Fulfillment workloads and any Storefront consumer that trusts the same unofficial pipeline pattern.
Bounded by: inherited release and identity interfaces, Chapter 3 prohibited roles, and the registered exit contract. None of those repair a path that treats unofficial as success.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Platform questions

- **User and job:** Fulfillment must finish `ship-on-paved-road` with identity and artifact defaults intact. Skipping a slow template by promoting `latest` is not that job.
- **Isolation:** Not yet applicable as an environment lease. Chapter-local implication: dropping workload-identity claims and `no-cluster-admin` reopens the shared-authority path Chapter 3 interrupted.
- **Contract and exit:** The contract is `northwind-production-path` version 1.0. Teams leave it through a registered exit that names owner, lost defaults, remaining guardrails, and `review_at`. An unofficial fork is not an exit.
- **Platform-product evidence:** A running template is not proof the product works. Required proof is computed `paved-road-completion` with inherited defaults present. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the template mandatory without an exit encourages two failures at once. Teams who comply wait. Teams who ship fork unofficially and drop the defaults the inherited delivery path required. The catalog made the path discoverable, so Fulfillment knew what to skip. Productizing artifact promotion without conformance made `latest` look like a shortcut rather than a broken contract.
The missing supported exit makes the road a cage. The missing inherited digest default makes promotion tribal again. The unofficial path makes Chapter 13's support model impossible: there is no product to support.

Correction

The completed model does not accept Fulfillment's unofficial fork. `fulfillment-api` returns to the paved road with digest, identity, and no cluster-admin intact. `notification-service` shows the other legal move: a registered exit that loses only the golden template and keeps remaining guardrails. Unofficial forks fail conformance either way.

This failure is a path-contract failure. It is not a runtime isolation event that needs containment and plane recovery. Do not call the rejected fork **Evidence of restored isolation**. Nothing tenant-runtime was restored. The path stopped treating shadow infrastructure as completion.
That correction changes later decisions:

- Chapter 6 must lease environments that this path can assume, not tickets that recreate unofficial YAML.
- Chapter 9 must bind remaining guardrails on paved and exited paths alike; an exit is not an exception to digest identity.
- Chapter 12 must version and deprecate unofficial paths rather than bless them as "whatever shipped."
- Chapter 13 must support paved and exited systems as products, and refuse unofficial forks as unsupported.

The design is practical because the failed check is the product. Adding an arbitrary repository-scaffold command would not make it more practical.

5. Production reality

Common paved-road errors

Making the template mandatory and calling that a product

A cage produces unofficial forks. If the only way off the path is shadow infrastructure, the path is not supported.

Treating `latest` as a default

Inherited promotion is by digest. `latest` is a forbidden default, not a convenience.

Putting remaining guardrails in `lost_defaults`

An exit that loses workload identity is an unofficial fork with paperwork.

Emitting conformance success from the system under test

A path that scores itself cannot fail a fork. Keep observations (`path`, `defaults_present`) separate from expectations.

Paving Storefront and leaving Fulfillment unofficial “until the template is faster”

The second tenant is why the path exists. If only Storefront is paved, Northwind still has tribal YAML.

Using a green pipeline UI as paved-road completion

A template run is mechanism evidence. `paved-road-completion` requires inherited defaults intact.

6. What changed

Before	After
Fulfillment forked unofficially to skip a slow template.	**Unofficial forks fail; Fulfillment completes the paved path or registers an exit.**
Promotion used latest.	**The path consumes inherited artifact-digest promotion.**
Identity defaults were tribal YAML.	**Workload-identity claims are a required default and a remaining guardrail.**
The golden path had no exit.	**notification-service leaves the template, keeps guardrails, and has a review date.**
Support meant debugging whatever shipped.	**Paved and exited are supported; unofficial is not.**
A valid schema could appear to prove a path.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a versioned path contract that later chapters can lease, guard, and support without treating a golden cage or a shadow fork as shipping.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Paved-road contract	paved-road/contract.yaml	It retains the job, version, steps, and inherited defaults later guardrails must not rename.
Supported-exit record	paved-road/exits.yaml	It retains how a team leaves the scaffold without becoming unofficial, including remaining guardrails and a review date.

These artifacts should change when Northwind's delivery defaults, inherited interfaces, or support model materially change—not whenever a template is restyled.

What You Learned

A paved road is a versioned product contract with steps, defaults, and support. A supported exit names owner, lost defaults, remaining guardrails, and a review date. An unofficial fork that skips a slow template and drops inherited identity and artifact defaults fails conformance. Schema checks can prove structural completeness within declared scope. They cannot prove a live repository was scaffolded. A design earns its place when leaving the path is a reviewed contract rather than shadow infrastructure.

Prove It

> **Independent Practice — Decide a Storefront exit without copying the Fulfillment fork**
>
> Storefront wants to skip the golden template because it injects a dashboard widget they already declined in Chapter 2.

Extend the Chapter 5 model without adding environment policy yet:

1. Decide whether Storefront should stay paved, register an exit, or whether an unofficial fork would fail and why.
2. Name lost defaults that are not `artifact-digest` or `workload-identity-claims`.
3. State remaining guardrails that Chapter 9 must still bind.
4. Choose an owner and a `review_at` that is not "when the template is faster."
5. Identify one observation that would falsify the exit—for example `storefront-api` promoting `latest` while claiming the exit.
6. Explain whether `order-worker` must follow the same treatment or can remain paved independently.

Do not copy the Fulfillment unofficial-fork row and relabel it an exit. Storefront already ships; the dashboard widget is Chapter 2's left capability, not a missing digest. Your durable output is the exit decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 5 capability when you can explain why unofficial is not an exit, trace the path to `ship-on-paved-road` and inherited digest promotion, describe evidence that would falsify conformance, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

The path exists, but environments are still tickets against a shared cluster. The paved road assumes an environment. Provisioning is still a wait, or a namespace that shares quota and credentials.

Chapter 6 offers self-service environments without sharing blast radius, so finishing the path does not require cluster-admin or an unbounded namespace.

6. Offer Self-Service Environments Without Sharing Blast Radius

Chapter 5 paved a path Fulfillment can complete or leave. That path still assumes an environment. Provisioning is a ticket. Namespaces share quota and credentials. Chapter 2 already productized environment provisioning against `obtain-bounded-environment`. Chapter 3 already named `fulfillment-nonprod` and denied `cluster-admin`. A ticket that grants "dev cluster admin" so Fulfillment can ship is the cumulative product failure continuing under a friendlier name.

The production question is now:

> How can a team obtain a bounded environment without inheriting everyone else's cluster?

Wait time returns, or self-service creates unbounded namespaces with production-like authority. This chapter records an environment product, a request, and a lease with **TTL (Time To Live)**, quota, and tenant-scoped identity. Storefront must not consume Fulfillment credentials. Fulfillment must not scale Storefront's environment to steal quota from `cluster-capacity-pool`. Isolation is a product invariant, not a namespace label.

1. A shared admin that still steals quota

A weak record says:

```
environment: storefront-nonprod
granted_role: dev-cluster-admin
mutated_by: fulfillment-team
```

It does not identify a Chapter 1 job, a Chapter 3 tenant environment, a request, an expiry, a quota bound, or inherited federated identity. "Dev cluster admin" may sound narrower than `cluster-admin`. It is still shared change authority. Fulfillment can scale Storefront's environment and consume the shared pool. The paved road has somewhere to deploy, and every tenant's blast radius is that somewhere.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 6 baseline:

```
make chapter-06-baseline
```

The command succeeds when it detects the intended unsafe product:

```
chapter 06 baseline: shared dev-cluster-admin quota steal correctly
detected
```

The fixture grants `dev-cluster-admin`, lets Fulfillment mutate `storefront-nonprod`, leaves `fulfillment-nonprod` without expiry, and replaces inherited short-lived federated identity with a referenced rotatable secret. Chapter 3 removed cluster-admin from tenant roles. The unofficial environment path put it back.

That residual drives the chapter.

2. The production model: a lease is the environment

> *Theory — Environment as product*
>
> This model enables Fulfillment to finish `obtain-bounded-environment` inside Chapter 3's blast-radius boundary, with a lease that expires and credentials Storefront cannot consume.

An environment is a leased instance of a tenant, not a namespace anyone can create

Chapter 3 already listed the environment ids this product must use: `storefront-nonprod`, `storefront-prod`, `fulfillment-nonprod`, `fulfillment-prod`. The product does not invent a fifth name. A request names tenant, environment, requester, and class. A lease binds that request to expiry, quota, credentials, and isolation. If the environment is not in the tenant inventory, it is not a product. It is an unbounded namespace.

Ephemeral environments (non-production) and durable environments (production) are both leases. Durable is not infinite. Production still expires or is reviewed. The difference is TTL length, not whether isolation applies.

The shared plane is `kubernetes-control-plane`. The quota pool is `cluster-capacity-pool`. Those identifiers were declared in Chapter 3. This chapter consumes them. Chapter 11 will allocate floors and ceilings against the same pool. Do not rename the pool here to make quota look new.

Credentials are tenant-scoped and short-lived

The inherited DevOps identity interface requires claims `subject`, `issuer`, `audience`, `expiry`, and `policy`. Its credential model is short-lived federated identity. Referenced rotatable secrets are unsupported. The environment product must not drop that model to make provisioning easier.

Audience is the shared plane, not a peer tenant. Subject must be scoped to the tenant or the environment. Storefront must not present `fulfillment-nonprod` as a credential. Fulfillment must not receive a credential that outlives the lease.

Best Practice: Bind each lease to a request and to a Chapter 3 environment id. An environment that cannot name its request is a ticket that skipped the product.

Production Practice: Isolation is computed. A lease cannot emit `status: isolated`. This chapter loads Chapter 3's `denied_inheritance` lists for network and secrets. Cross-tenant mutation, shared env-admin, expired-but-active leases, and those denied surfaces fail the invariant.

Quota on a lease is a bound, not a FinOps program

Each lease names `cluster-capacity-pool` and a unit count. That bound is what Fulfillment would steal by scaling Storefront. It is not portfolio showback. Chapter 11 owns floors, ceilings, and unit economics. Chapter 6 must already deny unlimited burst into a peer tenant's lease. Chapter 3 already denied `unlimited-burst-into-peer-quota`. The lease makes that denial a product check.

`dev-cluster-admin` is not a new role to productize. It is `cluster-admin` wearing a non-prod badge. Forbidden roles on the environment product include both.

3. Offer the environment product

The completed Chapter 6 model uses three files:

```
environments/product.yaml
environments/requests.yaml
environments/leases.yaml
```

The separation is deliberate. The product names the job, shared plane, quota pool, credential model, forbidden roles, and TTL. Requests are the intake. Leases are the granted instances. The evaluator joins them to Chapter 3 tenants and the inherited identity interface.

> **Practice — Bind the product to the job and the inherited identity**
>
> Consume `kubernetes-control-plane`, `cluster-capacity-pool`, and short-lived federated identity rather than inventing new names for the same surfaces.

Open `environments/product.yaml`:

```yaml
job: obtain-bounded-environment
shared_plane: kubernetes-control-plane
quota_pool: cluster-capacity-pool
credential_model: short-lived-federated
forbidden_roles: [cluster-admin, dev-cluster-admin, platform-operator]
ttl_hours: 168
```

The evaluator loads `inherited/devops-v1.1/identity/interface.yaml` and fails the product if federated identity is replaced by a referenced rotatable secret. It fails if the shared plane or quota pool is renamed.

> **Practice — Request a tenant environment that already exists in the inventory**
>
> Fulfillment asks for `fulfillment-nonprod`, not a new namespace string.

Open `environments/requests.yaml`. The requester must be the Chapter 3 tenant owner. A request from Fulfillment for `storefront-nonprod` is already a cross-tenant act.

> **Practice — Issue a lease with expiry, bound quota, and tenant-scoped claims**
>
> Keep Storefront's credentials unusable as Fulfillment, and keep Fulfillment unable to mutate Storefront's lease.

Open `environments/leases.yaml`. Fulfillment's non-production lease is:

```
- id: lease-fulfillment-nonprod
  request: request-fulfillment-nonprod
  tenant: fulfillment
  environment: fulfillment-nonprod
  granted_role: tenant-operator
  expires_at: "2026-08-23T00:00:00Z"
  quota:
    pool: cluster-capacity-pool
    units: 4
  credentials:
    subject: fulfillment-nonprod
    issuer: northwind-workload-identity
    audience: kubernetes-control-plane
    expiry: "2026-08-23T00:00:00Z"
    policy: tenant-scoped
  isolation:
    network: tenant-default-deny
    secrets: tenant-scoped
```

There is no `mutated_by` from another tenant. There is no `dev-cluster-admin`. Network is not `peer-tenant-workload-network`; the evaluator joins that denial from `tenancy/isolation.yaml` rather than copying the id. Secrets are not `peer-tenant-secret-read`. The lab does not provision a real namespace or **VPC (Virtual Private Cloud)**. Completeness is whether the lease keeps those invariants.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-06-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 06 checkpoint: tenant-scoped leases and isolation invariants
verified
```

The audit validates the three Chapter 6 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the product names `obtain-bounded-environment` and inherited federated identity;
- shared plane and quota pool reuse Chapter 3 identifiers;
- required leases exist for `storefront-nonprod` and `fulfillment-nonprod`;
- each lease binds a request, a known tenant, and an inventoried environment;
- granted roles are not `cluster-admin` or `dev-cluster-admin`;
- `mutated_by` cannot be another tenant's owner;
- expiry is present and unreclaimed leases are not past `as_of`;
- quota is bound on `cluster-capacity-pool`;
- credentials carry inherited claims, tenant-scoped subjects, and plane audience; and
- network and secrets are not in Chapter 3 `denied_inheritance`.

The expected identifiers and `as_of` live in a separate checkpoint file. A lease under test does not emit its own passing isolation status.

The checkpoint does not prove that a real cluster created a namespace, that a real identity provider issued the claims, or that a real network policy blocked traffic. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 6 evidence
Mechanism evidence	Schemas and the lease-state evaluator operated successfully.
Decision evidence	Job, plane, pool, TTL, forbidden roles, and isolation invariants are explicit.
Outcome evidence	Fulfillment has a computed lease for fulfillment-nonprod with no shared env-admin. That is not proof a production namespace exists.
Recovery evidence	Not produced. A denied cross-tenant scale is an isolation invariant, not restored tenant isolation after a live steal.

Chapter 6 produces mechanism, decision, and limited outcome evidence for `obtain-bounded-environment`. Pretending the local checkpoint proves `time-to-first-environment` in a real cluster would weaken Chapters 10 and 11.

4. Test the design under failure

Cumulative product failure — A shared “dev cluster admin” still lets Fulfillment scale Storefront’s environment to steal quota

> **Practice — Deny cross-tenant mutation and shared env-admin**
>
> Inject a Fulfillment scale onto the completed Storefront lease and refuse the invariant.

The completed product is healthy. The failure command does not rewrite it. It injects the residual shared-admin case against that snapshot:

```
make chapter-06-failure
```

Expected output:

```
chapter 06 failure: cross-tenant environment scale correctly rejected
```

The injected record is:

```
environment: storefront-nonprod
granted_role: dev-cluster-admin
mutated_by: fulfillment-team
```

TTL, federated identity, and Fulfillment’s own lease stay current. The baseline’s missing expiry and secret-model drop are not required for this failure. Shared env-admin can appear after the product already looks complete. Cross-tenant mutation and `dev-cluster-admin` must both fail on their own terms.

Severity: high; every later quota, control-plane, and recovery conversation will optimize a shared environment instead of a tenant lease.
Plausible harm: Fulfillment starves Storefront’s order path by scaling `storefront-nonprod`; Storefront credentials leak into Fulfillment; cluster-admin returns as “just non-prod.”
Potential blast radius: both application tenants and the shared `cluster-capacity-pool`; Storefront’s order path is no longer a separate failure domain.
Bounded by: Chapter 3 tenant inventory and prohibited roles, inherited federated identity, and lease TTL. None of those repair a product that still grants `dev-cluster-admin`.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `obtain-bounded-environment`. Scaling Storefront's namespace with `dev-cluster-admin` is not that job; it is shared authority substituting for a lease.
- **Isolation:** The boundary is the Chapter 3 tenant plus this lease: environment id, tenant-scoped credentials, quota bound on `cluster-capacity-pool`, default-deny network, tenant-scoped secrets, and no cross-tenant `mutated_by`. Fulfillment's blast radius must stop at Fulfillment.
- **Contract and exit:** The contract is the environment product and the lease. Teams leave an environment when the lease expires and is reclaimed, not by holding `dev-cluster-admin`. A paved-road exit is a different contract; this chapter does not replace it.
- **Platform-product evidence:** A created namespace is not proof the product works. Required later proof is `time-to-first-environment` inside this lease. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the grant "dev cluster admin" encourages the same unofficial path Chapter 3 interrupted: Fulfillment is blocked, someone shares environment authority, Storefront's quota moves, and the order path becomes a noisy neighbor. The paved road then deploys into a shared blast radius. Self-service without a lease produces unbounded namespaces. Tickets without isolation produce the same sharing, only slower.

The missing TTL makes every environment durable by accident. The missing federated identity makes credentials something Storefront and Fulfillment can copy. The missing quota bound makes `cluster-capacity-pool` a slogan. Shared env-admin makes Chapter 3's prohibited inheritance ornamental.

Correction

The completed model does not grant `dev-cluster-admin`. Fulfillment receives `fulfillment-nonprod` through a request and a tenant-operator lease with expiry, bound quota, federated claims, and default-deny isolation. Storefront's lease cannot be mutated by `fulfillment-team`. Isolation is recorded as a product invariant. Unreclaimed expiry fails. Cross-tenant secret and network sharing fail.

This failure is a lease-isolation invariant. It is not a runtime plane recovery. Do not call the denied scale **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live steal. The product stopped treating shared env-admin as provisioning.

That correction changes later decisions:

- Chapter 7 must offer infrastructure contracts inside these leases, not modules that recreate shared namespaces.
- Chapter 8 must bind plane subjects that tenants still may not inherit; `dev-cluster-admin` is not a plane role.
- Chapter 11 must allocate floors and ceilings against `cluster-capacity-pool` without renaming it.
- Chapter 14 must recover the shared plane without replaying one tenant's lease into another.

The design is practical because the denied mutation is the product. Adding an arbitrary cluster-provision command would not make it more practical.

5. Production reality

Common environment-product errors

Renaming cluster-admin to dev-cluster-admin

Non-prod is not a different blast-radius physics. Forbidden is forbidden.

Creating namespaces that are not in the tenant inventory

A new string is how unbounded environments start. Use Chapter 3's environment ids.

Issuing credentials that outlive the lease

A dead lease with a live secret is a stolen identity waiting for a user.

Treating TTL as optional on production

Durable is a longer lease, not an immortal namespace.

Using Storefront's order metrics as environment health

Green orders can hide Fulfillment waiting on a ticket, or Fulfillment scaling Storefront. Platform-product evidence is a tenant-scoped lease.

Building a quota program in this chapter

Bound the lease. Leave floors, showback, and unit economics to Chapter 11. Do not invent a second pool name.

6. What changed

Before	After
Fulfillment scaled Storefront with dev-cluster-admin.	**Cross-tenant mutation and shared env-admin fail the lease invariant.**
Environments were tickets or unbounded namespaces.	**A request creates a tenant-scoped lease with TTL.**
Credentials were copyable secrets.	**Leases consume inherited federated identity with tenant-scoped claims.**
Quota was the shared cluster.	**Each lease is bound on cluster-capacity-pool.**
Isolation was a namespace label.	**Network default-deny and tenant-scoped secrets are lease fields.**
A valid schema could appear to prove a provisioner.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has an environment product that later chapters can contract, reconcile, meter, and recover without treating shared env-admin as self-service.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Environment product contract	environments/product.yaml	It retains the job, shared plane, quota pool, federated identity, forbidden roles, and TTL later quota and plane chapters must not rename.
Lease and isolation evidence	environments/leases.yaml	It retains tenant-scoped expiry, quota bounds, credentials, and the invariant that Fulfillment cannot mutate Storefront.

These artifacts should change when Northwind’s tenants, environment classes, or inherited identity model materially change—not whenever a namespace prefix is restyled.

What You Learned

An environment is a leased instance of a tenant with expiry, bound quota, and tenant-scoped identity. `dev-cluster-admin` is shared authority. Schema checks can prove structural completeness within declared scope. They cannot prove a real cluster provisioned a namespace. A design earns its place when Fulfillment can obtain `fulfillment-nonprod` without scaling Storefront.

Prove It

> **Independent Practice — Lease a read-only analytics environment without copying Fulfillment**
>
> A data-analytics team needs a bounded read environment for order events and must not consume Storefront credentials or inherit `dev-cluster-admin`.

Extend the Chapter 6 model without adding infrastructure-contract policy yet:

1. Decide whether analytics uses a new Chapter 3 tenant and environment id, or a bounded class of Storefront.
2. Name TTL and quota so a forgotten analytics namespace cannot starve `cluster-capacity-pool`.
3. State which inherited identity claims still apply, and which Storefront credential must fail if presented.
4. Deny the network and secret sharing that would let analytics restart `storefront-api`.
5. Identify one observation that would falsify the lease—for example `mutated_by: analytics-team` on `storefront-prod`.
6. Explain which material change would trigger review of the environment product, not just one lease.

Do not copy the Fulfillment non-prod lease and rename it. Read-only analytics has different durability, secret, and change-authority consequences. Your durable output is the isolation decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 6 capability when you can explain why `dev-cluster-admin` is still shared authority, trace a lease to a request, a Chapter 3 environment, and inherited identity claims, describe evidence that would falsify isolation, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Environments exist, but teams still own raw modules instead of versioned contracts. Isolated namespaces still require each team to compose storage, identity, and networking primitives.

Chapter 7 abstracts infrastructure behind reviewable contracts, so a tenant request binds to a version rather than to a hidden module.

7. Abstract Infrastructure Behind Reviewable Contracts

Chapter 6 leased `fulfillment-nonprod`. Fulfillment can obtain a bounded environment. That environment still asks the team to compose storage, identity, and networking primitives. Tickets return as module reviews. Copy-paste returns as Terraform addresses in the tenant request. Chapter 2 already productized environment provisioning and artifact promotion. A lease that still exposes the module is the same ticket queue with nicer YAML.

The production question is now:

> What contract can a team rely on without owning the underlying infrastructure modules?

Abstractions that hide too much become magic. Abstractions that hide too little recreate the ticket queue. This chapter records an infrastructure **API (Application Programming Interface)**: versioned capabilities, tenant parameters, hidden module internals, and a compatibility policy. Storefront and Fulfillment bind to a version. They do not name a Terraform resource. A module refactor that changes a tenant-visible field without a version bump is not an internal cleanup. It is an independent control failure that breaks Fulfillment silently.

1. A module that became the tenant API

A weak record says:

```yaml
capability: tenant-storage
version: "1.0"
tenant_parameters: [sku, size-gb]
parameters:
  class: small
  terraform-resource-address: google_storage_bucket.storefront
```

It does not identify a tenant-visible field list Fulfillment can keep sending, a hidden-module boundary, a compatibility rule for rename, or inherited federated identity. “We refactored the module” may be true in the platform repo. Fulfillment still sends `class`. The request now includes a Terraform address. The network request can name `peer-tenant-workload-network`. Compatibility does not call parameter rename breaking.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 7 baseline:

```
make chapter-07-baseline
```

The command succeeds when it detects the intended unsafe contracts:

```
chapter 07 baseline: leaked module internals and silent field rename
correctly detected
```

The fixture lets Storefront bind `terraform-resource-address`, silently renames storage `class` to `sku` while Fulfillment still sends `class`, replaces inherited short-lived federated identity with a referenced rotatable secret, lets Fulfillment bind Chapter 3’s denied `peer-tenant-workload-network`, and names no breaking parameter changes. Isolated environments exist. The primitives are still the product. That substitution drives the chapter.

2. The production model: a versioned contract, not a module checkout

> *Theory — Infrastructure API*
>
> This model enables Fulfillment to request storage, identity, network, and promotion inside a Chapter 6 lease by binding a contract version, without owning the module that implements it.

A contract is the tenant API; the module is not

An infrastructure contract names a capability, a version, the parameters a tenant may set, and the module fields the tenant must not set. Platform-owned modules implement the version. Tenant parameters are the supported request. Hidden internals—Terraform resource addresses, provider projects, **CNI (Container Network Interface)** plugin config, **IAM (Identity and Access Management)** role ARNs, **OIDC (OpenID Connect)** thumbprints, registry credentials—are how the platform keeps the promise. They are not a second API.

The Northwind catalog productizes four capabilities tenants already need to finish jobs:

Capability	Tenant parameters	Hidden on purpose
tenant-storage	class, size-gb	Terraform address, provider project
workload-identity	subject	OIDC thumbprint, IAM role ARN
tenant-network	isolation	CNI config, security-group id
artifact-promotion	artifact-digest	registry credentials, Helm release name

Those are not new jobs. Storage and network sit inside `obtain-bounded-environment`. Identity and promotion already exist as inherited DevOps interfaces. The contract is how a team relies on them without composing the modules.

Hiding every field creates magic: Fulfillment cannot tell what they requested, and support cannot tell what broke. Publishing every field recreates the ticket queue: every module refactor becomes a tenant outage or a review of internals. The supported surface is the versioned parameter list. Leaving a contract is a new version plus a migration note, not a paved-road exit and not an exception to digest identity.

Compatibility is a policy, not a changelog comment

Adding, removing, or renaming a tenant parameter is breaking. A hidden-module refactor is not. Breaking changes require a new version and a migration note. Recording a rename against the only live version, with no note, is how Fulfillment keeps sending `class` after the module now expects `sku`.

Production Practice: Some organizations treat additive optional parameters as non-breaking. Northwind's policy is stricter because tenant requests must fully describe the accepted surface for a given version.

The evaluator does not trust a comment in the module. Completeness is computed: bindings must use declared tenant parameters; hidden keys must not appear in the request; a breaking change on a single version fails.

Best Practice: Bind each tenant request to a capability version. A request that cannot name its version is still composing primitives.

Production Practice: Isolation is a contract constraint, not a new tenancy model. Chapter 3 already denied `peer-tenant-workload-network`. This chapter loads that `denied_inheritance` list. A tenant-network binding may not reintroduce it.

Inherited interfaces stay tenant-visible

The inherited DevOps release interface promotes by `artifact_digest`. The inherited identity interface is short-lived federated identity. `artifact-digest` belongs in tenant parameters, not in `hidden_module`. Workload identity must keep the federated credential model. Dropping either to make the module "simpler" is the unofficial path from Chapter 5 wearing an infrastructure coat.

Chapter 8 will reconcile these versions. This chapter must not invent a cluster operator so the contract looks applied. Chapter 11 will meter `cluster-capacity-pool`. This chapter must not invent a second pool so storage `class` looks like quota.

3. Publish the contract catalog

The completed Chapter 7 model uses three files:

```
contracts/catalog.yaml
contracts/versions.yaml
contracts/compatibility.yaml
```

The separation is deliberate. The catalog names capabilities, owners, and tenant bindings. Versions name the tenant-visible fields and the hidden module. Compatibility names what is breaking and records changes. The evaluator joins them to Chapter 3 tenants, Chapter 3 isolation, and the inherited release and identity interfaces.

> **Practice — Bind tenants to versions, not to modules**
> Consume Chapter 3 environment ids and keep Terraform addresses out of the request.

Open `contracts/catalog.yaml`. Storefront and Fulfillment bind `storefront-nonprod` and `fulfillment-nonprod` to `tenant-storage 1.0` with `class` and `size-gb`. They do not name `terraform-resource-address`.

> **Practice — Keep inherited identity and artifact-digest on the tenant surface**
> A contract that hides digest promotion or replaces federated identity has left the inherited delivery path.

Open `contracts/versions.yaml`. Workload identity stays `short-lived-federated` with tenant parameter `subject`. Artifact promotion keeps `artifact-digest` as a tenant parameter. Storage hides the Terraform address. Network hides CNI config.

> **Practice — Treat parameter rename as breaking**
> Hidden-module refactors may stay on the same version. Tenant-visible add, remove, and rename may not.

Open `contracts/compatibility.yaml`. Breaking kinds are `tenant-parameter-add`, `tenant-parameter-remove`, and `tenant-parameter-rename`. `hidden-module-refactor` may proceed without a bump. The completed catalog has no in-place breaking change to record.

The lab does not apply a real Terraform or Helm module. Completeness is whether Fulfillment can bind a version whose tenant fields remain stable, and whether a silent rename fails.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-07-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 07 checkpoint: versioned infrastructure contracts and
compatibility verified
```

The audit validates the three Chapter 7 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- required capabilities exist and have living Chapter 1 owners;
- bindings use Chapter 3 tenants and inventoried environment ids;
- binding parameters are declared tenant parameters, not hidden module keys;
- workload identity keeps inherited federated identity;
- artifact promotion keeps inherited `artifact-digest` on the tenant surface;
- tenant-network values are not in Chapter 3 network `denied_inheritance`;
- compatibility names parameter add, remove, and rename as breaking; and

- a breaking change on a single version, or without a migration note, fails.

The expected capabilities, environments, and breaking kinds live in a separate checkpoint file. A contract under test does not emit its own passing grade.

The checkpoint does not prove that a real module created a bucket, that a real identity provider issued a federated subject, or that a real CNI enforced default-deny. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 7 evidence
Mechanism evidence	Schemas and the contract-compatibility evaluator operated successfully.
Decision evidence	Capabilities, tenant parameters, hidden internals, and breaking kinds are explicit.
Outcome evidence	Fulfillment has a computed binding to tenant-storage 1.0 without a Terraform address. That is not proof a bucket exists.
Recovery evidence	Not produced. A denied silent rename is a compatibility-contract failure, not restored tenant isolation after a live module outage.

Chapter 7 produces mechanism, decision, and limited outcome evidence for an infrastructure API inside existing leases. Pretending the local checkpoint proves Terraform apply in a real cloud would weaken Chapter 8's reconciler.

4. Test the design under failure

Independent control failure — A module refactor changes a tenant-visible field without a contract version, breaking fulfillment silently

> **Practice — Fail compatibility before Fulfillment discovers the rename in production**
>
> Inject a `class` → `sku` rename onto the completed storage contract and refuse the in-place break.

The completed contracts are healthy. The failure command does not rewrite them. It injects the silent-refactor case against that snapshot:

```
make chapter-07-failure
```

Expected output:

```
chapter 07 failure: silent tenant-parameter rename correctly rejected
```

The injected record is:

```yaml
capability: tenant-storage
version: "1.0"
tenant_parameters: [sku, size-gb]
changes:
  - capability: tenant-storage
    version: "1.0"
    kind: tenant-parameter-rename
    from_field: class
    to_field: sku
```

Fulfillment's binding still sends `class`. Identity, network isolation, artifact-digest, and Storefront's other bindings stay current. The baseline's leaked Terraform address, dropped federated identity, denied network join, and missing breaking policy are not required for this failure. A module refactor can happen after the catalog already looks complete. Breaking without a version, a missing migration note, and the stale `class` parameter must all fail on their own terms.

Severity: high; every later reconcile, guardrail, and support conversation will apply a contract Fulfillment can no longer satisfy.
Plausible harm: `fulfillment-api` cannot obtain storage; Storefront appears healthy on `sku`; the platform debugs a "tenant misconfiguration" that was a silent API change.
Potential blast radius: every tenant still bound to `tenant-storage 1.0`, plus any workload that assumed `class` remained the request.
Bounded by: Chapter 3 environment ids, Chapter 6 leases, inherited federated identity, and the compatibility policy. None of those repair a version that changed its tenant fields in place.
Primary principles: explicit contracts, blast-radius control, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `obtain-bounded-environment` with storage, identity, and network it can request. Sending a Terraform address, or discovering `sku` only after a module merge, is not that job.
- **Isolation:** The boundary is still Chapter 3's tenant plus this contract constraint: a tenant-network binding must not reintroduce `peer-tenant-workload-network`. This chapter does not invent a new tenancy model. Isolation is applicable as a denied sharing join, not as a recovered live network.
- **Contract and exit:** The contract is the versioned infrastructure API. Teams leave a version by binding a new version with a migration note, not by forking the module. A paved-road exit is a different contract; this chapter does not replace it. An exit is not an exception to digest identity.
- **Platform-product evidence:** A successful Terraform apply is not proof the product works. Required later proof is a tenant request bound to a named version whose compatibility still holds. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the change an internal refactor encourages module controls: rename the field, keep version `1.0`, skip the tenant note. Fulfillment's request still says `class`. Storefront may already speak `sku` because the platform team tested the new module. Compatibility that does not name rename as breaking cannot fail. Support sees a tenant error. The catalog still looks versioned.

The leaked Terraform address makes every module path a tenant API. The dropped federated model makes identity something teams copy again. The denied network value makes Chapter 3's isolation ornamental. The missing bump makes the version string a label, not a contract.

Correction

The completed model does not rename `class` to `sku` on `1.0`. Tenant-visible rename is breaking. It requires another version and a migration note. Bindings that still send `class` fail until they move. Hidden-module refactors may stay on `1.0`. Parameter rename may not. Completeness is computed. The failure command proves the check still fails when only storage fields change and identity stays federated.

This failure is a compatibility-contract failure. It is not a runtime plane recovery. Do not call the denied rename **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live module outage. The product stopped treating an in-place field change as an internal cleanup.

That correction changes later decisions:

- Chapter 8 must reconcile these versions with a tenant-scoped plane subject; it must not apply a hidden field Fulfillment never requested.
- Chapter 9 must bind remaining guardrails to inherited defaults, not to Terraform internals.
- Chapter 12 must treat a contract version bump as a fleet change, not a silent module merge.
- Chapter 13 must debug a failed binding as a product incident when the version moved under the tenant.

The design is practical because the failed compatibility check is the product. Adding an arbitrary Terraform apply would not make it more practical.

5. Production reality

Common infrastructure-contract errors

Publishing the module as the request

`terraform-resource-address` in a tenant binding is the ticket queue. Hide the address; version `class` and `size-gb`.

Renaming a tenant field without a bump

`class` → `sku` on the same version is how Fulfillment breaks silently. Add a version and a migration note.

Treating hidden-module refactors as breaking, or tenant renames as not

The first recreates magic and endless version churn. The second recreates outages. Name both in policy.

Reintroducing `peer-tenant-workload-network` as an isolation parameter

Chapter 3 already denied it. Join that list. Do not hardcode a second copy and hope it stays true.

Dropping federated identity or hiding `artifact-digest`

Those are inherited delivery contracts. They are tenant-visible on purpose.

Building the reconciler in this chapter

Contracts are the API. Chapter 8 operates the plane that applies them. Do not grant cluster-admin so the YAML looks live.

6. What changed

Before	After
Fulfillment sent class after a silent sku rename.	**In-place tenant-parameter rename fails compatibility.**
Storefront bound a Terraform address.	**Hidden module internals are not tenant API.**
Each team composed storage, identity, and network primitives.	**Tenants bind Chapter 3 environments to versioned capabilities.**
Compatibility was a module changelog.	**Add, remove, and rename of tenant parameters are breaking and need a migration note.**
Identity and digest could disappear into the module.	**Contracts consume inherited federated identity and artifact-digest.**
A valid schema could appear to prove Terraform applied.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has an infrastructure contract catalog that later chapters can reconcile, guard, and support without treating a module refactor as the tenant API.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Infrastructure contract catalog	contracts/catalog.yaml and contracts/versions.yaml	They retain capability versions, tenant parameters, and hidden internals later reconcilers must not promote into the tenant API.
Compatibility policy	contracts/compatibility.yaml	It retains the rule that tenant-visible add, remove, and rename require a version and a migration note.

These artifacts should change when Northwind's tenant-visible infrastructure API materially changes—not whenever a module is restyled behind the same version.

What You Learned

An infrastructure contract is a versioned tenant API with hidden modules, compatibility rules, and inherited identity and artifact constraints. A silent field rename on `1.0` is a break, not a cleanup. Schema checks can prove structural completeness within declared scope. They cannot prove a real Terraform or Helm apply. A design earns its place when Fulfillment can bind `tenant-storage 1.0` without owning the module, and when an in-place rename fails.

Prove It

> **Independent Practice — Version a tenant-cache contract without copying the storage rename**
>
> A data-analytics path needs bounded object cache in `storefront-nonprod` and must not receive Storefront's Terraform state or Fulfillment's storage `class`.

Extend the Chapter 7 model without adding a control-plane reconciler yet:

1. Decide whether cache is a new capability or a parameter class of `tenant-storage`.
2. Name tenant parameters and hidden module fields so a cache refactor cannot become the request.
3. State which inherited identity and artifact fields still apply, and which Chapter 3 network value must fail if bound.
4. Choose what is breaking versus `hidden-module-refactor` when cache `tier` is later renamed to `sku`.
5. Identify one observation that would falsify the contract—for example a binding that still sends `tier` after version `1.0` only lists `sku`.
6. Explain which material change would trigger review of the compatibility policy, not just one binding.

Do not copy the storage `class` → `sku` row and rename it. Cache has different durability, eviction, and isolation consequences than block storage. Your durable output is the version-and-compatibility decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 7 capability when you can explain why a module field is not a tenant API, trace a binding to a version, Chapter 3 environment, and inherited interface, describe evidence that would falsify compatibility, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Contracts exist, but the shared reconciler that applies them is still an unofficial cluster operator. A single controller still runs with broad authority and no tenant-scoped subject.

Chapter 8 operates a shared control plane as a product, so reconciliation cannot become shared root.

8. Operate a Shared Control Plane as a Product

Chapter 7 versioned the infrastructure API. Fulfillment can bind `tenant-storage 1.0` without a Terraform address. A single reconciler still applies those versions with broad authority and no tenant-scoped subject. Tickets return as “just let the controller do it.” Cluster-admin returns as a token “so onboarding works.” Chapter 3 already named `kubernetes-control-plane` as shared-control-plane and denied cluster-admin. A reconciler that still holds that token is the cumulative product failure continuing under a friendlier name.

The production question is now:

> How does the shared control plane reconcile tenant intent without becoming shared root?

Control-plane convenience recreates cluster-admin. Tenants cannot tell whether the plane is a product or a privileged script. This chapter records the plane as a product: a tenant-scoped subject, admission of Chapter 7 contract versions, inherited **GitOps (Git-based operations)** state that must not rewrite source, and a failed upgrade that retains last known good. Storefront must not be mutated by a Fulfillment reconcile. The plane must not approve its own change.

1. A reconciler that is still cluster-admin

A weak record says:

```
subject: plane-reconciler
granted_role: cluster-admin
mutated_tenants: [fulfillment, storefront]
approved_by: plane-reconciler
```

It does not identify a Chapter 3 sharing mode, a plane identity distinct from tenant identity, an admitted contract version, inherited GitOps state, or last known good after a failed upgrade. “So onboarding works” may ship `fulfillment-api` this week. Fulfillment intent can rewrite Storefront. The plane can approve its own upgrade and keep running the version that failed.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 8 baseline:

```
make chapter-08-baseline
```

The command succeeds when it detects the intended unsafe plane:

```
chapter 08 baseline: cluster-admin reconciler correctly detected
```

The fixture grants the reconciler `cluster-admin`, lets a Fulfillment reconcile mutate Storefront, lets the plane approve its own upgrade, continues onto version `1.1` after that upgrade failed, rewrites source, and replaces inherited short-lived federated identity with a referenced rotatable secret. Contracts exist. The operator is still shared root.

That residual drives the chapter.

2. The production model: a plane you consume, not a token you inherit

> *Theory — Shared control plane as product*
>
> This model enables Fulfillment to finish `obtain-bounded-environment` and `ship-on-paved-road` by having admitted contract versions reconciled, without inheriting cluster-admin or mutating Storefront.

The plane is `kubernetes-control-plane`, not a new cluster

Chapter 3 already declared the shared surface: `kubernetes-control-plane`, mode `shared-control-plane`, tenants may not inherit `cluster-admin` or `platform-operator`. Chapter 6 already used that id as the environment product's shared plane and as credential audience. This chapter does not rename it to make the reconciler look new.

A **shared-nothing** platform would give each tenant its own plane. Northwind chose shared-control-plane. The reconciler is therefore a product with an owner, forbidden roles, and a subject. Shared cluster-admin is not that product. It is the unofficial path Chapter 3 interrupted, now running continuously.

`dev-cluster-admin` is not a plane role. Chapter 6 already forbade it on leases. The plane product forbids it too.

Reconcile applies admitted intent; rewrite is shared root

Reconcile means: take a Chapter 7 binding that admission allowed, and apply it inside one tenant. Rewrite means: change source, skip admission, or mutate another tenant so onboarding works.

The inherited GitOps interface requires `reviewed_intent`, `immutable_artifact`, `independent_configuration`, and `reconciliation_result`. The controller may not rewrite source. Promoting `latest` is not immutable artifact identity. The inherited release interface still promotes by `artifact_digest`.

The inherited DevSecOps authorization interface already forbids self-approval and requires `subject`, `action`, `resource`, `context`, `policy_identity`, `result`, and `reason`. A plane change allowed by `plane-reconciler` as both actor and approver is self-approval. A living Chapter 1 user must approve plane upgrades. The reconciler must not.

Best Practice: Split plane identity from tenant identity. A tenant subject applies tenant work. A plane subject applies admitted versions and cannot approve its own change.

Production Practice: Isolation is a plane invariant joined to Chapter 3. This chapter loads `tenancy/sharing.yaml` and change-authority `denied_inheritance`. A plane subject with `cluster-admin` fails. A reconcile whose `mutated_tenants` includes another tenant fails.

A failed upgrade retains last known well

Last known good is the plane version still running after an upgrade fails. It is not a backup of tenant data. Chapter 14 will recover the plane against this retention. This chapter must already refuse to treat a failed upgrade as a successful move to `1.1`. Completeness is computed. The plane cannot emit `status: healthy` to hide a missing last known good.

Do not build quota floors here. Do not bind guardrail exceptions here. Do not restore from mixed backups here.

3. Operate the plane as a product

The completed Chapter 8 model uses four files:

```
control-plane/product.yaml
control-plane/subjects.yaml
control-plane/admission.yaml
control-plane/reconciliation.yaml
```

The separation is deliberate. The product names the plane id, jobs, federated identity, forbidden roles, and the GitOps rewrite prohibition. Subjects split plane identity from tenant identity. Admission names Chapter 7 versions and authorization decisions. Reconciliation records tenant applies and plane upgrades. The evaluator joins them to Chapter 3 tenants, isolation, sharing, and roles, Chapter 6's environment product, Chapter 7 versions, and the inherited GitOps, identity, release, and authorization interfaces.

> **Practice — Bind the plane to the Chapter 3 sharing id and inherited identity**
>
> Consume `kubernetes-control-plane` and short-lived federated identity rather than inventing a controller token.

Open `control-plane/product.yaml`:

```yaml
owner: platform-team
plane: kubernetes-control-plane
jobs: [obtain-bounded-environment, ship-on-paved-road]
credential_model: short-lived-federated
forbidden_roles: [cluster-admin, dev-cluster-admin]
controller_may_rewrite_source: false
sharing_mode: shared-control-plane
```

The evaluator loads `inherited/devops-v1.1/identity/interface.yaml` and fails the product if federated identity is replaced by a referenced rotatable secret. It loads the GitOps interface and fails if the controller may rewrite source. It fails if the plane id is renamed.

> **Practice — Give the reconciler platform-operator, never cluster-admin**
> Tenants still must not inherit `platform-operator`. That is the plane role, not a tenant grant.

Open `control-plane/subjects.yaml`. The plane subject is `plane-reconciler` with `granted_role: platform-operator` and `may_approve_plane_change: false`. Storefront and Fulfillment keep `tenant-operator`. Audience is `kubernetes-control-plane`. Policy on the plane is `plane-scoped`, not `cluster-admin`.

> **Practice — Admit Chapter 7 versions and deny plane self-approval**
> A reconcile of an unknown version is not a product. A plane that allows its own upgrade is not reviewed.

Open `control-plane/admission.yaml` and `control-plane/reconciliation.yaml`. Admitted versions are the Chapter 7 `1.0` contracts. Fulfillment's storage reconcile mutates `[fulfillment]` only. The failed upgrade from `1.0` to `1.1` is requested by the reconciler, approved by `platform-team`, and retains `last_known_good: "1.0"` as `current_version`.
The lab does not run a real Kubernetes control plane. Completeness is whether the reconciler stays a product: tenant-scoped apply, admitted versions, no source rewrite, no self-approval, last known good after failure.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-08-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 08 checkpoint: tenant-scoped plane subjects and last known good
verified
```

The audit validates the four Chapter 8 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- the product names `kubernetes-control-plane` and inherited federated identity;
- sharing mode is `shared-control-plane` from Chapter 3;
- required jobs remain `obtain-bounded-environment` and `ship-on-paved-road`;
- the plane subject exists, is not tenant-scoped, and is not `cluster-admin` or `dev-cluster-admin`;
- tenant subjects do not inherit change-authority denials from Chapter 3;
- admitted versions exist in Chapter 7;
- authorization decisions carry inherited fields and the plane cannot allow its own upgrade;
- reconciles bind Chapter 3 environment ids and mutate only their tenant;
- GitOps required state is present and source is not rewritten;
- immutable artifact is not `latest` when promotion identity is digest; and
- a failed upgrade retains last known good as the current version.

The expected plane id, jobs, and forbidden roles live in a separate checkpoint file. A plane under test does not emit its own passing health.

The checkpoint does not prove that a real apiserver admitted a contract, that a real controller applied an object, or that a real upgrade rolled back. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 8 evidence
Mechanism evidence	Schemas and the admission-and-reconcile evaluator operated successfully.
Decision evidence	Plane id, subjects, forbidden roles, admitted versions, and last known good are explicit.
Outcome evidence	Fulfillment has a computed reconcile for fulfillment-nonprod that does not mutate Storefront. That is not proof a live object exists.
Recovery evidence	Not produced. A denied cluster-admin reconcile is a plane-authority invariant, not restored tenant isolation after a live rewrite. Last known good is retained version evidence, not Chapter 14 restore.

Chapter 8 produces mechanism, decision, and limited outcome evidence for a shared plane product. Pretending the local checkpoint proves kube-apiserver behavior would weaken Chapter 12's fleet upgrades and Chapter 14's bounded recovery.

4. Test the design under failure

Cumulative product failure — The reconciler uses a cluster-admin token "so onboarding works," allowing one tenant intent to mutate another

> **Practice — Deny shared plane-admin and cross-tenant reconcile**
>
> Inject cluster-admin onto the completed plane subject and refuse a Fulfillment apply that mutates Storefront.

The completed plane is healthy. The failure command does not rewrite it. It injects the residual cluster-admin case against that snapshot:

```
make chapter-08-failure
```

Expected output:

```
chapter 08 failure: cluster-admin cross-tenant reconcile correctly
rejected
```

The injected record is:

```
id: plane-reconciler
granted_role: cluster-admin
# fulfillment-nonprod-storage
mutated_tenants: [fulfillment, storefront]
```

Federated identity, GitOps rewrite prohibition, admitted Chapter 7 versions, self-approval denial, and last known good stay current. The baseline's missing last known good, source rewrite, and secret-model drop are not required for this failure. Shared plane-admin can appear after the product already looks complete. Cluster-admin on the reconciler and cross-tenant mutate must both fail on their own terms.

Severity: high; every later guardrail, fleet, support, and recovery conversation will optimize a shared-root controller instead of a plane product.
Plausible harm: Fulfillment onboarding rewrites Storefront storage; the order path moves with a warehouse change; the plane then upgrades itself.
Potential blast radius: both application tenants and `kubernetes-control-plane`; Storefront's order path is no longer a separate failure domain.
Bounded by: Chapter 3 sharing and change-authority denials, Chapter 6 leases, Chapter 7 admitted versions, and inherited GitOps and authorization. None of those repair a reconciler that still holds `cluster-admin`.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `obtain-bounded-environment` and `ship-on-paved-road`. A cluster-admin token so onboarding works is not those jobs; it is shared authority substituting for a plane subject.
- **Isolation:** The boundary is Chapter 3's tenant plus this plane subject: `kubernetes-control-plane` as shared-control-plane, `platform-operator` that tenants may not inherit, no `cluster-admin`, and a reconcile that cannot list another tenant in `mutated_tenants`. Fulfillment's blast radius must stop at Fulfillment.
- **Contract and exit:** The contract is the plane product, admitted Chapter 7 versions, and last known good. Teams do not leave the plane the way they leave a paved-road scaffold. They consume it without inheriting it. A plane upgrade is a reviewed change, not a tenant exit and not an exception to digest identity.
- **Platform-product evidence:** A running controller is not proof the product works. Required later proof is tenant-scoped reconcile plus retained last known good. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the token “so onboarding works” encourages the same unofficial path Chapter 3 interrupted and Chapter 6 renamed `dev-cluster-admin`: Fulfillment is blocked, someone shares change authority, Storefront moves, and the order path becomes a noisy neighbor. Versioned contracts then apply through shared root. Self-approval makes the upgrade path ornamental. Missing last known good makes failure a successful move. Source rewrite makes GitOps a label on a privileged script.

The missing plane/tenant identity split makes Chapter 3’s `platform-operator` a second cluster-admin. The missing admission join makes Chapter 7’s versions documentation. Shared plane-admin makes shared-control-plane a slogan.

Correction

The completed model does not grant the reconciler `cluster-admin`. The plane subject is `platform-operator` with federated, plane-scoped claims. Fulfillment’s reconcile mutates Fulfillment. Storefront is not in `mutated_tenants`. Plane upgrades are approved by `platform-team`, not by `plane-reconciler`. A failed upgrade keeps version `1.0` as last known good. Source is not rewritten. Isolation is recorded as a plane invariant.

This failure is a plane-authority invariant. It is not a runtime plane recovery. Do not call the denied reconcile **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live rewrite. The product stopped treating cluster-admin as onboarding.

That correction changes later decisions:

- Chapter 9 must enforce defaults on this plane without granting the reconciler cluster-admin to “make the webhook stick.”
- Chapter 12 must upgrade the plane in cohorts against last known good, not by patching shared root in place.
- Chapter 13 must reject unofficial plane-admin the same way this chapter rejects the onboarding token.
- Chapter 14 must recover `kubernetes-control-plane` from last known good without replaying one tenant’s reconcile into another.

The design is practical because the denied cross-tenant apply is the product. Adding an arbitrary kubectl command would not make it more practical.

5. Production reality

Common control-plane errors

Granting cluster-admin so onboarding works

Onboarding is a tenant job. Cluster-admin is shared root. They are not a bargain.

Letting the plane approve its own change

Inherited authorization already forbids self-approval. A living owner must admit plane upgrades.

Treating a failed upgrade as the new current version

Last known good is the version still running. `to_version` is intent, not fact.

Rewriting Git source from the controller

The inherited GitOps interface forbids it. Reconcile applies reviewed intent. It does not edit the request.

Using a tenant subject as the plane, or a plane subject as a tenant

Split identity. Audience stays `kubernetes-control-plane`. Policy does not.

Building quota, guardrails, or restore in this chapter

Admit versions and retain last known good. Leave floors to Chapter 11, exceptions to Chapter 9, and mixed-backup restore to Chapter 14.

6. What changed

Before	After
The reconciler used cluster-admin so onboarding worked.	**Shared plane-admin and cross-tenant reconcile fail the plane invariant.**
One controller mutated whichever tenant was convenient.	**Each reconcile mutates only its Chapter 3 tenant.**
Plane and tenant identity were the same token.	**Plane subject is platform-operator; tenants remain tenant-operator.**
Contract versions were documentation.	**Admission joins Chapter 7; unknown versions cannot apply.**

Before	After
The plane approved its own upgrade and kept the failed version.	**A living owner approves; failed upgrades retain last known good.**
A valid schema could appear to prove kube-apiserver.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a control-plane product that later chapters can guard, upgrade, support, and recover without treating cluster-admin as reconciliation.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Control-plane product contract	control-plane/product.yaml	It retains the plane id, jobs, federated identity, forbidden roles, and GitOps rewrite prohibition later fleet and recovery chapters must not rename.
Tenant-scoped authority map	control-plane/subjects.yaml	It retains the split between plane-operator and tenant-operator, and the rule that the plane cannot approve its own change.

These artifacts should change when Northwind's shared plane, tenant set, or inherited GitOps model materially change—not whenever a controller image tag is restyled.

What You Learned

A shared control plane is a product with a tenant-scoped subject, admitted contract versions, and last known good. A cluster-admin token so onboarding works is shared root. Schema checks can prove structural completeness within declared scope. They cannot prove a real Kubernetes control plane. A design earns its place when Fulfillment's reconcile cannot mutate Storefront, and when a failed plane upgrade still runs the last known good version.

Prove It

> **Independent Practice — Scope a read-only analytics reconciler without copying cluster-admin**
>
> A data-analytics path needs the plane to apply a read contract in `storefront-nonprod` and must not mutate Fulfillment or approve plane upgrades.

Extend the Chapter 8 model without adding guardrail exceptions yet:

1. Decide whether analytics uses the existing `plane-reconciler` with a narrower admit list, or a second plane subject that still is not `cluster-admin`.

2. Name which Chapter 7 versions that subject may reconcile, and which tenant must be absent from `mutated_tenants`.
3. State which inherited GitOps fields still apply, and which authorization action must deny if the subject is the approver.
4. Choose last known good if this subject's apply fails—plane version, tenant binding, or both.
5. Identify one observation that would falsify isolation—for example `mutated_tenants: [storefront, fulfillment]` on an analytics reconcile.
6. Explain which material change would trigger review of the plane product, not just one reconcile.

Do not copy the Fulfillment cluster-admin row and rename it. Read-only analytics has different admission, mutate, and upgrade consequences than a shipping reconciler. Your durable output is the subject-and-admission decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 8 capability when you can explain why a cluster-admin reconciler is still shared root, trace a reconcile to a Chapter 3 tenant, a Chapter 7 version, and inherited GitOps state, describe evidence that would falsify last known good, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

The plane is a product, but defaults either trap teams or can be switched off quietly. Security and delivery defaults from earlier books are either mandatory with no exit or optional with no owner.

Chapter 9 enforces guardrails without a golden cage, so remaining defaults stay owned and exceptions stay temporary.

9. Enforce Guardrails Without a Golden Cage

Chapter 8 scoped the reconciler. The plane is a product. Delivery and security defaults from earlier books are still either mandatory with no exit or optional with no owner. A cage drives unofficial forks. Missing defaults recreate the copy-paste path Chapter 5 already failed. Chapter 5 named remaining guardrails: `artifact-digest`, `workload-identity-claims`, and `no-cluster-admin`. An exit is not an exception to digest identity. This chapter must bind those defaults, not invent a third naming scheme.

The production question is now:

> Which defaults must the platform enforce, and how do exceptions stay temporary without trapping teams?

A golden cage blocks the Chapter 5 exit. A scorecard that stays green after an inherited exception expires, while Fulfillment disables digest pinning, is not a road. This chapter records owned defaults, scorecards that cannot emit their own grade, and exception rows that only bind an inherited DevSecOps exception ID. Owner, scope, compensation, and expiry stay on the inherited record. The platform adds tenant, remaining isolation, and scorecard effect.

1. A green scorecard after the exception expired

A weak record says:

```
system: fulfillment-api
defaults_present: [workload-identity-claims, no-cluster-admin]
reported_status: green
exception: exception-dependency-mirror-2026-08
owner: platform-security
expires_at: "2026-12-01T00:00:00Z"
```

It does not identify Chapter 5 remaining guardrails as the default set, a living platform owner, an exit that is still allowed, or expiry resolved from the inherited exception. Copying `expires_at` onto the platform row lets someone extend a date without renewing the DevSecOps exception. Forbidding exits turns the paved road into a cage. “Green” may describe a portal badge. It cannot survive an expired exception and a disabled digest.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 9 baseline:

```
make chapter-09-baseline
```

The command succeeds when it detects the intended unsafe guardrails:

```
chapter 09 baseline: expired exception green scorecard correctly detected
```

The fixture forbids exits, copies owner and expiry onto the binding, leaves `exception-dependency-mirror-2026-08` expired against `as_of`, lets Fulfillment drop `artifact-digest`, and still reports green. The plane is scoped. The defaults are not a product.

That substitution drives the chapter.

2. The production model: defaults you can exit, exceptions you cannot copy

> *Theory — Guardrail and exception binding*
>
> This model enables Fulfillment to finish `ship-on-paved-road` with inherited digest and identity defaults enforced, and Storefront to keep the Chapter 5 exit, without treating a scorecard badge as policy.

A guardrail is a default with an owner, not a cage

The defaults this chapter enforces are the Chapter 5 remaining guardrails:

Default	Why it exists	Enforcement surface
artifact-digest	Inherited DevOps promotion identity	artifact-promotion
workload-identity-claims	Inherited federated identity claims	workload-identity
no-cluster-admin	Chapter 3 / Chapter 8 prohibited inheritance	kubernetes-control-plane

They apply on the paved road and on a supported exit. They do not apply by trapping teams on the scaffold. `exits_allowed: true` keeps the Chapter 5 exit a road. `exits_allowed: false` is a golden cage.

Do not rename these ids. Chapter 9 binds them. It does not invent `digest-pinned-artifact` as a default, or treat `golden-template-scaffold` as a remaining guardrail. An exit may lose the template. It may not lose digest identity.

The inherited control catalog may be exposed. It may not be rebuilt. Threat-model rebuild remains a Platform Chapter 1 non-goal.

An exception is a binding, not a second lifecycle

Leaving a contract, leaving a scaffold, and waiving a default are three different acts:

Act	Chapter	What it is
Supported exit	5	Leave the scaffold; remaining guardrails stay
Contract version bump	7	Leave a tenant-visible API version with a migration note
Guardrail exception	9	Temporary deviation from a default; expiry lives on the inherited DevSecOps record

An exception is not an exit. An exit is not an exception to `artifact-digest`. A version bump is not a scorecard waiver.

Each `guardrails/exceptions.yaml` row references an inherited DevSecOps exception ID. It must not copy owner, scope, compensation, or expiry. The evaluator resolves those fields from `inherited/devsecops-v1.0/exceptions/records.yaml`. The platform row adds tenant, system, paved or exit path, remaining isolation, and scorecard effect. `duplicate_lifecycle_fields_in_platform_bindings` is false. Completeness is computed.

Best Practice: Bind remaining isolation on the exception. Waiving digest does not waive `no-cluster-admin` or workload-identity claims.

Production Practice: A scorecard cannot emit `reported_status: green`. The inherited evidence map requires an independent producer. Expiry is the inherited `expires_at` compared to `as_of`, not a date the platform row restates.

Expired means the waiver is gone

`exception-dependency-mirror-2026-08` expired at `2026-08-15T12:00:00Z`. Northwind's `as_of` is `2026-08-16T12:00:00Z`. Status `active` on the inherited record does not keep it current. A scorecard that still waives `artifact-digest` after that instant fails. Renew the inherited exception, or restore the default. Do not copy a later `expires_at` onto the platform binding.

An active inherited exception may waive digest while remaining isolation holds. That is a reviewed temporary deviation. It is not a paved-road exit and not a contract version.

Do not deploy a real admission webhook here. Do not build Chapter 10 measurements here.

3. Publish defaults, scorecards, and bindings

The completed Chapter 9 model uses three files:

```
guardrails/defaults.yaml
guardrails/scorecards.yaml
guardrails/exceptions.yaml
```

The separation is deliberate. Defaults name the Chapter 5 remaining guardrails, owners, enforcement surfaces, and that exits remain allowed. Scorecards record which defaults each catalog system currently presents. Exception rows only bind. The evaluator joins them to Chapter 5's contract and exit, Chapter 3 tenants, the catalog, inherited exception records, and the inherited release, identity, control, and evidence interfaces.

> **Practice — Enforce the remaining guardrails on paved and exit paths**
>
> Keep `artifact-digest`, `workload-identity-claims`, and `no-cluster-admin`. Allow the Chapter 5 exit.

Open `guardrails/defaults.yaml`:

```yaml
owner: platform-team
exits_allowed: true
defaults:
  - id: artifact-digest
    owner: platform-team
    enforcement_point: artifact-promotion
    inherited_policy: devops-v1.1-release
    applies_to: [paved, exit]
```

The evaluator loads the inherited release interface and fails if `artifact-digest` is missing while promotion identity is digest. It loads identity and fails if workload-identity claims are missing. It fails if exits are forbidden while Chapter 5 still has a supported exit.

> **Practice — Record observations, not a self-emitted grade**
>
> `defaults_present` is an observation. Completeness is computed. Green is not a field the scorecard may use to pass.

Open `guardrails/scorecards.yaml`. `fulfillment-api` is paved with all three defaults. `notification-service` is the Chapter 5 exit and still presents the remaining guardrails. There is no `reported_status`.

> **Practice — Bind inherited exception IDs without copying lifecycle**
>
> Tenant, remaining isolation, and scorecard effect are platform-local. Owner, scope, compensation, and expiry are not.

Open `guardrails/exceptions.yaml`. The completed product has no active waiver. Bindings is an empty list. That is valid. A later binding must name `exception-dependency-mirror-2026-08` or another inherited id, not a platform-invented exception.

The lab does not deploy an admission webhook. Completeness is whether defaults stay named, exits stay allowed, bindings stay bindings, and an expired inherited exception cannot keep a green digest-off scorecard.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-09-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 09 checkpoint: guardrail defaults and exception bindings verified
```

The audit validates the three Chapter 9 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- required defaults are the Chapter 5 remaining guardrails and have living owners;
- `artifact-digest` and workload-identity claims still consume inherited interfaces;
- remaining guardrails apply on exit; exits stay allowed;
- scorecards exist for required catalog systems and bind Chapter 3 tenants;
- the Chapter 5 exit still presents remaining guardrails;
- exception rows reference inherited IDs and do not copy lifecycle fields;
- expiry is resolved from the inherited record against `as_of`; and
- `reported_status: green` fails because the producer is not independent.

The expected defaults, systems, and `as_of` live in a separate checkpoint file. A scorecard under test does not emit its own passing grade.

The checkpoint does not prove that a real webhook blocked `latest`, that a real identity provider issued claims, or that a real exception ticket was renewed. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 9 evidence
Mechanism evidence	Schemas and the exception-binding evaluator operated successfully.
Decision evidence	Defaults, exit permission, remaining isolation, and inherited references are explicit.
Outcome evidence	Fulfillment has a computed scorecard with digest present and no expired waiver. That is not proof a webhook fired.
Recovery evidence	Not produced. A failed green scorecard is a conformance invariant, not restored isolation after a live policy bypass.

Chapter 9 produces mechanism, decision, and limited outcome evidence for guardrails on the paved road and the supported exit. Pretending the local checkpoint proves admission in a cluster would weaken Chapter 12's fleet change.

4. Test the design under failure

Independent control failure — A scorecard stays green after an exception expires and a tenant disables artifact digest pinning

> **Practice — Fail the scorecard when expiry lapses and digest is off**
>
> Inject an expired inherited exception and a green Fulfillment scorecard without `artifact-digest`, and refuse both.

The completed guardrails are healthy. The failure command does not rewrite them. It injects the expired-waiver case against that snapshot:

```
make chapter-09-failure
```

Expected output:

```
chapter 09 failure: expired exception and disabled digest correctly
rejected
```

The injected record is:

```
system: fulfillment-api
defaults_present: [workload-identity-claims, no-cluster-admin]
reported_status: green
exception: exception-dependency-mirror-2026-08
scorecard_effect: waive-artifact-digest
```

The binding does not copy owner or expiry. Identity, `no-cluster-admin`, and the Chapter 5 exit stay current. The baseline's golden cage and copied lifecycle fields are not required for this failure. An exception can expire after the scorecard already looks complete. Green status, inherited expiry, and the missing digest must all fail on their own terms.

Severity: high; every later measurement and fleet conversation will treat a green badge as digest pinning.
Plausible harm: `fulfillment-api` promotes `latest` after the mirror exception lapses; Storefront's path still looks compliant; support debugs a "tenant choice."
Potential blast radius: every consumer of `fulfillment-api` artifacts, plus any scorecard that trusted this waiver as current.
Bounded by: Chapter 5 remaining guardrails, inherited exception expiry, and independent evidence production. None of those repair a scorecard allowed to stay green.
Primary principles: explicit contracts, trustworthy evidence, blast-radius control.

Platform questions

- **User and job:** Fulfillment must finish `ship-on-paved-road` with digest identity intact. A green badge after disabling pinning is not that job. Storefront must still be able to use the Chapter 5 exit.
- **Isolation:** Guardrails are not a new tenancy model. Chapter-local implication: remaining isolation on an exception must keep `no-cluster-admin`. Waiving digest does not grant plane-admin. Isolation is applicable as a remaining-guardrail constraint, not as recovered live network.
- **Contract and exit:** The contract is the default set plus exception bindings. Teams leave the scaffold through Chapter 5's exit, not through an expired waiver. They leave a contract version through Chapter 7. An exception is a temporary deviation with inherited expiry, not an exit and not a version bump.
- **Platform-product evidence:** `reported_status: green` is a self-emitted claim. Required proof is computed presence of remaining guardrails against inherited expiry. This is not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling the scorecard green because the portal rendered encourages wiki controls: keep the badge, copy a new expiry, skip digest. After `2026-08-15T12:00:00Z`, the inherited mirror exception cannot waive pinning. Fulfillment's `defaults_present` without `artifact-digest` is a disabled default. Copying `expires_at` onto the platform row would hide that. Forbidding exits would hide the other failure: teams who cannot leave the scaffold will disable defaults unofficially.

The missing independent producer makes Chapter 4's green-catalog failure again. The missing inherited join makes DevSecOps expiry ornamental. The missing remaining-guardrail names make Chapter 5's exit a slogan.

Correction

The completed model does not let `fulfillment-api` report green without `artifact-digest`. It does not copy exception lifecycle onto the platform row. Expiry is resolved from the inherited record. A current inherited exception may waive digest with remaining isolation named. An expired one may not. Exits stay allowed. Completeness is computed. The failure command proves the check still fails when only Fulfillment's digest and a binding are swapped.

This failure is a conformance-contract failure. It is not a runtime plane recovery. Do not call the failed scorecard **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live bypass. The product stopped treating an expired waiver as a green default.

That correction changes later decisions:

- Chapter 10 must not count green scorecards or portal adoption as job completion.
- Chapter 12 must not fleet-upgrade a cohort whose remaining guardrails are waived by an expired exception.
- Chapter 13 must escalate a green digest-off scorecard as a product incident, not a tenant preference.
- Support may not renew expiry by editing `guardrails/exceptions.yaml`.

The design is practical because the failed scorecard is the product. Adding an arbitrary webhook command would not make it more practical.

5. Production reality

Common guardrail errors

Copying owner, scope, compensation, or expiry onto the platform row

Those fields already live on the inherited exception. A local copy will drift. Bind the id.

Treating an exception as an exit, or an exit as an exception

Exits leave the scaffold. Exceptions waive a default temporarily. Digest identity is remaining on both.

Forbidding exits so defaults “always apply”

That is a golden cage. Unofficial forks return. Keep `exits_allowed`.

Letting the scorecard emit green

Independent evidence production forbids it. Observe `defaults_present`. Compute the rest.

Waiving `no-cluster-admin` to make a webhook stick

Chapter 8 already refused cluster-admin as onboarding. A guardrail exception is not a plane token.

Rebuilding the DevSecOps control catalog here

The platform may expose inherited controls. It may not rebuild threat models.

6. What changed

Before	After
Fulfillment dropped digest pinning and stayed green after expiry.	**Expired inherited exceptions cannot waive artifact-digest; green fails.**
Exception rows copied owner and expiry.	**Bindings reference an inherited ID; lifecycle is resolved, not copied.**
Defaults were mandatory with no exit, or optional with no owner.	**Chapter 5 remaining guardrails are owned and apply on paved and exit paths.**
An exit, an exception, and a version bump were the same idea.	**Each is a distinct leaving act with a distinct record.**
A valid schema could appear to prove a webhook.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has a guardrail catalog and an exception-binding contract that later measurement, fleet, and support chapters can trust without treating a green badge as digest pinning.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Guardrail catalog	guardrails/defaults.yaml	It retains Chapter 5 remaining guardrails, owners, and exit permission later fleet change must not rename.
Exception-binding contract	guardrails/exceptions.yaml	It retains the rule that platform rows reference inherited exception IDs and do not copy lifecycle.

These artifacts should change when Northwind’s remaining guardrails or inherited exception set materially change—not whenever a scorecard theme is restyled.

What You Learned

A guardrail is an owned default that applies on the paved road and on a supported exit. An exception is a binding to an inherited DevSecOps record, not a second expiry calendar. A green scorecard after digest pinning is disabled and the exception has expired is a failed product. Schema checks can prove structural completeness within declared scope. They cannot prove a real webhook. A design earns its place when Fulfillment cannot stay green without `artifact-digest`, and when Storefront can still leave the scaffold.

Prove It

> **Independent Practice — Bind a Storefront exception without copying the Fulfillment expiry**
>
> `notification-service` already has a Chapter 5 exit. A temporary digest waiver, if it exists at all, must not turn that exit into an unofficial fork.

Extend the Chapter 9 model without adding DevEx measurements yet:

1. Decide whether Storefront needs an exception binding at all, or whether the exit already covers the lost template.
2. If you bind, name the inherited exception ID and the remaining isolation that must still include `no-cluster-admin`.
3. State which field you must not copy, and which inherited timestamp would falsify "the waiver is current."
4. Choose whether `reported_status: green` may exist on `notification-service` during the waiver.
5. Identify one observation that would collapse exit and exception—for example putting `artifact-digest` in `lost_defaults`.
6. Explain which material change would trigger review of the guardrail catalog, not just one binding.

Do not copy the Fulfillment expired-mirror row and rename it. An exited notification service has different durability and digest consequences than a paved `fulfillment-api`. Your durable output is the binding-or-exit decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 9 capability when you can explain why a green scorecard is not a current exception, trace defaults to Chapter 5 remaining guardrails and inherited interfaces, describe evidence that would falsify expiry, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Guardrails exist, but success is still counted in portal clicks and survey smiles. Leadership asks for adoption percentage, CSAT, and ticket volume as proof the platform works.

Chapter 10 measures developer experience without vanity metrics, so job-completion time stays the product indicator.

10. Measure Developer Experience Without Vanity Metrics

Chapter 9 bound remaining guardrails. Scorecards exist. Leadership still asks for adoption percentage, **CSAT (Customer Satisfaction)**, and ticket volume as proof the platform works. A green scorecard and a smiling survey can hide Fulfillment waiting. Chapter 1 already refused `portal-launch` as success evidence and named later proofs: `time-to-first-environment`, `paved-road-completion`, and `catalog-freshness`. Chapter 6 said a created namespace is not that proof. This chapter makes the measurement contract explicit.

The production question is now:

> Which measurements prove teams finish jobs, and which measurements only prove the platform looked busy?

Vanity metrics reward forcing teams onto the road and hiding wait time in Slack. **Goodhart's law** is the pressure: when adoption becomes the target, deleting the unofficial path produces 100% adoption while time-to-first-environment gets worse. That is not developer experience. It is a busier-looking cage.

1. One hundred percent adoption and a longer wait

A weak record says:

```
indicators: [adoption-percentage, order_success_ratio]
samples:
  - indicator: adoption-percentage
    value: 100
    unofficial_paths_deleted: true
  - indicator: time-to-first-environment
    value: 120
    prior_value: 48
    unit: hours
```

It does not identify Chapter 1 jobs, later proofs as owned indicators, samples for those proofs, or an explicit non-metric register. "Everyone is on the road" may be true after the unofficial path is deleted. Fulfillment's wait moved from 48 hours to 120. Storefront's order-success ratio can stay green through that wait. The inherited observability contract already tracks `order_success_ratio` and `order_latency`. Those are tenant workload outcomes. They are not platform-product **SLIs (Service Level Indicators)**, and they are not portfolio **SLOs (Service Level Objectives)**.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 10 baseline:

```
make chapter-10-baseline
```

The command succeeds when it detects the intended unsafe measurement:

```
chapter 10 baseline: adoption vanity and missing job samples correctly
detected
```

The fixture treats adoption and Storefront order-success as product indicators, omits a catalog-freshness sample, leaves adoption off the non-metric register, and records 100% adoption after unofficial paths are deleted while time-to-first-environment worsens from 48 hours to 120. Guardrails exist. Success is still a portal smile.

That substitution drives the chapter.

2. The production model: job time is the product indicator

> *Theory — Developer-experience measurement*
>
> This model enables Northwind to treat finished user jobs as product evidence and to refuse metrics that only prove the platform looked busy.

A platform-product SLI is a job proof, not a portfolio SLO

Chapter 1's later proofs are the retained indicators:

Indicator	Job	Evidence kind
time-to-first-environment	obtain-bounded-environment	lagging
paved-road-completion	ship-on-paved-road	lagging
catalog-freshness	publish-owned-path	lagging

They are platform-product SLIs. They are not portfolio SLOs. Portfolio SLO governance remains a Platform Chapter 1 non-goal with remaining owner `reliability-program`. Do not put `class: portfolio-slo` on a job proof to make it look like SRE.

Lagging evidence is that the job finished, and how long it took. Leading evidence—self-service rate, request-to-lease time—can sit beside those proofs. It cannot replace them. A leading dashboard with no sample for `time-to-first-environment` is still vanity.

Vanity is recorded, not used

Adoption percentage, CSAT, ticket volume, and portal launch are forbidden as indicators. They are required as non-metrics. Forcing every team onto the paved road by deleting unofficial paths will drive adoption to 100% and can make wait time worse. CSAT can rise while Fulfillment still files tickets in Slack. Ticket volume can fall because people stopped asking. A portal launch is Chapter 1's already-rejected proof.

The inherited DevOps observability interface names `order_success_ratio` and `order_latency`. This chapter loads that list. Those outcomes stay Storefront's workload evidence. They must appear in the non-metric register so nobody can "borrow" a green order path as platform health.

Best Practice: Map every retained indicator to a Chapter 1 job and to that job's `later_proof`. A metric without a job is a dashboard.

Production Practice: Missing samples fail the contract. Completeness is computed. A measurement file cannot emit `status: healthy`. The lab does not survey real developers.

Production Practice: The non-metric register is one list with two exclusion kinds. `category: vanity` means the signal is gameable. `category: tenant-workload` means the signal is real and belongs to a tenant path. Northwind records both so a later debugger can tell "stop treating adoption as success" from "stop borrowing Storefront's order-success."

Deleting the unofficial path is not completion

The independent failure is Goodhart in one number: adoption 100, unofficial paths deleted, time-to-first-environment 120 hours after 48. The unofficial path was evidence that the product was slow. Deleting it without improving job time hides the wait. Restore job-time evidence as the product indicator. Keep adoption on the non-metric register.

Do not allocate quota here. Chapter 11 will use these indicators so a cheaper shared bill cannot replace Fulfillment's wait.

3. Adopt the measurement contract

The completed Chapter 10 model uses four files:

```
devex/contract.yaml
devex/indicators.yaml
devex/non-metrics.yaml
devex/samples.yaml
```

The separation is deliberate. The contract names retained indicators and non-metrics. Indicators bind jobs, owners, and SLI class. Non-metrics name vanity and tenant-workload outcomes the product refuses. Samples are observations. The evaluator joins them to Chapter 1's brief, jobs, and non-goals, and to the inherited observability interface.

> **Practice — Keep Chapter 1 later proofs as the indicator set**
>
> Do not replace `time-to-first-environment` with adoption, CSAT, or Storefront order-success.

Open `devex/contract.yaml` and `devex/indicators.yaml`. The three later proofs are the indicators. Each row names a Chapter 1 job, `platform-team` as owner, class `platform-product-sli`, and evidence kind `lagging`.

> **Practice — Register vanity and tenant outcomes as non-metrics**
>
> Tag each refusal. Vanity is gameable. Inherited order outcomes are the wrong layer.

Open `devex/non-metrics.yaml`. The inherited order indicators are there on purpose. Refusing them is a decision, not an omission. The category says which failure mode a later debugger hit:

```
- id: adoption-percentage
  category: vanity
  reason: forcing-teams-onto-the-road
- id: order_success_ratio
  category: tenant-workload
  reason: tenant-workload-not-platform-product
```

`reason` is the row-specific why. `category` is the kind. Tagging adoption as `tenant-workload`, or Storefront order-success as `vanity`, fails. They are not interchangeable refusals.

> **Practice — Attach a sample to every retained indicator**
>
> Fulfillment's time-to-first-environment is 48 hours. That number is an observation. It is not proof a real cluster provisioned a namespace.

Open `devex/samples.yaml`. `paved-road-completion` records one Fulfillment completion. `catalog-freshness` records current entries. There is no adoption sample, because adoption is not an indicator.

The lab does not survey developers and does not scrape a portal. Completeness is whether job proofs have owners, samples, and a non-metric register that includes vanity and inherited tenant outcomes.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-10-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 10 checkpoint: job-completion indicators and non-metrics verified
```

The audit validates the four Chapter 10 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- contract owner is a living Chapter 1 user;
- every Chapter 1 `later_proof` and brief `success_evidence` is a retained indicator with a sample;
- each indicator maps to the job that named that proof;
- vanity ids are non-metrics with category `vanity`, not indicators;
- inherited `outcome_indicators` are non-metrics with category `tenant-workload`, not indicators;
- no indicator uses class `portfolio-slo`; and
- 100% adoption after deleting unofficial paths cannot hide a worse time-to-first-environment.

The expected vanity list lives in a separate checkpoint file. A measurement under test does not emit its own passing grade.

The checkpoint does not prove that 48 hours is the right wait for a real company, that developers are satisfied, or that a real clock measured provisioning. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 10 evidence
Mechanism evidence	Schemas and the measurement-contract evaluator operated successfully.
Decision evidence	Retained indicators, non-metrics with exclusion category, SLI class, and job mappings are explicit.
Outcome evidence	Fulfillment has a computed time-to-first-environment sample. That is not proof a developer finished in 48 hours.
Recovery evidence	Not produced. A failed adoption target is a measurement invariant, not restored isolation.

Chapter 10 produces mechanism, decision, and limited outcome evidence for job-completion measurement. Pretending the local checkpoint proves developer happiness would weaken Chapter 11's quota and Chapter 13's support.

4. Test the decision under failure

Independent control failure — Adoption hits 100% after the unofficial path is deleted, while time-to-environment gets worse

> **Practice — Fail the contract when adoption hides wait**
>
> Inject 100% adoption after unofficial paths are deleted and a time-to-first-environment sample that moved from 48 hours to 120, and refuse both.

The completed contract is healthy. The failure command does not rewrite it. It injects the Goodhart case against that snapshot:

```
make chapter-10-failure
```

Expected output:

```
chapter 10 failure: adoption hiding worse time-to-environment correctly
rejected
```

The injected record is:

```
indicators: [time-to-first-environment, paved-road-completion, catalog-
freshness, adoption-percentage]
samples:
  - indicator: adoption-percentage
    value: 100
    unofficial_paths_deleted: true
  - indicator: time-to-first-environment
    value: 120
    prior_value: 48
    unit: hours
```

Catalog-freshness and paved-road-completion samples stay current. The baseline's missing catalog sample and borrowed order-success indicator are not required for this failure. Adoption can become a target after the contract already looks complete. Vanity-as-indicator and hidden wait must both fail on their own terms. 120 is greater than 48. The arithmetic is the story.

Severity: high; every later quota, fleet, and support conversation will optimize a 100% adoption chart instead of Fulfillment's wait.
Plausible harm: unofficial paths are deleted; wait moves to Slack; leadership funds the portal; `obtain-bounded-environment` still does not finish.
Potential blast radius: every application team measured as "adopted," plus capacity and support decisions that trust that number.
Bounded by: Chapter 1 later proofs, the inherited observability outcome list, and the non-metric register. None of those repair a contract that treats adoption as success.
Primary principles: trustworthy evidence, explicit contracts.

Platform questions

- **User and job:** Fulfillment must finish `obtain-bounded-environment`. 100% adoption after deleting unofficial paths is not that job.
- **Isolation:** Not a new tenancy model. Chapter-local implication: Storefront `order_success_ratio` must not stand in for Fulfillment's wait. Per-tenant job time is how the product sees that Storefront being green does not isolate Fulfillment from delay.
- **Contract and exit:** The contract is the measurement set. Teams do not leave it the way they leave a paved-road scaffold. Vanity leaves the indicator set by being listed as a non-metric. That is not a Chapter 5 exit, not a Chapter 7 version bump, and not a Chapter 9 exception.
- **Platform-product evidence:** Adoption, CSAT, and ticket volume prove the platform looked busy. Required proof is sampled job time and completion. This is a platform-product SLI, not a portfolio SLO.

Diagnosis

Calling adoption success encourages Goodhart controls: delete unofficial YAML, count every remaining team as adopted, ignore the 48-to-120 hour wait. CSAT and ticket volume move the same way. Borrowing `order_success_ratio` from inherited observability makes Storefront's orders the platform's health. Missing samples make the later proofs ornamental.

The missing non-metric register makes Chapter 1's vanity refusal a memory. The missing job mapping makes Chapter 6's lease a namespace again. 100% is a target that consumed the unofficial path as evidence.

Correction

The completed model does not retain adoption as an indicator. Time-to-first-environment stays a sampled lagging SLI mapped to `obtain-bounded-environment`. Unofficial-path deletion cannot improve the grade. Inherited order metrics stay non-metrics. Completeness is computed. The failure command proves the check still fails when only adoption and the 48-to-120 hour change are injected.

This failure is a measurement-contract failure. It is not a runtime plane recovery. Do not call the failed adoption target **Evidence of restored isolation**. Nothing tenant-runtime was restored. The product stopped treating a 100% chart as job completion.

That correction changes later decisions:

- Chapter 11 must not treat a lower shared bill as product health if time-to-first-environment worsened.
- Chapter 12 must not call a fleet complete because adoption is 100%.
- Chapter 13 must not close a support ticket because CSAT moved.
- Quota and fleet change may not replace job-time samples with portal clicks.

The decision is practical because the failed vanity check is the product. Adding an arbitrary survey command would not make it more practical.

5. Production reality

Common measurement errors

Counting adoption after deleting unofficial paths

That is Goodhart. Measure job time. Leave adoption on the non-metric register.

Using CSAT or ticket volume as completion

Smiles and silence are not `ship-on-paved-road`.

Borrowing Storefront order-success as platform health

The inherited observability contract already owns those outcomes. They are tenant workload indicators. Record them with `category: tenant-workload`, not `vanity`.

Calling a job proof a portfolio SLO

Platform Chapter 1 refused portfolio SLO governance. Keep class `platform-product-sli`.

Shipping indicators without samples

A named proof with no observation is a slogan. Missing samples fail.

Building quota or surveys in this chapter

Adopt the contract. Leave floors to Chapter 11. Do not survey real developers.

6. What changed

Before	After
Adoption hit 100% after unofficial paths were deleted while wait grew from 48 hours to 120.	**Vanity adoption and hidden worse job time fail the contract.**
CSAT, tickets, and portal launch counted as success.	**Those labels are required non-metrics.**
Storefront order-success stood in for platform health.	**Inherited tenant outcomes are refused as product indicators.**
Chapter 1 later proofs had no samples.	**Each later proof is an owned SLI with an observation.**
A valid schema could appear to prove developers were happy.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a measurement contract later quota, fleet, and support chapters can use without treating a 100% adoption chart as Fulfillment's finished job.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Developer-experience measurement contract	devex/contract.yaml and devex/indicators.yaml	They retain Chapter 1 later proofs as platform-product SLIs later quota and support must not replace with vanity.
Non-metric register	devex/non-metrics.yaml	It retains the refusal of adoption, CSAT, ticket volume, and portal launch as vanity, and of inherited tenant-outcome metrics as the wrong layer.

These artifacts should change when Northwind's jobs or inherited observability outcomes materially change—not whenever a dashboard theme is restyled.

What You Learned

Developer experience is sampled job completion, not adoption, CSAT, or a green order path. Deleting unofficial paths to hit 100% adoption while wait grows is Goodhart, not success. Schema checks can prove structural completeness within declared scope. They cannot survey real developers. A decision earns its place when Fulfillment's time-to-first-environment has a sample, and when 100% adoption cannot hide that sample getting worse.

Prove It

> **Independent Practice — Measure a read-only analytics path without copying the adoption row**
>
> A data-analytics team needs bounded read access. Leadership will ask for "analytics adoption %."

Extend the Chapter 10 model without adding quota floors yet:

1. Decide which Chapter 1 job, if any, analytics finishes—or whether this is still not a platform job.
2. Name one platform-product SLI and one non-metric you must refuse if asked for adoption percentage, and which exclusion category that non-metric uses.
3. State whether Storefront `order_latency` may be reused as analytics health, given the inherited observability list.
4. Choose a sample that would falsify "analytics can self-serve"—for example time-to-first-environment moving from 48 hours to 120 with unofficial paths deleted.
5. Identify whether that sample is leading or lagging, and which owner remains accountable.
6. Explain which material change would trigger review of the measurement contract, not just one dashboard tile.

Do not copy the Fulfillment 100% adoption row and rename it. Analytics has different wait, isolation, and job-proof consequences than `obtain-bounded-environment` for `fulfillment-api`. Your durable output is the indicator-and-non-metric decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 10 capability when you can explain why 100% adoption is not job completion, trace each retained indicator to a Chapter 1 later proof and sample, describe evidence that would falsify the contract, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Honest measurements exist, but tenants still contend for unbounded shared capacity. Environments and the control plane share an unowned pool. A lower shared bill can hide Fulfillment starvation.

Chapter 11 allocates quota, cost, and capacity across tenants, so bursting cannot silently consume another tenant’s floor.

11. Allocate Quota, Cost, and Capacity Across Tenants

Chapter 10 made job time the product indicator. Fulfillment's time-to-first-environment is sampled. Adoption cannot hide a longer wait. Environments and the control plane still share an unowned pool. Chapter 3 already named `cluster-capacity-pool` and denied `unlimited-burst-into-peer-quota`. Chapter 6 already bound each lease on that pool. Isolation labels exist. Floors do not. A Fulfillment burst can still consume Storefront's environment floor, and a cheaper shared bill can call that success.

The production question is now:

> How does the platform allocate scarce capacity so one tenant cannot starve another, and what unit cost is honest?

A lower shared bill hides Fulfillment starvation, or chargeback without quality gates rewards cheap broken environments. This chapter records tenant floors and ceilings on the same pool, platform units with quality gates, and showback that cannot count a starved burst as a useful unit. Storefront's floor is not Fulfillment's headroom. A cheaper bill is not a platform-product **SLI (Service Level Indicator)**.

1. A burst that still eats the other floor

A weak record says:

```yaml
pool: cluster-capacity-pool
capacity: 32
tenants: [storefront, fulfillment]
showback:
  - tenant: fulfillment
    unit: environment-hour
    usage: 24
    billed_units: 24
  - tenant: fulfillment
    unit: order_success_ratio
    billed_units: 1
```

It does not identify a floor, a ceiling, a quality gate, or the Chapter 6 lease commitment the floor must cover. Isolation labels may still say Storefront cannot be starved below a floor. There is no floor. Fulfillment uses 24 of 32. Storefront's leases still commit 12. Eight remain. `32 - 24 = 8`, and `8 < 12`. Storefront's order-success ratio can stay green through that squeeze. The inherited observability contract already tracks `order_success_ratio`. Chapter 10 already refused it as product evidence. Billing it as a platform unit is the same borrow, now in currency.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 11 baseline:

```
make chapter-11-baseline
```

The command succeeds when it detects the intended unsafe allocation:

```
chapter 11 baseline: fulfillment burst starving storefront floor correctly
detected
```

The fixture lists both tenants on `cluster-capacity-pool` without floors, lets Fulfillment use 24 environment-hours, bills that burst as useful, and treats Storefront order-success as a cost unit. Leases exist. The pool is still unowned.

That residual drives the chapter.

2. The production model: floors on the pool you already named

> *Theory — Quota, showback, and useful platform units*
>
> This model enables Storefront and Fulfillment to finish `obtain-bounded-environment` on a shared pool without one tenant's burst becoming the other's incident, and without treating a cheaper bill as job completion.

The pool is `cluster-capacity-pool`, not a new bill

Chapter 3 already declared the shared surface: `cluster-capacity-pool`, mode `shared-quota-pool`, tenants may not inherit `unlimited-burst-into-peer-quota`. Chapter 6 already used that id as the environment product's quota pool and as each lease bound. This chapter does not rename it to make showback look like **FinOps (Financial Operations)** of the order path. Inherited DevOps useful-unit thinking applies here to the platform product: count environment-hours and successful provisions that passed a quality gate. Do not reteach Storefront's cost of serving orders.

A **shared-nothing** platform would give each tenant its own capacity account. Northwind chose a shared pool. The pool is therefore a product with an owner, floors, ceilings, and units. Unlimited burst is not that product. It is the noisy neighbor Chapter 3 already denied, now running as a cheaper bill.

Do not invent a second pool so storage class or a control-plane slice looks like quota. The plane shares this pool. Tenants are metered on it.

A floor is a lease commitment; a ceiling must leave the other floor

Storefront's leases commit `4 + 8 = 12` units on `storefront-nonprod` and `storefront-prod`. Fulfillment's leases commit the same split on `fulfillment-nonprod` and `fulfillment-prod`. Those bounds are Chapter 6's product check against unlimited lease size. They are not yet a floor the other tenant must leave behind.

This chapter sets:

Tenant	Floor	Ceiling
storefront	12	20
fulfillment	12	20

Pool capacity is 32. `12 + 20 = 32`. A ceiling that cannot be used without taking the peer's floor is not a ceiling. It is unlimited burst with extra YAML. `floor_peer + ceiling_self <= capacity` is the allocation invariant. Usage must stay at or under the ceiling. Remaining capacity after one tenant's usage must still cover the other's floor. Completeness is computed. A quota file cannot emit `status: healthy` to hide a missing floor.

Best Practice: Set each floor at least as high as that tenant's Chapter 6 lease sum. A floor below the lease commitment makes the lease ornamental.

Production Practice: Ceilings may sum to more than capacity. `20 + 20 = 40`, and `40 > 32`. That is a guaranteed floor plus an oversubscribed best-effort ceiling, not a latent overflow. The runtime check—remaining capacity after one tenant's usage still covers the peer floor—is what prevents both tenants bursting at once from starving either floor.

Production Practice: Isolation is a quota invariant joined to Chapter 3. This chapter loads quota `denied_inheritance`. A burst that leaves a peer below its floor—or below its lease sum when the floor is missing—fails `unlimited-burst-into-peer-quota`. Clearing that denial stops the inheritance error. The numeric starvation check still runs. They are not the same check.

Showback counts useful units, not a cheaper shared average

Two units are retained:

Unit	Meters	Quality gate
environment-hour	cluster-capacity-pool	lease-bound-on-pool
successful-provision	obtain-bounded-environment	time-to-first-environment-sampled

An environment-hour that is not bound on the Chapter 6 pool is not a useful unit. A successful provision without a Chapter 10 `time-to-first-environment` sample is a namespace create, not job completion. Billing a starved burst as `quality_gate_passed: true` is Goodhart in currency: Fulfillment looks cheap and busy while Storefront's floor is gone.

Chapter 10's non-metric register still applies. `category: vanity` ids are not units. `category: tenant-workload` ids—including `order_success_ratio` and `order_latency`—are not units. Chapter 1 later proofs are job-time SLIs, not cost units. A cheaper bill must not replace `time-to-first-environment`.

Do not onboard a fleet here. Chapter 12 will change contracts against this allocation. This chapter must already refuse to treat a burst through Storefront's floor as headroom.

3. Allocate the pool as a product

The completed Chapter 11 model uses three files:

```
quota/tenants.yaml
quota/units.yaml
quota/showback.yaml
```

The separation is deliberate. The policy names the pool, capacity, floors, and ceilings. Units name what is metered and which gate makes a unit useful. Showback is observation. The evaluator joins them to Chapter 3 tenants, isolation, and sharing, Chapter 6 leases and environment product, and Chapter 10 indicators, samples, and non-metric categories.

> **Practice — Bind floors to the Chapter 3 pool and Chapter 6 lease sums**
>
> Consume `cluster-capacity-pool` rather than inventing a billing account name.

Open `quota/tenants.yaml`:

```yaml
owner: platform-team
pool: cluster-capacity-pool
capacity: 32
tenants:
  - tenant: storefront
    floor: 12
    ceiling: 20
  - tenant: fulfillment
    floor: 12
    ceiling: 20
```

The evaluator loads Chapter 3 sharing and Chapter 6 `quota_pool`. It fails if the pool is renamed. It fails if a floor is below that tenant's lease sum. It fails if `floor_peer + ceiling_self` exceeds capacity.

> **Practice — Declare platform units with quality gates**
>
> Environment-hours meter the pool. Successful provisions meter `obtain-bounded-environment`. Neither meters Storefront orders.

Open `quota/units.yaml`. `environment-hour` meters `cluster-capacity-pool` with gate `lease-bound-on-pool`. `successful-provision` meters `obtain-bounded-environment` with gate `time-to-first-environment-sampled`. The evaluator joins Chapter 10 samples and non-metrics. Tagging `order_success_ratio` as a unit fails as tenant workload, not as vanity.

> **Practice — Bill only units that passed the gate**
>
> A starved burst is not a successful environment-hour. A provision without a job-time sample is not a successful provision.

Open `quota/showback.yaml`. Storefront and Fulfillment each show 12 environment-hours, matching the lease sum, billed 12, gate passed. Fulfillment shows one successful provision because Chapter 10 sampled Fulfillment's time-to-first-environment. There is no Storefront successful-provision row, because that sample does not exist. There is no adoption row, because adoption is not a unit.

The lab does not read a real cloud bill or scheduler. Completeness is whether floors cover lease commitments, ceilings leave the other floor, and showback cannot count a starved burst or a borrowed order metric as a useful unit.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-11-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 11 checkpoint: tenant floors, ceilings, and quality-gated showback
verified
```

The audit validates the three Chapter 11 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- policy owner is a living Chapter 1 user;
- the pool is Chapter 3 `cluster-capacity-pool` and matches the Chapter 6 environment product;
- every Chapter 3 tenant has a floor and a ceiling;
- each floor covers that tenant's Chapter 6 lease sum;
- a ceiling cannot be used without leaving the peer floor;
- required units are `environment-hour` and `successful-provision` with the declared gates;
- vanity, tenant-workload, and job-proof ids are not units;
- showback bills only gated usage; and
- remaining capacity after one tenant's usage still covers the other's floor.

The expected pool, units, and denied burst id live in a separate checkpoint file. An allocation under test does not emit its own passing grade.

The checkpoint does not prove that a real cluster throttled Fulfillment, that a real bill charged environment-hours, or that 32 is the right pool size for a real company. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 11 evidence
Mechanism evidence	Schemas and the quota-showback evaluator operated successfully.
Decision evidence	Pool, floors, ceilings, units, quality gates, and non-unit refusals are explicit.
Outcome evidence	Fulfillment and Storefront have computed usage at the lease sum. That is not proof a scheduler reserved those units.
Recovery evidence	Not produced. A denied burst is a quota invariant, not restored isolation after a live noisy-neighbor outage.

Chapter 11 produces mechanism, decision, and limited outcome evidence for allocation on an existing pool. Pretending the local checkpoint proves a cloud bill would weaken Chapter 12's fleet change and Chapter 14's bounded recovery.

4. Test the design under failure

Connected consequence — Fulfillment burst consumes Storefront's environment floor after isolation labels exist but quota does not

> **Practice — Fail the burst that bills Storefront's floor as Fulfillment headroom**
>
> Inject Fulfillment environment-hour usage of 24 onto the completed showback and refuse the ceiling overflow, the starved floor, and the useful-unit bill.

The completed allocation is healthy. The failure command does not rewrite it. It injects the burst against that snapshot:

```
make chapter-11-failure
```

Expected output:

```
chapter 11 failure: fulfillment burst consuming storefront floor correctly
rejected
```

The injected record is:

```
- tenant: fulfillment
  unit: environment-hour
  usage: 24
  billed_units: 24
  quality_gate_passed: true
```

Storefront stays at 12. Fulfillment's successful-provision sample stays at 1. Floors stay 12. Ceilings stay 20. The baseline's missing floors and borrowed order-success unit are not required for this failure. Burst can appear after the policy already looks complete. `24 > 20`. `32 - 24 = 8`. `8 < 12`. The arithmetic is the story.

Severity: high; every later fleet, support, and recovery conversation will optimize a cheaper shared bill instead of Storefront's reserved floor.
Plausible harm: Fulfillment scales; Storefront's environments cannot hold their lease; wait returns as "the cluster is busy"; leadership funds the lower unit cost.
Potential blast radius: both application tenants and `cluster-capacity-pool`; Storefront's order path is no longer a separate capacity domain.
Bounded by: Chapter 3 quota denial, Chapter 6 lease sums, Chapter 10 job-time samples, and the non-metric register. None of those repair a showback file that still bills 24.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Storefront and Fulfillment must finish `obtain-bounded-environment`. A Fulfillment burst to 24 is not that job for Storefront. It is a noisy neighbor substituting for a floor.
- **Isolation:** The boundary is Chapter 3's quota dimension plus this policy: `cluster-capacity-pool` as shared-quota-pool, floor 12, ceiling 20, and no `unlimited-burst-into-peer-quota`. Fulfillment's blast radius must stop before Storefront's floor. This is not a new tenancy model. It is the quota check Chapter 3 deferred and Chapter 6 could only bound per lease.
- **Contract and exit:** The contract is the quota policy and the unit catalog. Teams do not leave it the way they leave a paved-road scaffold. A burst is not a Chapter 5 exit, not a Chapter 7 version bump, not a Chapter 9 exception, and not a Chapter 10 non-metric demotion. A ceiling is not an exception to a floor.
- **Platform-product evidence:** A lower shared bill is not proof the product works. Required proof is usage at or under ceiling that still leaves the peer floor, billed only when the quality gate passed. This is a platform-product SLI input, not a portfolio **SLO (Service Level Objective)**.

Diagnosis

Calling 24 environment-hours "headroom" encourages the same unofficial path Chapter 3 interrupted: Fulfillment is blocked, someone shares the pool, Storefront's reserved units move, and the order path becomes a noisy neighbor. Isolation labels stay green. The lease still says 12. Remaining capacity is 8. Showback then bills the burst as a useful unit, so the cheaper average looks like efficiency.

The missing floor makes Chapter 3's starvation blast-radius statement ornamental. The missing ceiling makes Chapter 6's lease bound a per-namespace cap with no peer. Billing `order_success_ratio` makes Chapter 10's tenant-workload refusal a memory. Billing a starved burst makes useful-unit thinking a slogan.

Correction

The completed model does not let Fulfillment use 24. Usage stays at the lease sum until a ceiling that still leaves Storefront's 12 is granted and observed. Showback does not count a starved burst as a useful environment-hour. Inherited order metrics stay non-metrics, not units. Completeness is computed. The failure command proves the check still fails when only Fulfillment's environment-hour row moves from 12 to 24.

This failure is a quota-invariant failure. It is not a runtime plane recovery. Do not call the denied burst **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live noisy-neighbor outage. The product stopped treating Storefront's floor as Fulfillment headroom.

That correction changes later decisions:

- Chapter 12 must not onboard or upgrade a tenant by bursting through another tenant's floor.
- Chapter 13 must not close a capacity ticket because the shared unit cost moved down.
- Chapter 14 must recover `cluster-capacity-pool` without replaying one tenant's burst into another's floor.
- Fleet change may not replace floors with a cheaper bill or with Chapter 10 adoption.

The design is practical because the failed burst check is the product. Adding an arbitrary cloud-bill import would not make it more practical.

5. Production reality

Common quota errors

Sharing a pool with labels and no floors

Chapter 3 already denied unlimited burst. Labels without numbers are still a shared cluster.

Setting a ceiling that cannot leave the peer floor

`floor_peer + ceiling_self > capacity` is unlimited burst with a friendlier name. Summing both ceilings above capacity is the opposite case: oversubscription is allowed. Do not "fix" `20 + 20 > 32` by shrinking both ceilings until they statically reserve the whole pool. That removes burst. The runtime check is what keeps both bursts from happening at once.

Billing failed or starved usage as a useful unit

Inherited useful-unit thinking does not mean "count every hour." It means count hours that did not take another tenant's job.

Borrowing Storefront order-success as platform cost

Chapter 10 already refused those outcomes as product indicators. They are tenant workload, not showback.

Replacing time-to-first-environment with a cheaper bill

Chapter 10's later proof remains the job SLI. Showback is not allowed to demote it.

Building fleet onboarding in this chapter
Allocate the pool. Leave cohorts, freezes, and deprecation to Chapter 12. Do not read a real cloud bill.

6. What changed

Before	After
Fulfillment used 24 of 32 and left Storefront 8 against a 12-unit lease.	**Ceiling overflow and peer-floor starvation fail the quota invariant.**
Isolation labels existed and floors did not.	**Each Chapter 3 tenant has a floor covering its Chapter 6 lease sum.**
A cheaper shared bill counted the burst as useful.	**Starved burst cannot pass the environment-hour quality gate as billed usage.**
Storefront order-success stood in for platform cost.	**Tenant-workload ids are refused as units.**
A valid schema could appear to prove a cloud bill.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely three YAML files. Northwind now has an allocation later fleet, support, and recovery chapters can use without treating a cheaper shared bill as Storefront's reserved floor.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Tenant quota policy	quota/tenants.yaml	It retains cluster-capacity-pool, floors, and ceilings later fleet and recovery chapters must not rename or burst through.
Platform unit-cost model	quota/units.yaml and quota/showback.yaml	They retain quality-gated environment-hours and successful provisions, and the refusal to bill vanity, tenant workload, or starved burst as useful units.

These artifacts should change when Northwind's tenant set, lease commitments, or Chapter 10 job proofs materially change—not whenever a billing dashboard theme is restyled.

What You Learned

Quota is floors and ceilings on the pool Chapter 3 already named, not a new account and not a cheaper average. A Fulfillment burst to 24 that leaves Storefront 8 against a 12-unit floor is a noisy neighbor, not headroom. Schema checks can prove structural completeness within declared scope. They cannot read a real cloud bill. A design earns its place when Storefront's floor still has 12 after Fulfillment runs, and when 24 billed environment-hours cannot hide that `8 < 12`.

Prove It

> **Independent Practice — Meter a read-only analytics path without copying the 24-unit burst**
>
> A data-analytics team needs bounded read environments on the same pool. Leadership will ask for a lower analytics unit cost after "bursting into unused Storefront capacity."

Extend the Chapter 11 model without adding fleet onboarding yet:

1. Decide whether analytics is a new Chapter 3 tenant—or still not a platform tenant—and which job, if any, it finishes.
2. Name a floor that covers its lease commitment, and a ceiling that still leaves Storefront's 12.
3. State whether `order_latency` or `time-to-first-environment` may be reused as an analytics cost unit, given Chapter 10's register.
4. Choose a sample that would falsify "analytics can burst safely"—for example analytics environment-hours moving to a number that leaves Storefront remaining capacity below 12.
5. Identify which quality gate that sample must fail, and which owner remains accountable.
6. Explain which material change would trigger review of the quota policy, not just one showback tile.

Do not copy the Fulfillment 24 / Storefront 8 row and rename it. Analytics has different wait, isolation, and job-proof consequences than `obtain-bounded-environment` for `fulfillment-api`. Your durable output is the floor-ceiling-and-unit decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 11 capability when you can explain why a cheaper shared bill is not Storefront's floor, trace each tenant to a Chapter 6 lease sum and a Chapter 3 pool id, describe evidence that would falsify the allocation, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Capacity is allocated, but the fleet still cannot upgrade or leave old contracts. Two tenants consume the platform. Contract v1 must be retired. A new paved-road version is ready. Forced upgrades break tenants. Eternal v1 support becomes a second unofficial platform.

Chapter 12 runs fleet lifecycle: onboard, upgrade, and deprecate, so Fulfillment can join and leave a contract version without an outage lottery.

12. Run Fleet Lifecycle: Onboard, Upgrade, Deprecate

Chapter 11 reserved Storefront's floor. Capacity is allocated. Two tenants already consume the platform. Contract `1.0` must be retired. A new version is ready: `tenant-storage 2.0` renames `class` to `sku` with a migration note. That is the Chapter 7 bump done as a bump. It is not yet a fleet. Forced upgrades break tenants. Eternal `1.0` support becomes a second unofficial platform.

The production question is now:

> How do tenants join, absorb a platform upgrade, and leave an old contract without an outage lottery?

This chapter records onboarding without cluster-admin, a freeze window, progressive cohorts, rollback to last known good contract `1.0`, and a deprecation window that cannot close while Fulfillment still legally sends `class`. Storefront can move first. Fulfillment's `1.0` binding stays legal until migration evidence exists. Applying `2.0` to every tenant at once is the cumulative product failure continuing under a friendlier name.

1. Version 2 for everyone, while v1 was still legal

A weak record says:

```yaml
granted_role: cluster-admin
cohorts:
  - tenants: [storefront, fulfillment]
    status: complete
applied_version: "2.0"   # fulfillment
evidence: pending
window_ends_at: "2026-08-15T12:00:00Z"
remaining_tenants: [fulfillment]
```

It does not identify a freeze, a Storefront-then-Fulfillment cohort, rollback to `1.0`, or an open deprecation window. "Just cut over" may ship `sku` this week. Fulfillment still sends `class`. Chapter 7 already refused that rename on a single version. Doing it to the whole fleet in one step is the same break, now with a schedule. Cluster-admin so onboarding works is Chapter 8's residual wearing a lifecycle coat.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 12 baseline:

```
make chapter-12-baseline
```

The command succeeds when it detects the intended unsafe fleet:

```
chapter 12 baseline: all-at-once v2 apply breaking fulfillment v1
correctly detected
```

The fixture onboards Fulfillment with `cluster-admin`, applies `tenant-storage 2.0` to both tenants in one cohort, skips freeze and rollback, rewrites source, bills the apply without migration evidence, and closes `1.0` while Fulfillment remains. Quota exists. The cutover is still an outage lottery.

That residual drives the chapter.

2. The production model: a bump is not a blast radius

> *Theory — Fleet lifecycle*
>
> This model enables Fulfillment to finish `ship-on-paved-road` on a still-legal `1.0` binding while Storefront absorbs `2.0`, without inheriting cluster-admin or treating a forced cutover as a version bump.

Onboarding is a tenant-operator join, not a plane token

Fulfillment is already a Chapter 3 tenant with a Chapter 4 catalog entry, a Chapter 6 lease, and a Chapter 11 floor. Fleet onboarding records that join as complete: `fulfillment-api` on `fulfillment-nonprod`, paved road `northwind-production-path`, `granted_role: tenant-operator`, `quota_floor_respected: true`. Cluster-admin is not onboarding. Chapter 8 already refused it so onboarding works. This chapter loads that refusal. A fleet that grants `cluster-admin` to finish the join fails.

Do not burst through Storefront's floor to make room for the new tenant. Chapter 11 already failed that burst.

A fleet upgrade consumes a Chapter 7 bump; it does not replace it

`tenant-storage 2.0` exists in Chapter 7 with `sku` and `size-gb`, and compatibility records `class → sku` against version `2.0` with note `class-becomes-sku`. That is the leaving act Chapter 7 named: a contract version bump. The fleet act is how that bump reaches tenants: freeze, cohort, rollback, migration evidence.

They are not the same act:

Act	Chapter	What it is
Supported exit	5	Leave the scaffold; remaining guardrails stay
Contract version bump	7	Leave a tenant-visible API version with a migration note
Guardrail exception	9	Temporary deviation; expiry lives on the inherited DevSecOps record
Non-metric demotion	10	Vanity leaves the indicator set
Fleet upgrade	12	Roll a Chapter 7 bump across tenants with freeze, cohort, and rollback

A fleet upgrade is not a Chapter 5 exit. Teams do not leave the paved road by absorbing `2.0`. A freeze is not a Chapter 9 exception. Rollback is not a burst through a floor. Eternal `1.0` after the window, with no exception, is a second unofficial platform.

Best Practice: Split plane version from contract version. Chapter 8's last known good is plane `1.0` after a failed plane upgrade to `1.1`. This chapter's last known good is contract `tenant-storage 1.0`. Do not collapse them because both say `1.0`.

Production Practice: Isolation is a fleet invariant joined to Chapter 3 and Chapter 8. A cohort apply mutates one tenant. **GitOps (Git-based operations)** still forbids source rewrite. Clearing `controller_may_rewrite_source` stops the rewrite error. The all-at-once check still runs. They are not the same check.

Freeze, cohort, then deprecate

Storefront moves first. Fulfillment stays on `1.0` and keeps sending `class`. The freeze is `2026-08-16T00:00:00Z` to `2026-08-23T00:00:00Z`. `as_of` is inside that window. Deprecation of `1.0` stays open until `2026-11-16T00:00:00Z` with `remaining_tenants: [fulfillment]`. Closing the window without an inherited exception binding fails. Applying `2.0` to Fulfillment with `evidence: pending` fails: still-legal `v1` broke.

Production Practice: Those dates are Northwind's standing cadence, not a one-off. Seven days is Chapter 6's `ttl_hours: 168`. `2026-11-16T00:00:00Z` is the same quarterly review already used for the Chapter 5 exit `review_at` and the Chapter 6 production-lease `expires_at`. A freeze is one TTL. A deprecation window is one quarter.

Rollback `last_known_good: "1.0"` must remain allowed. A complete result that already moved every tenant without evidence has no rollback. Completeness is computed. A fleet file cannot emit `status: healthy` to hide a missing freeze.

Do not invent support tickets here. Chapter 13 will change the platform through a reviewed subject. This chapter must already refuse an all-at-once apply.

3. Run the fleet as a product

The completed Chapter 12 model uses four files:

```
fleet/onboarding.yaml
fleet/upgrades.yaml
fleet/deprecations.yaml
fleet/migrations.yaml
```

The separation is deliberate. Onboarding records the join. Upgrades name freeze, cohorts, and rollback. Deprecations name the window and who remains. Migrations are evidence. The evaluator joins them to Chapter 3 tenants, Chapter 4 systems, Chapter 5 paved road, Chapter 6 leases, Chapter 7 versions and catalog bindings, Chapter 8 subjects, Chapter 9 exception bindings, Chapter 11 quota, and the inherited GitOps interface.

> **Practice — Onboard Fulfillment without cluster-admin**
>
> Consume `tenant-operator`, the paved road, the lease, and the Chapter 11 floor rather than a plane token.

Open `fleet/onboarding.yaml`. Storefront and Fulfillment are `complete` with `granted_role: tenant-operator` and `quota_floor_respected: true`. The evaluator loads Chapter 8 subjects and fails `cluster-admin`. It loads Chapter 4 and fails an unknown system. It loads Chapter 11 and fails a join that does not respect the floor.

> **Practice — Freeze, then move one cohort**
>
> Storefront can complete `2.0`. Fulfillment's cohort stays pending while `class` is still legal.

Open `fleet/upgrades.yaml`:

```yaml
capability: tenant-storage
from_version: "1.0"
to_version: "2.0"
freeze:
  starts_at: "2026-08-16T00:00:00Z"
  ends_at: "2026-08-23T00:00:00Z"
cohorts:
  - id: storefront-cohort
    tenants: [storefront]
    status: complete
  - id: fulfillment-cohort
    tenants: [fulfillment]
    status: pending
rollback:
  last_known_good: "1.0"
  allowed: true
result: in-progress
source_rewritten: false
```

Chapter 7 already admitted `2.0` as a version and Chapter 8 admitted it for Storefront's reconcile. Fulfillment's catalog binding and reconcile stay `1.0` with `class`. The evaluator fails a single cohort that lists every tenant as complete, a missing freeze, source rewrite, and rollback to `2.0`.

> **Practice — Keep 1.0 legal until migration evidence exists**
>
> A pending Fulfillment row is not failure. Applying `2.0` without evidence is.

Open `fleet/migrations.yaml` and `fleet/deprecations.yaml`. Storefront's note is `class-becomes-sku`, `applied_version: "2.0"`, `evidence: complete`. Fulfillment's applied version is still `1.0` with evidence pending. Deprecation of `1.0` is open. The window has not closed.

The lab does not upgrade a live fleet. Completeness is whether Fulfillment joined without cluster-admin, Storefront absorbed `2.0` inside a freeze, and `1.0` remains legal until evidence exists.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-12-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 12 checkpoint: onboard, freeze, cohort, and deprecation window
verified
```

The audit validates the four Chapter 12 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- onboarding owner is a living Chapter 1 user;
- every Chapter 3 tenant is onboarded as `tenant-operator` on the paved road with a lease and a respected floor;
- `tenant-storage 2.0` exists as a Chapter 7 version, not an in-place rename of `1.0`;
- the upgrade has a freeze that covers `as_of`, two cohorts, and rollback to `1.0`;
- GitOps still forbids source rewrite;
- Storefront migration evidence is complete and Fulfillment's `1.0` apply is still pending;
- deprecation of `1.0` stays open while Fulfillment remains; and
- a closed window with remaining tenants and no Chapter 9 exception binding fails.

The expected capability, forbidden roles, and `as_of` live in a separate checkpoint file. A fleet under test does not emit its own passing grade.

The checkpoint does not prove that a real cluster rolled a Deployment, that a real freeze stopped merges, or that 2.0 is the right next storage API. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 12 evidence
Mechanism evidence	Schemas and the fleet-lifecycle evaluator operated successfully.
Decision evidence	Onboarding role, freeze, cohorts, rollback, deprecation window, and migration notes are explicit.
Outcome evidence	Storefront has a computed 2.0 apply. Fulfillment has a computed still-legal 1.0. That is not proof a cluster changed.
Recovery evidence	Not produced. A denied all-at-once apply is a fleet invariant, not restored isolation after a live cutover outage.

Chapter 12 produces mechanism, decision, and limited outcome evidence for lifecycle on existing contracts. Pretending the local checkpoint proves a live rolling upgrade would weaken Chapter 13's change authority and Chapter 14's bounded recovery.

4. Test the design under failure

Cumulative product failure — Platform v2 is applied to all tenants at once; fulfillment's still-legal v1 contract breaks

> **Practice — Fail the cutover that applies 2.0 before Fulfillment's class binding has a note**
>
> Inject a skipped freeze, both cohorts complete, and Fulfillment `applied_version: "2.0"` with pending evidence, and refuse the all-at-once apply.

The completed fleet is healthy. The failure command does not rewrite it. It injects the residual all-at-once case against that snapshot:

```
make chapter-12-failure
```

Expected output:

```
chapter 12 failure: all-at-once v2 apply breaking fulfillment v1 correctly
rejected
```

The injected record is:

```
freeze: {}
result: complete
cohorts:
  - tenants: [storefront]
    status: complete
  - tenants: [fulfillment]
    status: complete
# fulfillment-storage-2-0
applied_version: "2.0"
evidence: pending
```

Onboarding stays `tenant-operator`. Deprecation stays open. Storefront's completed migration stays `2.0` with evidence. The baseline's cluster-admin join, source rewrite, and closed window are not required for this failure. An all-at-once apply can appear after the policy already looks complete. Fulfillment still had a legal `1.0` `class` binding. `2.0` expects `sku`. Pending evidence means the note was not executed. Last known good was not restored.

Severity: high; every later support and recovery conversation will optimize a fleet-wide cutover instead of a still-legal tenant contract.

Plausible harm: Fulfillment keep-alive traffic sends `class`; storage `2.0` rejects it; warehouse effects stop while Storefront's `sku` path looks healthy.

Potential blast radius: both application tenants and `tenant-storage`; Fulfillment's still-legal `1.0` is no longer a separate compatibility domain.

Bounded by: Chapter 7 version `2.0` and migration note, Chapter 8 admission and last known good, Chapter 9 exception bindings, and inherited GitOps. None of those repair a complete result that already moved every tenant.

Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `ship-on-paved-road` on a contract it can still send. An all-at-once `2.0` apply is not that job. It is a cutover substituting for a cohort.
- **Isolation:** The boundary is Chapter 3's tenant plus this cohort: Storefront can complete `2.0` without Fulfillment absorbing `sku` in the same step. Fulfillment's blast radius must stop at Fulfillment. This is not a new tenancy model. It is the compatibility window Chapter 7 deferred to fleet.
- **Contract and exit:** The contract is Chapter 7 `tenant-storage` versions plus this freeze and deprecation window. Teams do not leave it the way they leave a paved-road scaffold. Absorbing `2.0` is a Chapter 7 bump rolled by the fleet, not a Chapter 5 exit, not a Chapter 9 exception, not a Chapter 10 non-metric, and not a Chapter 11 burst.
- **Platform-product evidence:** A finished cohort dashboard is not proof the product works. Required proof is freeze, pending Fulfillment `1.0`, complete Storefront evidence, and rollback still allowed. This is not a portfolio **SLO (Service Level Objective)**. Adoption at 100% after deleting unofficial paths is still Chapter 10 vanity.

Diagnosis

Calling the cutover "v2 is ready" encourages the same unofficial path Chapter 7 interrupted in-place and Chapter 8 renamed cluster-admin: Fulfillment is blocked, someone shares the change, Storefront's `sku` and Fulfillment's `class` collide, and the warehouse path becomes the outage. Freeze labels stay in the runbook. Both cohorts show complete. Evidence stays pending. Last known good stays a field that was not used.

The missing freeze makes Chapter 7's migration note a comment. The missing cohort makes Chapter 8's tenant-scoped reconcile a slogan. Closing `1.0` while Fulfillment remains makes deprecation a second unofficial platform. Cluster-admin onboarding makes the join shared root again.

Correction

The completed model does not mark Fulfillment's cohort complete. Freeze stays in force. Fulfillment's applied version stays `1.0` with pending evidence. Storefront stays on `2.0` with complete evidence. Rollback to `1.0` stays allowed. Onboarding stays `tenant-operator`. Completeness is computed. The failure command proves the check still fails when only freeze, both cohort statuses, Fulfillment's applied version, and result are injected.

This failure is a fleet-invariant failure. It is not a runtime plane recovery. Do not call the denied cutover **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live apply. The product stopped treating every tenant as one compatibility domain.

That correction changes later decisions:

- Chapter 13 must not patch the plane in place to "finish the cutover."
- Chapter 14 must recover `kubernetes-control-plane` without replaying Storefront's `2.0` reconcile into Fulfillment's `1.0`.
- Support may not close a ticket because the fleet dashboard shows 100% on `2.0`.
- A deprecation window may not replace migration evidence.

The design is practical because the failed all-at-once apply is the product. Adding an arbitrary kubectl rollout would not make it more practical.

5. Production reality

Common fleet errors

Granting cluster-admin so onboarding works

Onboarding is a tenant join. Cluster-admin is shared root. Chapter 8 already refused the bargain.

Applying the new version to every tenant in one step

That is Chapter 7's silent rename at fleet scale. Freeze, then one cohort.

Closing v1 while a tenant still legally sends it

Eternal v1 without a window is an unofficial platform. A closed window without an exception is an outage.

Collapsing plane 1.0 and contract 1.0

Chapter 8 retains plane last known good. This chapter retains contract last known good. Same string, different products.

Rewriting source so the cutover sticks

Inherited GitOps already forbids it. A fleet upgrade is reviewed intent, not a controller edit.

Building support-on-call in this chapter

Run freeze, cohort, and deprecation. Leave unofficial plane-admin edits to Chapter 13. Do not upgrade a live fleet.

6. What changed

Before	After
2.0 was applied to Storefront and Fulfillment in one step while Fulfillment still sent class.	**All-at-once apply and broken v1 without migration evidence fail the fleet invariant.**
Fulfillment onboarded with cluster-admin.	**Onboarding is tenant-operator on the paved road.**
There was no freeze and no rollback to contract 1.0.	**Freeze covers as_of; last known good stays 1.0 and allowed.**
1.0 closed while Fulfillment remained.	**Deprecation stays open until evidence exists or an exception binds.**
A valid schema could appear to prove a live rollout.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a fleet lifecycle later support and recovery chapters can use without treating a finished dashboard as Fulfillment's still-legal contract.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Fleet lifecycle policy	fleet/onboarding.yaml and fleet/upgrades.yaml	They retain the no-cluster-admin join, freeze, cohorts, and contract last known good later change and recovery chapters must not skip.
Upgrade-and-deprecation record	fleet/deprecations.yaml and fleet/migrations.yaml	They retain the open 1.0 window, Storefront's completed class-becomes-sku evidence, and Fulfillment's still-legal pending row.

These artifacts should change when Northwind's contract versions, tenant set, or freeze calendar materially change—not whenever a rollout dashboard theme is restyled.

What You Learned

A fleet upgrade is how a Chapter 7 version bump reaches tenants: freeze, cohort, rollback, and migration evidence. Applying `2.0` to everyone while Fulfillment's `1.0` `class` binding is still legal is an outage lottery, not a cutover. Schema checks can prove structural completeness within declared scope. They cannot upgrade a live fleet. A design earns its place when Storefront can be on `sku` while Fulfillment still sends `class`, and when both cohorts complete without evidence cannot hide that break.

Prove It

> **Independent Practice — Onboard a read-only analytics tenant without copying the all-at-once 2.0 row**
>
> A data-analytics path needs to join `storefront-nonprod` for reads and must absorb a later contract bump without taking Fulfillment's `1.0` window with it.

Extend the Chapter 12 model without adding support-on-call yet:

1. Decide whether analytics is a new Chapter 3 tenant—or still not a platform tenant—and which job, if any, it finishes.
2. Name the granted role that is not `cluster-admin`, and whether its join may burst Storefront's floor.
3. State which Chapter 7 version it binds first, and which cohort it must not share with Fulfillment's pending `1.0` row.
4. Choose a sample that would falsify "analytics can cut over safely"—for example both analytics and Fulfillment marked complete on `2.0` with analytics evidence pending.
5. Identify last known good if that apply fails—contract version, plane version, or both.

6. Explain which material change would trigger review of the fleet policy, not just one cohort tile.

Do not copy the Fulfillment all-at-once `2.0` row and rename it. Analytics has different admission, freeze, and remaining-window consequences than `tenant-storage` for `fulfillment-api`. Your durable output is the onboard-cohort-and-window decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 12 capability when you can explain why an all-at-once `2.0` apply is still a silent rename at fleet scale, trace a tenant to a Chapter 7 version, a freeze, and inherited GitOps state, describe evidence that would falsify rollback, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

The fleet can move, but platform change still happens through heroes in a shared chat. Support is still “message the people who built it.” Platform engineers apply live fixes with plane-admin. Tenants cannot tell a product incident from an application incident.

Chapter 13 supports, escalates, and changes the platform safely, so a live unofficial edit is rejected and a tenant ticket maps to a product or application owner.

13. Support, Escalate, and Change the Platform Safely

Chapter 12 can move the fleet. Storefront is on `tenant-storage 2.0`. Fulfillment's `1.0 class` binding is still legal. Support is still "message the people who built it." Platform engineers apply live fixes with plane-admin. Tenants cannot tell a product incident from an application incident. Chapter 4 already named living escalation contacts. Chapter 8 already refused cluster-admin so onboarding works. A patch in place to clear a ticket is that token wearing a pager.

The production question is now:

> Who may change the platform, how are tenant incidents escalated, and what prevents informal production edits?

This chapter records support tiers, escalation that joins the catalog, a platform-product job-time budget rather than **CSAT (Customer Satisfaction)**, and a change path whose subject is reviewed. An unofficial plane-admin edit fails. Closing Fulfillment's wait because Storefront's orders look green fails. The inherited incident interface already names `terminal_order_outcomes`. Those are tenant-path recovery evidence. They are not **Evidence of bounded platform-product recovery**.

1. A live patch to clear a ticket

A weak record says:

```yaml
id: live-plane-patch
resource: kubernetes-control-plane
subject: plane-reconciler
approved_by: plane-reconciler
granted_role: cluster-admin
last_known_good: "1.1"
unofficial: true
ticket: fulfillment-warehouse-delay
escalation: chat-history
closed_reason: csat
error_budget_indicators: [order_success_ratio]
```

It does not identify a Chapter 4 on-call, a split between platform-product and tenant-application, a living approver, or Chapter 8's retained plane last known good `1.0`. "Just patch it" may clear the chat. The reconciler approves itself. Last known good moves to the version Chapter 8 already failed. Fulfillment's warehouse delay is closed because a survey smiled. Storefront's order-success stands in for the platform-product job-time budget.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 13 baseline:

```
make chapter-13-baseline
```

The command succeeds when it detects the intended unsafe support model:

```
chapter 13 baseline: unofficial plane-admin patch correctly detected
```

The fixture lets `plane-reconciler` patch `kubernetes-control-plane` with `cluster-admin`, skip review, move last known good to `1.1`, escalate Fulfillment to chat history, close the ticket for CSAT, and meter the platform on `order_success_ratio`. The fleet can move. Change is still a hero in a shared chat.

That residual drives the chapter.

2. The production model: a ticket has a class, a change has a subject

> *Theory — Support, escalation, and platform-change authority*
>
> This model enables Fulfillment to finish `obtain-bounded-environment` and `ship-on-paved-road` by escalating a product wait to `platform-oncall` and an application delay to `fulfillment-oncall`, without a live plane patch substituting for a reviewed change.

A tenant ticket is not a platform incident

Chapter 4 already bound every runnable system to a living owner and an escalation contact. This chapter loads that map. `fulfillment-api` is a tenant-application: owner `fulfillment-team`, escalation `fulfillment-oncall`. `environment-provisioning` is a platform-product: owner `platform-team`, escalation `platform-oncall`. `notification-service` is still a tenant-application on a Chapter 5 exit. An exit is not unsupported. An unofficial fork is. Chat history is.

Misclassifying a platform wait as Fulfillment's warehouse bug sends the page to the wrong rotation. Classifying a warehouse delay as a plane incident invites the live patch this chapter refuses.

Best Practice: Map every catalog system to exactly one support class. Platform-product systems page `platform-oncall`. Tenant-application systems page the Chapter 4 tenant contact.

Production Practice: The platform-product job-time budget is Chapter 10's job proofs `time-to-first-environment` and `paved-road-completion`. It is not a portfolio **SLO (Service Level Objective)**. Portfolio error-budget policy remains a Platform Chapter 1 non-goal with remaining owner `reliability-program`. `order_success_ratio` and `order_latency` stay tenant-workload non-metrics. CSAT and ticket volume stay vanity. Closing a product incident because those numbers moved fails.

A platform change has a reviewed subject, not a pager token

Chapter 8's plane subject is `plane-reconciler` with `may_approve_plane_change: false`. Last known good after the failed plane upgrade to `1.1` is still `1.0`. That is plane last known good. It is not Chapter 12's contract last known good on `tenant-storage` `1.0`. Do not collapse them because both say `1.0`.

A reviewed change names `platform-team` as subject and approver, keeps `last_known_good: "1.0"`, and does not rewrite source. An unofficial change that grants `cluster-admin`, lets `plane-reconciler` approve itself, or moves last known good to `1.1` fails. The inherited evidence map requires an independent producer. A change file cannot emit `result: allow` to hide an unofficial patch.

Production Practice: Self-approval is not `subject == approved_by`. An allowed change fails when its `subject` or `approved_by` is a Chapter 8 plane identity (`kind: plane`). That identity is `plane-reconciler`, already flagged `may_approve_plane_change: false`. `platform-team` may appear in both fields because it is a living Chapter 1 user, not that plane subject. A governing team recorded as requester and approver inside its own domain is not an automation identity certifying itself.

They are not the same act:

Act	Chapter	What it is
Supported exit	5	Leave the scaffold; remaining guardrails stay
Contract version bump	7	Leave a tenant-visible API version with a migration note
Guardrail exception	9	Temporary deviation; expiry lives on the inherited record
Non-metric demotion	10	Vanity leaves the indicator set
Fleet upgrade	12	Roll a Chapter 7 bump with freeze, cohort, and rollback
Platform change	13	Reviewed subject on the plane; unofficial plane-admin fails

A live patch is not a fleet upgrade. It is not a Chapter 9 exception. It is not a Chapter 5 exit. Communication cadence is Chapter 12's freeze window: seven days, the same TTL. Heroes in chat are not that cadence.

The inherited DevOps incident interface still requires `desired_actual_agreement`, `healthy_consecutive_samples`, and `terminal_order_outcomes` when an order path recovers. A platform-product incident may not list those as its recovery evidence. They prove Storefront's workload, not that `obtain-bounded-environment` finished.

Do not restore mixed backups here. Chapter 14 will recover the plane against last known good and tenant isolation. This chapter must already refuse the unofficial patch that would become that mixed backup.

3. Support the product as a product

The completed Chapter 13 model uses four files:

```
support/model.yaml
support/escalation.yaml
support/changes.yaml
support/incidents.yaml
```

The separation is deliberate. The model names tiers, unsupported paths, freeze cadence, and the job-time budget. Escalation binds every catalog system. Changes are the plane-authority record. Incidents are observations. The evaluator joins them to Chapter 1 users and brief, Chapter 4 systems and ownership, Chapter 8 subjects, plane product, and last known good, Chapter 10 indicators and non-metrics, and the inherited observability, incident, and evidence-map interfaces.

> **Practice — Split platform-product from tenant-application**
>
> Page `platform-oncall` for environment waits. Page `fulfillment-oncall` for warehouse delays. Do not close either for CSAT.

Open `support/model.yaml` and `support/escalation.yaml`. Tiers are `platform-product` and `tenant-application`. Unsupported paths are `unofficial-fork` and `chat-history`. Job-time budget indicators are `time-to-first-environment` and `paved-road-completion`. Every Chapter 4 system has a route whose owner and escalation match the catalog. `notification-service` stays on `storefront-oncall`. It left the scaffold. It did not leave support.

> **Practice — Keep Chapter 8 last known good on a reviewed plane change**
>
> `platform-team` may update admission. `plane-reconciler` may not approve that change. Last known good stays plane `1.0`.

Open `support/changes.yaml`:

```yaml
id: reviewed-admission-note
resource: kubernetes-control-plane
subject: platform-team
approved_by: platform-team
action: update-admission
last_known_good: "1.0"
current_version: "1.0"
unofficial: false
source_rewritten: false
result: allow
```

The evaluator loads Chapter 8's failed plane upgrade and fails a change whose last known good is not `1.0`. It fails `cluster-admin`, a plane identity as subject or approver, and source rewrite. It loads Chapter 8 `forbidden_roles` live. Clearing that list does not make an `unofficial: true` patch legal. `platform-team` in both fields is the reviewed path, not the self-approval the live patch impersonates.

> **Practice — Record the ticket against the route, not against a smile**
>
> Fulfillment's warehouse delay stays a tenant-application incident. Environment wait stays a platform-product incident with `time-to-first-environment`.

Open `support/incidents.yaml`. Neither row reports green. Neither closes for CSAT or `order_success_ratio`. The platform row does not list `terminal_order_outcomes`. Completeness is computed.

The lab does not operate a real ticketing system. Completeness is whether every catalog system has a living route, whether unofficial plane-admin fails, and whether the job-time budget is still Chapter 10's job proofs.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-13-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 13 checkpoint: escalation routes and reviewed plane change
verified
```

The audit validates the four Chapter 13 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- model owner is a living Chapter 1 user and the plane is `kubernetes-control-plane`;
- communication cadence is Chapter 12's freeze window;
- job-time budget contains Chapter 1 brief success evidence and no vanity or tenant-workload ids;
- every Chapter 4 system has a route whose class, owner, and escalation match the catalog;
- chat history and unofficial forks are unsupported;
- an allowed change whose subject or approver is `plane-reconciler` fails self-approval; `platform-team` in both fields does not;
- unofficial or `cluster-admin` changes fail;
- plane last known good joins Chapter 8's retained `1.0`;
- incidents match their routes and cannot close for CSAT or order-success; and
- a platform incident cannot borrow inherited tenant recovery evidence.

The expected plane, cadence, budget, and forbidden roles live in a separate checkpoint file. A support model under test does not emit its own passing grade.

The checkpoint does not prove that a real pager fired, that a real freeze stopped a merge, or that 48 hours is the right wait. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 13 evidence
Mechanism evidence	Schemas and the support-and-change evaluator operated successfully.
Decision evidence	Tiers, routes, job-time budget indicators, reviewed subjects, and unsupported paths are explicit.
Outcome evidence	Fulfillment has a computed tenant-application incident. Environment wait has a computed platform-product incident. That is not proof an on-call engineer was paged.
Recovery evidence	Not produced. A denied live patch is a change-authority invariant, not restored isolation after a live plane edit.

Chapter 13 produces mechanism, decision, and limited outcome evidence for support on an existing plane. Pretending the local checkpoint proves a ticketing vendor would weaken Chapter 14's bounded recovery.

4. Test the design under failure

Independent control failure — An engineer patches the control plane in place to clear a ticket, leaving no review and no last known good

> **Practice — Fail the live patch that clears Fulfillment's ticket with plane-admin**
> Inject `plane-reconciler` as subject and approver, `cluster-admin`, last known good `1.1`, and `unofficial: true`, and refuse the change.

The completed support model is healthy. The failure command does not rewrite it. It injects the residual live-patch case against that snapshot:

```
make chapter-13-failure
```

Expected output:

```
chapter 13 failure: unofficial plane-admin patch correctly rejected
```

The injected record is:

```
id: live-plane-patch
resource: kubernetes-control-plane
subject: plane-reconciler
approved_by: plane-reconciler
action: patch-in-place
granted_role: cluster-admin
ticket: fulfillment-warehouse-delay
last_known_good: "1.1"
current_version: "1.1"
unofficial: true
source_rewritten: true
result: allow
```

Escalation routes stay on Chapter 4 contacts. Fulfillment's incident stays tenant-application. The reviewed admission note stays `1.0`. The baseline's chat-history route, CSAT close, and order-success as the job-time budget are not required for this failure. An unofficial patch can appear after the model already looks complete. Chapter 8 already failed the move to plane `1.1`. Last known good `1.1` is that failure, now used as a ticket-clearing write.

Severity: high; every later recovery conversation will restore an unofficial plane instead of reviewed last known good.
Plausible harm: Fulfillment's warehouse ticket closes; the reconciler holds cluster-admin; Storefront's `2.0` and Fulfillment's `1.0` share one patched plane; Chapter 14 then has a mixed backup.
Potential blast radius: both application tenants and `kubernetes-control-plane`; a tenant ticket is no longer a separate support domain.
Bounded by: Chapter 4 escalation, Chapter 8 last known good and forbidden roles, Chapter 10 non-metrics, and inherited incident and evidence-map interfaces. None of those repair an allowed unofficial patch.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `ship-on-paved-road` or `obtain-bounded-environment`. A live plane patch to clear a warehouse ticket is not those jobs. It is shared authority substituting for a reviewed subject.
- **Isolation:** The boundary is Chapter 3's tenant plus this classification: a tenant-application incident must not mutate the plane. Fulfillment's blast radius must stop at Fulfillment. This is not a new tenancy model. It is change authority joined to Chapter 8's plane subject.
- **Contract and exit:** The contract is the support model, the catalog route, and the reviewed change path. Teams do not leave it the way they leave a paved-road scaffold. A live patch is not a Chapter 5 exit, not a Chapter 7 version bump, not a Chapter 9 exception, not a Chapter 10 non-metric, and not a Chapter 12 fleet upgrade.
- **Platform-product evidence:** A closed ticket is not proof the product works. Required proof is a classified incident plus a reviewed plane change that still holds last known good `1.0`. This is a platform-product job-time budget, not a portfolio SLO. Storefront `order_success_ratio` remains tenant workload.

Diagnosis

Calling the write "so the ticket closes" encourages the same unofficial path Chapter 8 interrupted: Fulfillment is blocked, someone shares plane-admin, Storefront moves with the patch, and last known good becomes the broken version. Chat history becomes escalation. CSAT becomes closure. Order-success becomes the platform's health. Inherited `terminal_order_outcomes` then look like recovery.

The missing class split makes Chapter 4's contacts a directory. The missing review makes Chapter 8's `may_approve_plane_change: false` ornamental. Last known good `1.1` makes Chapter 8's failed upgrade a successful move. Closing for CSAT makes Chapter 10's vanity refusal a memory.

Correction

The completed model does not allow `live-plane-patch`. The reviewed change keeps subject `platform-team`, last known good `1.0`, and `unofficial: false`. Fulfillment's warehouse delay stays a tenant-application incident on `fulfillment-oncall`. Environment wait stays a platform-product incident on `time-to-first-environment`. Completeness is computed. The failure command proves the check still fails when only the unofficial patch is injected.

This failure is a change-authority invariant. It is not a runtime plane recovery. Do not call the denied patch **Evidence of restored isolation**. Nothing tenant-runtime was restored from a live edit. The product stopped treating a closed ticket as a plane change.

That correction changes later decisions:

- Chapter 14 must restore `kubernetes-control-plane` from reviewed last known good `1.0`, not from an unofficial `1.1` patch taken to clear a ticket.
- Mixed-backup restore must not replay a hero edit into another tenant.
- Support may not freeze every tenant because one warehouse ticket was noisy.
- Regional-loss and portfolio RTO remain SRE. This chapter does not claim them.

The design is practical because the failed unofficial patch is the product. Adding an arbitrary ticket vendor would not make it more practical.

5. Production reality

Common support errors

Messaging the people who built it
Chapter 4 already named `fulfillment-oncall` and `platform-oncall`. Chat history is an unsupported path.

Patching the plane in place to clear a tenant ticket
Tenant-application incidents do not grant plane-admin. Chapter 8 already refused that bargain for onboarding.

Closing because CSAT or ticket volume moved
Those are Chapter 10 vanity non-metrics. Job time is the product indicator.

Borrowing Storefront order-success as the job-time budget
The inherited observability contract already owns those outcomes. They are tenant workload.

Collapsing plane 1.0 and contract 1.0
Chapter 8 retains plane last known good. Chapter 12 retains contract last known good. A live patch must not move either by accident.

Building mixed-backup restore in this chapter
Refuse unofficial plane-admin. Leave isolated restore to Chapter 14. Do not operate a real ticketing system.

6. What changed

Before	After
plane-reconciler patched the plane with cluster-admin to clear a ticket.	**Unofficial plane-admin, self-approval, and missing last known good fail the change path.**
Fulfillment escalated to chat history.	**Every catalog system routes to its Chapter 4 contact.**
A product wait and a warehouse delay used the same queue.	**Platform-product and tenant-application are distinct classes.**
CSAT and order-success closed the incident.	**Vanity and tenant-workload ids cannot close a ticket or fill the job-time budget.**
A valid schema could appear to prove a pager fired.	**Structural, decision, outcome, and recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a support and change path Chapter 14 can recover against without treating a hero edit as last known good.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Support and escalation model	support/model.yaml and support/escalation.yaml	They retain the product/application split, freeze cadence, job-time budget, and Chapter 4 contacts later recovery must not replace with chat.
Platform-change authority record	support/changes.yaml	It retains the reviewed subject and Chapter 8 plane last known good an unofficial patch must not move.

These artifacts should change when Northwind's catalog systems, plane last known good, or Chapter 10 job proofs materially change—not whenever a chat tool is restyled.

What You Learned

Support is a classified ticket plus a reviewed plane change. A live patch to clear Fulfillment's warehouse delay is shared plane-admin, not a support path. Schema checks can prove structural completeness within declared scope. They cannot page a real on-call. A design earns its place when `environment-provisioning` pages `platform-oncall`, `fulfillment-api` pages `fulfillment-oncall`, and `plane-reconciler` cannot approve a patch that moves last known good to `1.1`.

Prove It

> **Independent Practice — Route a read-only analytics outage without copying the live-patch row**
>
> A data-analytics path in `storefront-nonprod` pages at 02:00. Someone will offer to patch the plane so the ticket closes.

Extend the Chapter 13 model without adding mixed-backup restore yet:

1. Decide whether analytics is a tenant-application, a platform-product, or still not a platform job.
2. Name the Chapter 4 owner and escalation contact if it is a catalog system—or which unsupported path it is if it is an unofficial fork.
3. State whether the outage may close because CSAT moved, or because Storefront `order_latency` recovered.
4. Choose a sample that would falsify “analytics can be patched safely”—for example `plane-reconciler` as approver with last known good `1.1`.
5. Identify which last known good that sample must retain—plane version, contract version, or both.
6. Explain which material change would trigger review of the support model, not just one ticket.

Do not copy the Fulfillment live-patch row and rename it. Analytics has different class, escalation, and last-known-good consequences than a warehouse delay cleared with cluster-admin. Your durable output is the class-route-and-change decision and its reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 13 capability when you can explain why a live plane patch is not support, trace a ticket to a Chapter 4 contact and a Chapter 8 last known good, describe evidence that would falsify unofficial plane-admin, distinguish structural validation from decision and outcome evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Support is explicit, but a control-plane outage can still become every tenant’s outage. Backup of the plane is a job-success metric. Tenant isolation during restore is undefined. Restoring from a mixed backup can replay one tenant’s intent into another, or freeze all application traffic unnecessarily.

Chapter 14 recovers a control-plane failure without taking tenants with it, so mixed-tenant replay is rejected and Storefront can continue or freeze by explicit decision, not by accident.

14. Recover a Control-Plane Failure Without Taking Tenants With It

Chapter 13 named a support class and a reviewed plane change. Last known good is still plane `1.0`. Storefront is still on `tenant-storage 2.0`. Fulfillment's `1.0 class` binding is still legal. Backup of the plane is a job-success metric. Tenant isolation during restore is undefined. Restoring from a mixed backup can replay Fulfillment's intent into Storefront, or freeze every application path because the shared plane was down. Chapter 8 already retained last known good after a failed move to `1.1`. A newest snapshot of that failed version is that residual wearing a restore coat.

The production question is now:

> How does Northwind restore the platform product after control-plane loss without reconstructing every tenant as one blast radius?

This chapter records independently verified plane evidence, an isolated restore to last known good, and a per-tenant continue or freeze that is explicit. Mixed-tenant replay fails. Storefront's order path continues or freezes by decision, not by accident. Fulfillment isolation holds. Regional-loss architecture and portfolio **RTO (Recovery Time Objective)** programs remain SRE.

1. The newest backup, with both tenants inside it

A weak record says:

```yaml
snapshot: plane-newest-corrupt
restored_version: "1.1"
last_known_good: "1.1"
mixed_backup: true
subject: plane-reconciler
replayed_from: fulfillment   # storefront row
decision: freeze
source: accident
recovered_indicators: [order_success_ratio]
```

It does not identify Chapter 8's retained plane `1.0`, a snapshot that is not mixed, an explicit tenant decision, or a restore subject that is not the reconciler. "Just restore newest" may bring the API back. Storefront receives Fulfillment's `1.0 class` intent. Both tenants freeze because the plane was down. Order-success stands in for platform recovery. The unofficial `1.1` patch Chapter 13 refused becomes last known good.

Work from the lab working tree using the How to Use This Book procedure. From the Platform lab root, run the Chapter 14 baseline:

```
make chapter-14-baseline
```

The command succeeds when it detects the intended unsafe restore:

```
chapter 14 baseline: mixed-backup restore replaying fulfillment into
storefront correctly detected
```

The fixture applies the corrupted newest snapshot, lets `plane-reconciler` approve itself with `cluster-admin`, moves last known good to `1.1`, replays Fulfillment into Storefront, freezes Storefront by accident, and meters recovery on `order_success_ratio`. Support exists. Restore is still shared authority.

That residual drives the chapter.

2. The production model: restore the plane, not the blast radius

> *Theory — Isolated control-plane restore*
>
> This model enables Fulfillment to finish `obtain-bounded-environment` and `ship-on-paved-road` after plane loss by restoring independently verified last known good, without inheriting Storefront's `2.0` intent or freezing every tenant because one plane was down.

Plane evidence is not tenant data-plane evidence

Chapter 8's last known good is the plane version still running after the failed upgrade to `1.1`: `1.0`. That is not a backup of Storefront orders. It is not Chapter 12's contract last known good on `tenant-storage` `1.0`. Do not collapse them because both say `1.0`. This chapter restores the plane against Chapter 8's retention. Tenant storage versions stay where fleet left them: Storefront `2.0`, Fulfillment `1.0`.

The inherited restore contract requires five roots: `reviewed_intent`, `artifact_identity`, `configuration_identity`, `durable_data`, and `identity_policy`. Completeness of those roots is not isolation. The newest snapshot in this lab carries all five and is still mixed, corrupt, and unverified. Applying it fails.

Best Practice: Inventory last known good and newest separately. Restore the independently verified isolated snapshot. Keep the mixed newest record so a later debugger can see what was refused.

Production Practice: Newest is not last known good. A snapshot can hold every inherited root and still replay one tenant into another. Clearing `mixed_tenants` stops the mixed-backup error. The newest-is-not-LKG check still runs. They are not the same check.

Continue and freeze are tenant decisions, not restore defaults

A shared-plane outage tempts two mistakes: return every tenant's traffic because the API answered, or freeze every tenant because the plane was down. Both treat tenants as one blast radius.

Storefront may **continue**. Storefront may **freeze**. Fulfillment likewise. The source must be `explicit-tenant-decision`. Accident fails. An explicit freeze is not an accidental freeze. A frozen tenant must not return traffic. A continuing tenant must present the inherited traffic-return evidence: `reconciled_business_state` and `verified_service_outcome`. Those gates are not Storefront `order_success_ratio`. That id remains tenant-workload. It cannot prove bounded platform-product recovery.

Production Practice: Traffic return is not plane restore. Restoring the five roots does not return traffic. Meeting the inherited traffic-return pair does not choose continue for a tenant. Continue and freeze are explicit decisions joined to Chapter 3's tenant boundary. Accidental freeze of Storefront is not that join.

Quota still binds. Restored units must match Chapter 6 lease sums. Remaining capacity after one tenant's restored usage must still cover the peer floor. A restore that replays Fulfillment's burst into Storefront's floor fails `unlimited-burst-into-peer-quota` the same way Chapter 11 failed the live burst.

A restore is not a leave, a bump, or a live patch

They are not the same act:

Act	Chapter	What it is
Supported exit	5	Leave the scaffold; remaining guardrails stay
Contract version bump	7	Leave a tenant-visible API version with a migration note
Guardrail exception	9	Temporary deviation; expiry lives on the inherited record
Non-metric demotion	10	Vanity leaves the indicator set
Fleet upgrade	12	Roll a Chapter 7 bump with freeze, cohort, and rollback
Platform change	13	Reviewed subject on the plane; unofficial plane-admin fails
Isolated restore	14	Restore plane last known good; mixed-tenant replay fails

A mixed backup apply is not a Chapter 8 plane upgrade. It is not a Chapter 12 fleet upgrade. It is not a Chapter 13 unofficial patch, even when it uses `plane-reconciler` and last known good `1.1`.

Production Practice: Self-approval on a restore is still not `subject == approved_by`. An allowed restore fails when its `subject` or `approved_by` is a Chapter 8 plane identity (`kind: plane`). That identity is `plane-reconciler`, already flagged `may_approve_plane_change: false`. `platform-team` may appear in both fields because it is a living Chapter 1 user, not that plane subject. The restore YAML repeats Chapter 13's healthy pattern; it does not invent a symmetry check.

Do not build a regional-loss program here. Do not assign a portfolio RTO. Chapter 1 already left those with remaining owner `reliability-program`. This chapter records that limitation on the verification. A verification file cannot emit `status: recovered` to hide a mixed apply.

3. Recover the plane as a product

The completed Chapter 14 model uses four files:

```
recovery/plane-evidence.yaml
recovery/isolation.yaml
recovery/restore-trace.yaml
recovery/verification.yaml
```

The separation is deliberate. Plane evidence inventories snapshots. Isolation records per-tenant continue or freeze, replay, and restored contract version. Restore-trace is the plane-authority record. Verification is the bounded claim. The evaluator joins them to Chapter 1 users, Chapter 3 tenants and isolation, Chapter 6 leases, Chapter 8 subjects, plane product, and last known good, Chapter 11 quota, Chapter 12 migrations, Chapter 13 changes and incidents, and the inherited restore contract. It does not load the **GitOps (Git-based operations)**, observability, or incident interfaces those earlier chapters already consumed.

> **Practice — Keep last known good and newest in the same inventory**
>
> Restore `plane-lkg-1-0`. Leave `plane-newest-corrupt` on file as the refused snapshot. Do not delete the mixed record to make the inventory look clean.

Open `recovery/plane-evidence.yaml`. Owner is `platform-team`. Plane is `kubernetes-control-plane`. Last known good is `1.0`. The verified snapshot is not newest, not mixed, not corrupt. The newest snapshot has the same five roots and is still mixed.

> **Practice — Restore Chapter 8 last known good with a reviewed subject**
>
> `platform-team` may restore. `plane-reconciler` may not approve that restore. Last known good stays plane `1.0`.

Open `recovery/restore-trace.yaml`:

```yaml
id: restore-plane-lkg
snapshot: plane-lkg-1-0
resource: kubernetes-control-plane
subject: platform-team
approved_by: platform-team
action: restore-last-known-good
restored_version: "1.0"
last_known_good: "1.0"
mixed_backup: false
unofficial: false
source_rewritten: false
result: allow
```

The evaluator loads Chapter 8's failed plane upgrade and fails a restore whose last known good is not `1.0`. It fails mixed backups, newest selection, unverified snapshots, `cluster-admin`, a plane identity as subject or approver, and source rewrite. It loads Chapter 8 `forbidden_roles` live. Clearing that list does not make a mixed apply legal. `platform-team` in both fields is the reviewed path, not the self-approval the newest restore impersonates.

> **Practice — Name continue or freeze per tenant, then keep Fulfillment's version**
>
> Storefront continues on `tenant-storage 2.0`. Fulfillment continues on `1.0`. Neither row may list the other tenant under `mutated_tenants`.

Open `recovery/isolation.yaml` and `recovery/verification.yaml`. Both tenants use `source: explicit-tenant-decision`. Storefront's restored version is `2.0`, matching Chapter 8's reconcile and Chapter 12's completed migration. Fulfillment stays `1.0`. Quota units are the Chapter 6 lease sums, 12 and 12. Traffic return lists the inherited pair, not `order_success_ratio`. Limitations include `not-regional-loss` and `not-portfolio-rto`. Completeness is computed.

The lab does not restore a real etcd. Completeness is whether mixed backup is rejected, whether each tenant's restored version matches the living fleet, and whether continue or freeze is explicit.

Prove the capability

Run the artifact audit and completed checkpoint:

```
make audit
make chapter-14-checkpoint
```

Expected output includes:

```
inherited interface verification: passed
artifact validation: passed
chapter 14 checkpoint: isolated plane restore and tenant continue/freeze
verified
```

The audit validates the four Chapter 14 files against pinned structural schemas. The checkpoint then applies semantic checks that schema validation alone cannot provide:

- plane evidence owner is a living Chapter 1 user and the plane is `kubernetes-control-plane`;
- last known good joins Chapter 8's retained `1.0`, not Chapter 12's contract `1.0`;
- an allowed restore of a mixed, newest, corrupt, or unverified snapshot fails;
- complete inherited roots do not legalize mixed tenants;
- an allowed restore whose subject or approver is `plane-reconciler` fails self-approval; `platform-team` in both fields does not;
- unofficial or `cluster-admin` restores fail;
- every Chapter 3 tenant has an explicit continue or freeze;
- Fulfillment intent replayed into Storefront fails, including a Storefront row restored to Fulfillment's `1.0`;
- restored quota units match leases and cannot starve a peer floor;
- a continuing tenant presents inherited traffic-return evidence, and a frozen tenant does not;
- `order_success_ratio` cannot prove bounded platform-product recovery; and
- verification cannot emit `status: recovered` and must record that this is not regional-loss or portfolio RTO.

The expected plane, jobs, limitations, and forbidden indicators live in a separate checkpoint file. A restore under test does not emit its own passing grade.

The checkpoint does not prove that a real control plane was restored, that a real freeze stopped Storefront checkout, or that 60 minutes is the right RTO. Those remain claims a local lab cannot make.

This distinction is the chapter's evidence model:

Evidence category	Chapter 14 evidence
Mechanism evidence	Schemas and the plane-restore evaluator operated successfully.
Decision evidence	Last known good versus newest, explicit continue or freeze, refused mixed snapshots, and SRE limitations are explicit.
Outcome evidence	Storefront has a computed continue on 2.0. Fulfillment has a computed continue on 1.0. That is not proof etcd was restored.
Recovery evidence	Produced and bounded. Mixed backup is rejected. Plane last known good 1.0 is the restored version. Tenant isolation holds in the model. That is **Evidence of restored isolation** and **Evidence of bounded platform-product recovery**. It is not DevSecOps restored trust, not a live-cluster recovery test, and not a regional-loss program.

Chapter 14 produces mechanism, decision, outcome, and bounded recovery evidence for isolated plane restore. Pretending the local checkpoint proves a multi-region disaster-recovery exercise would hand the planned SRE book a claim this lab cannot make.

4. Test the design under failure

Cumulative product failure — A corrupted newest plane backup is applied and replays Fulfillment intent into Storefront

> **Practice — Fail the newest mixed restore that writes Fulfillment into Storefront**
> Inject `plane-newest-corrupt` as an allowed restore with last known good `1.1`, and a Storefront row replayed from Fulfillment onto `1.0`, and refuse the apply.

The completed recovery model is healthy. The failure command does not rewrite it. It injects the residual mixed-backup case against that snapshot:

```
make chapter-14-failure
```

Expected output:

```
chapter 14 failure: mixed-backup restore correctly rejected
```

The injected restore is:

```
id: restore-newest-mixed
snapshot: plane-newest-corrupt
subject: plane-reconciler
approved_by: plane-reconciler
granted_role: cluster-admin
restored_version: "1.1"
last_known_good: "1.1"
mixed_backup: true
unofficial: true
source_rewritten: true
result: allow
```

The injected Storefront isolation row is:

```
tenant: storefront
replayed_from: fulfillment
mutated_tenants: [storefront, fulfillment]
restored_version: "1.0"
```

Fulfillment's own row stays `replayed_from: fulfillment`, `restored_version: "1.0"`, and `decision: continue`. Plane evidence still lists last known good `1.0`. The baseline's accidental freeze, order-success recovered indicators, and missing SRE limitation are not required for this failure. A mixed apply can appear after the inventory already looks complete. Chapter 8 already failed the move to plane `1.1`. Last known good `1.1` is that failure, now used as a restore source. Storefront's living version is `2.0`. Restoring Fulfillment's `1.0` onto Storefront is the replay.

Severity: high; every later conversation will restore an unofficial mixed plane instead of reviewed last known good and tenant isolation.
Plausible harm: Storefront's `sku` path becomes Fulfillment's `class` intent; warehouse and checkout share one reconstructed plane; both tenants look recovered because the API answered.
Potential blast radius: both application tenants, `kubernetes-control-plane`, and `cluster-capacity-pool`; Fulfillment's blast radius no longer stops at Fulfillment.
Bounded by: Chapter 3 isolation, Chapter 6 leases, Chapter 8 last known good and forbidden roles, Chapter 11 floors, Chapter 12 migrations, Chapter 13 change authority, and the inherited restore contract. None of those repair an allowed mixed apply.
Primary principles: blast-radius control, explicit contracts, trustworthy evidence.

Platform questions

- **User and job:** Fulfillment must finish `obtain-bounded-environment` or `ship-on-paved-road` after the plane returns. A mixed newest restore that writes Fulfillment into Storefront is not those jobs. It is shared authority substituting for last known good.
- **Isolation:** The boundary is Chapter 3's tenant plus this restore: Storefront's reconstructed intent must remain Storefront. Fulfillment's blast radius must stop at Fulfillment. This is not a new tenancy model. It is Chapter 8's tenant-scoped reconcile applied to recovery.
- **Contract and exit:** The contract is independently verified plane last known good plus per-tenant continue or freeze. Teams do not leave it the way they leave a paved-road scaffold. A mixed apply is not a Chapter 5 exit, not a Chapter 7 version bump, not a Chapter 9

exception, not a Chapter 10 non-metric, not a Chapter 11 burst, not a Chapter 12 fleet upgrade, and not a Chapter 13 reviewed change.

- **Platform-product evidence:** A restored apiserver is not proof the product works. Required proof is last known good `1.0`, no cross-tenant replay, explicit continue or freeze, and named platform jobs. This is not a portfolio **SLO (Service Level Objective)**. Storefront `order_success_ratio` remains tenant workload.

Diagnosis

Calling the write “newest is the best backup” encourages the same unofficial path Chapter 8 interrupted and Chapter 13 refused: Fulfillment is blocked, someone shares plane-admin, Storefront moves with the restore, and last known good becomes the broken version. Newest looks complete because it has every inherited root. Freeze looks safe because nobody is serving. Order-success looks like recovery because Storefront’s path flickered green.

The missing LKG join makes Chapter 8’s failed upgrade a successful restore. The missing replay check makes Chapter 12’s two contract versions one reconstructed tenant. Accidental freeze makes Chapter 3’s blast-radius statement a comment. Closing on `order_success_ratio` makes Chapter 10’s non-metric a memory.

Correction

The completed model does not allow `restore-newest-mixed`. The reviewed restore keeps subject `platform-team`, snapshot `plane-lkg-1-0`, last known good `1.0`, and `mixed_backup: false`. Storefront continues on `2.0` from Storefront. Fulfillment continues on `1.0` from Fulfillment. Completeness is computed. The failure command proves the check still fails when only the mixed apply and the Storefront replay are injected.

The denied mixed apply is a restore-authority invariant. It is not by itself the recovery. **Evidence of restored isolation** is the completed verification: plane last known good restored, tenant replay absent, continue or freeze explicit, platform jobs named, regional-loss not claimed. Nothing in that record is DevSecOps restored trust. Nothing in it is a multi-region fail-over.

That correction changes later decisions:

- The conclusion may assemble an owned internal platform only if restore cannot reconstruct every tenant as one blast radius.
- Mixed-backup restore must not replay a hero edit or a peer contract version into another tenant.
- Support may not freeze every tenant because one warehouse ticket was noisy during restore.
- Regional-loss and portfolio RTO remain SRE. This chapter records that limitation rather than filling it.

The design is practical because the failed mixed apply is the product. Adding an arbitrary etcd snapshot tool would not make it more practical.

5. Production reality

Common restore errors

Restoring newest because it completed most recently

Chapter 8 already failed plane `1.1`. Newest is that failure plus whatever mixed after it.

Treating complete restore roots as isolation

The inherited five roots prove reconstruction identity. They do not prove Storefront's intent stayed Storefront.

Freezing every tenant because the plane was down

Continue and freeze are explicit. Accident fails. An explicit freeze is legal and must not return traffic.

Borrowing Storefront order-success as platform recovery

The inherited traffic-return pair is `reconciled_business_state` and `verified_service_outcome`. `order_success_ratio` stays tenant workload.

Collapsing plane 1.0 and contract 1.0

Chapter 8 retains plane last known good. Chapter 12 retains contract last known good. Restore must not move either by applying Fulfillment's `1.0` onto Storefront's `2.0`.

Claiming regional-loss or portfolio RTO in this chapter

Record the limitation. Leave those programs to SRE. Do not restore a real cluster.

6. What changed

Before	After
Newest mixed 1.1 was applied as last known good.	**Mixed, newest, corrupt, and unverified snapshots fail the restore path.**

Before	After
Fulfillment intent replayed into Storefront.	**Each tenant restores its own living contract version.**
Both tenants froze because the plane was down.	**Continue and freeze are explicit tenant decisions.**
Order-success closed recovery.	**Tenant-workload ids cannot prove bounded platform-product recovery.**
A valid schema could appear to prove disaster recovery.	**Structural, decision, outcome, and bounded recovery evidence remain distinct.**

What changed was not merely four YAML files. Northwind now has a restore path the conclusion can name without treating a mixed newest backup as the product.

Durable outputs

Retain these reviewed outputs:

Artifact	Location	Keep it because
Control-plane recovery contract	recovery/plane-evidence.yaml and recovery/restore-trace.yaml	They retain independently verified last known good and the refused mixed newest snapshot later recovery must not promote.
Bounded isolation verification	recovery/isolation.yaml and recovery/verification.yaml	They retain explicit continue or freeze, per-tenant restored versions, and the SRE limitation a restored apiserver must not erase.

These artifacts should change when Northwind's plane last known good, living contract versions, or tenant freeze policy materially change—not whenever a backup tool is restyled.

What You Learned

Restore is independently verified last known good plus an explicit tenant decision. A newest mixed snapshot that writes Fulfillment into Storefront is shared plane-admin, not recovery. Schema checks can prove structural completeness within declared scope. They cannot restore a real control plane. A design earns its place when `plane-lkg-1-0` is restored, `plane-newest-corrupt` is refused, Storefront continues on `2.0`, Fulfillment continues on `1.0`, and `plane-reconciler` cannot approve a restore that moves last known good to `1.1`.

Prove It

> **Independent Practice — Restore after Storefront-only evidence loss without copying the mixed-backup row**
>
> Storefront's nonprod plane evidence is unverified. Fulfillment asks to keep shipping warehouse effects. Someone will offer to restore newest so both tenants return together.

Extend the Chapter 14 model without claiming regional-loss:

1. Decide whether Storefront continues, freezes, or is still waiting on independently verified evidence.
2. Name which snapshot is last known good—and which newest snapshot must not be selected even if it lists every inherited root.
3. State whether Fulfillment may continue while Storefront is frozen, or whether a shared-plane restore forces both.
4. Choose a sample that would falsify “Storefront-only loss can restore newest safely”—for example Storefront `replayed_from: fulfillment` with restored version `1.0`.
5. Identify which last known good that sample must retain—plane version, contract version, or both.
6. Explain which material change would trigger review of the recovery contract, not just one restore.

Do not copy the Fulfillment-into-Storefront mixed-backup row and rename it. A Storefront-only evidence gap has different continue/freeze and last-known-good consequences than a mixed newest apply. Your durable output is the snapshot-decision-and-isolation reasoning, not the number of YAML lines changed.

You can demonstrate the Chapter 14 capability when you can explain why newest is not last known good, trace a restore to Chapter 8 last known good and a Chapter 3 tenant decision, describe evidence that would falsify mixed-tenant replay, distinguish structural validation from decision, outcome, and recovery evidence, and explain what the baseline, checkpoint, and failure command do and do not prove.

Next

Product, tenancy, paved road, control plane, fleet, support, and bounded recovery now exist as one owned internal platform. The remaining work is to say what that platform is, what the five principles demand of it, what this book does not claim, and what the planned fourth book, *Practical SRE Engineering*, must take from here.

15. Conclusion — An Owned Internal Platform

Northwind began with a working delivery path and a working security path. Storefront could promote a digest, bind workload identity, and stop a bad release. Fulfillment still needed `fulfillment-api`. The residual looked like missing cluster access. Solving it exposed the larger production failure behind every ticket: a platform that was really a queue plus a cluster everyone could break.

A named user was useful only when a job could finish. A finished job still could not isolate Fulfillment from Storefront. Isolation still could not be consumed until a catalog named a living owner. A catalog still could not ship until a paved road existed that a team could leave. A road still shared blast radius until environments were leases, infrastructure was a versioned contract, and the shared plane was a product with a subject that could not approve itself.

The same reasoning continued beyond the first environment. Guardrails that could not expire became a cage. Measurement that could be gamed became a portal score. Quota that could starve a peer became shared root by another name. A fleet that applied `2.0` to every tenant at once broke a still-legal `1.0`. Support that patched the plane to close a ticket moved last known good to a version Chapter 8 had already refused. A restore of newest mixed both tenants back into one blast radius.

The book's central argument is therefore broader than a developer portal:

> Production platform engineering is the design of an owned internal product: users and jobs, tenant isolation, explicit contracts, fleet change, and recovery that does not reconstruct every tenant as one blast radius.

What Northwind can now do

Northwind can now:

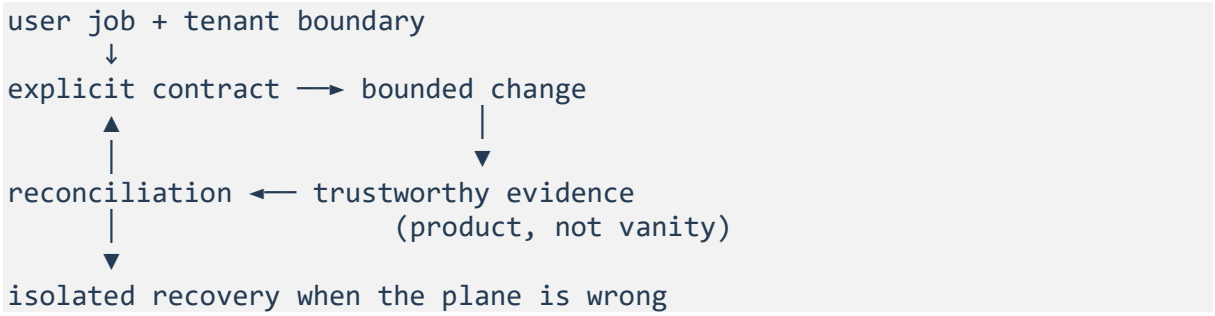
- name platform users, finished jobs, a promise, and refusals with remaining owners, and reject a portal launch as success;
- productize, leave, or decline a capability against those jobs instead of against ticket volume;
- isolate Storefront and Fulfillment so cluster-admin is not a temporary onboarding gift;
- publish a catalog whose owners and escalation contacts are living, not a deleted group;
- offer a paved road that remaining guardrails survive when a team leaves the scaffold;
- lease environments without a shared env-admin that can scale a peer tenant;
- bind infrastructure through reviewable contract versions, with a breaking rename as a bump;
- operate `kubernetes-control-plane` as a product, retain plane last known good `1.0`, and refuse plane self-approval;
- bind guardrails as defaults, reference inherited exceptions, and fail a green scorecard after expiry;
- measure developer experience as job time, with vanity and tenant-workload ids as distinct non-metrics;
- allocate floors and ceilings on `cluster-capacity-pool` so a burst cannot eat a peer floor;
- onboard without cluster-admin, upgrade by freeze and cohort, and keep Fulfillment's `1.0` legal until evidence exists;
- escalate a product wait to `platform-oncall` and an application delay to the tenant contact, and refuse unofficial plane-admin;
- restore the plane from independently verified last known good, reject mixed-tenant replay, and continue or freeze each tenant by explicit decision.

These capabilities form one product. They are not fourteen independent checklists. The promise they keep is the Chapter 1 contract:

> Application teams finish production jobs on a reviewed paved road, inside an explicit tenant boundary, without inheriting shared cluster authority.

The five principles as a product-control loop

The recurring principles now connect as one loop around the platform product, not around a single team's pipeline:



Blast-radius control keeps Fulfillment's change, quota, incident, and restore from becoming Storefront's outage.

Explicit contracts make the paved road, lease, infrastructure version, plane subject, exception binding, freeze window, and restore snapshot reviewable.

Trustworthy evidence separates observation from expectation. A scorecard, measurement file, change record, or verification cannot emit its own passing grade. Self-approval is not `subject == approved_by`: `platform-team` may be recorded as requester and approver; `plane-reconciler` may not.

Reconciliation compares desired tenant intent, admitted contract version, recorded last known good, and actual restore. Disagreement is owned. Newest is not last known good. Plane `1.0` is not contract `1.0`.

Recovery requires **Evidence of restored isolation** and **Evidence of bounded platform-product recovery**—not merely that the API answered, a ticket closed, or a backup job completed. Chapter 14 is the first chapter that produces that recovery evidence, and it is bounded: mixed backup rejected, plane last known good `1.0` restored, tenant isolation held in the model. It is not DevSecOps restored trust, and it is not proof a live portal, cluster, or backup platform recovered.

The four platform questions remain the reading contract:

1. Who is the user, and what job must finish?
2. What isolation boundary limits tenant blast radius?
3. What contract can a team rely on, and how do they leave it?
4. What evidence proves the platform product is healthy—not only a tenant workload?

A later change that cannot answer those four questions is not an improvement. It is a new unofficial path.

What this book does not claim

The companion lab is intentionally deterministic. It proves product decisions, tenancy, contracts, fleet windows, support authority, and bounded restore without requiring a live developer portal, identity provider, cluster fleet, billing export, or ticketing system. It does not prove that a particular organization's Backstage, Kubernetes, identity provider, cost system, or backup platform behaves as the fixture does.

Production adoption requires replacing each simulated boundary with observed evidence from the real implementation. Values such as time-to-first-environment, freeze length, floors, ceilings, and continue-or-freeze policy must be measured under the actual tenants and failure modes.

The scope is also deliberately bounded:

- *Practical DevOps Engineering* remains the delivery path for one team: fast feedback, verifiable artifacts, reconciled infrastructure, bounded runtime, workload identity, observable outcomes, progressive delivery, compatible data change, safe asynchronous processing, **GitOps (Git-based operations)**, cost of one workload, incident coordination, and reconstruction from durable evidence. This book productizes those capabilities. It does not reteach them.
- *Practical DevSecOps Engineering* remains the security of that path: threat and risk, attributable identity, privileged delegation, supply chain, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance. This book may expose those outcomes as defaults and exception UX. It does not rebuild them.
- *Practical SRE Engineering*, the planned fourth book, owns **SRE (Site Reliability Engineering)** programs: portfolio **SLO (Service Level Objective)** governance, error-budget policy across many services, on-call system design, regional-loss architecture, recurring game days, and reliability learning across a service portfolio. This book may define platform-product **SLIs (Service Level Indicators)** such as time-to-first-environment and paved-road completion. It does not own those SRE programs.
- Governance may consume platform contracts, scorecards, and exceptions. This book does not certify Northwind against a legal or industry framework.
- A dedicated AI-platform chapter, operator textbook, service-mesh reference, portal-product manual, and public-cloud landing-zone guide are out of scope.

Chapter 14 proves that inventoried plane evidence can be restored and tenant isolation can hold in the model. It does not claim that Northwind has completed an enterprise disaster-recovery program or a portfolio **RTO (Recovery Time Objective)**.

What the SRE book must take from here

The fourth book should inherit the platform product rather than rediscover it. The jobs `obtain-bounded-environment`, `ship-on-paved-road`, and `publish-owned-path` are already named. The isolation boundary is already Chapter 3's tenant plus Chapter 14's restore invariant. The platform product already has a job-time budget: time-to-finish for its defined jobs, not Storefront `order_success_ratio`. Escalation contacts already exist; they are not an on-call system. Last known good already exists in two places that must not collapse: plane `1.0` and contract `tenant-storage 1.0`.

What remains is reliability across a service portfolio: SLOs for Storefront and Fulfillment as user-facing services, error-budget policy that can freeze a fleet for a reason this book did not own, regional loss that Chapter 14 explicitly refused, and game days that exercise more than one mixed-backup fixture. Those are SRE questions. They are not unfinished platform chapters.

The production question to keep asking

When evaluating a new portal, path, guardrail, or restore tool, ask:

> Who is the user, what job must finish, where does tenant blast radius stop, what contract can they rely on and leave, and what evidence proves the platform product—not only one workload—is healthy?

That question keeps concepts subordinate to production work. It also prevents "best practice" from becoming a golden path copied without a job, an exit, or a restore that still isolates tenants.

The Northwind model and companion implementation are complete for the promise of this book. The lasting skill is not reproducing its YAML, cohort dates, or quota integers. It is being able to redesign the same internal platform when the organization, tenants, contracts, failure modes, and constraints are different.

Glossary and Abbreviations

Abbreviations

Abbreviation and full form	Use in this book
API (Application Programming Interface)	The tenant-visible contract for a platform capability, not the hidden Terraform or module internals.
CI/CD (Continuous Integration and Continuous Delivery)	The inherited delivery surface this book productizes; it is not taught from first principles.
CNI (Container Network Interface)	A hidden network-plugin detail. Tenants bind a network contract version; they do not set CNI config.
CSAT (Customer Satisfaction)	A vanity non-metric. A smiling survey cannot prove a job finished or close a platform incident.
FinOps (Financial Operations)	Cost of a useful, quality-gated unit. This book applies that idea to platform showback, not to Storefront's order path.
GitOps (Git-based operations)	Inherited reconciliation that forbids source rewrite. A fleet upgrade or plane restore is reviewed intent, not a controller edit.
IAM (Identity and Access Management)	A hidden identity-provider detail. Tenants bind workload-identity parameters; they do not name role ARNs.
OIDC (OpenID Connect)	A hidden federation detail used by the platform to keep short-lived identity. It is not a tenant API field.
RTO (Recovery Time Objective)	Maximum acceptable time to restore a required outcome. Portfolio RTO programs remain SRE.
SLI (Service Level Indicator)	A measurement of a platform-product job, such as time-to-first-environment. It is not a tenant-workload metric.
SLO (Service Level Objective)	An acceptable target for an SLI over a window. Portfolio SLO governance remains SRE.
SRE (Site Reliability Engineering)	Reliability engineering through service objectives, operations, and learning. The planned fourth book.
TTL (Time To Live)	The Chapter 6 lease lifetime. Chapter 12's freeze window is one TTL: 168 hours.
VPC (Virtual Private Cloud)	A real network the local lab does not provision. A lease models isolation; it does not create a VPC.

Production terms

Blast radius
The tenants, environments, jobs, or authority exposed to a change, failure, or restore. Fulfillment's blast radius must stop at Fulfillment.

Catalog freshness
Computed proof that a catalog owner and escalation contact are still living. A catalog file cannot emit `reported_status: green`.

Cohort
A fleet upgrade group of one or more tenants. Storefront can complete `tenant-storage 2.0` without Fulfillment absorbing `sku` in the same step.

Contract version bump
The Chapter 7 leaving act: a tenant-visible API version with a migration note. It is not a paved-road exit, a guardrail exception, or a fleet apply.

Decision evidence
Proof that an owner evaluated users, jobs, isolation, and trade-offs and made a bounded decision.

Evidence of bounded platform-product recovery
Proof that the modeled platform-product recovery invariant holds: mixed backup rejected, plane last known good restored, tenant isolation held in the model, platform jobs still named. It is not proof a live portal, cluster, or backup platform recovered, and it is not DevSecOps restored trust.

Evidence of restored isolation
Proof that tenant blast radius held after a modeled plane restore: mixed backup rejected, last known good restored, per-tenant continue or freeze explicit. It is not DevSecOps restored trust.

Exception binding
A platform row that references an inherited DevSecOps exception ID. It must not copy owner, scope, compensation, or expiry.

Freeze window
A fleet change halt with a start and end. Northwind's freeze is one Chapter 6 TTL.

Guardrail
A remaining default that still applies on the paved road and after a supported exit. It is not a golden cage and not a temporary chat waiver.

Isolated restore
Restoring `kubernetes-control-plane` from independently verified last known good without replaying one tenant's intent into another.

Job
A finished production outcome a named user must obtain from the platform. Opening a ticket or browsing a catalog is not a job.

Job-time budget
Unreliability permitted against platform-product job time, such as time-to-first-environment. The lab field `error_budget_indicators` records that budget. It is not an SRE portfolio error budget and not Storefront `order_success_ratio`.

Last known good
The version retained after failure. Plane last known good is Chapter 8's `1.0` after a failed upgrade to `1.1`. Contract last known good is Chapter 12's `tenant-storage` `1.0`. Do not collapse them because both say `1.0`.

Mechanism evidence
Proof that a catalog, lease, controller, scorecard, or evaluator operated. It does not prove a job finished.

Mixed backup
Plane evidence that contains more than one tenant's intent, or is newest, corrupt, or unverified. Applying it is the Chapter 14 cumulative product failure.

Non-goal
Work the platform refuses, with a remaining owner. A refusal without an owner is a vacuum, and the ticket returns.

Non-metric
A signal the measurement contract must name so it cannot be used as success. `category: vanity` means the signal is gameable. `category: tenant-workload` means the signal is real and belongs to a tenant path.

Outcome evidence
Proof that a team finished a production job inside the contract, such as computed paved-road completion. It is not a portal launch.

Paved road
The supported default path with inherited defaults, conformance, and a supported exit. Teams may leave the scaffold; remaining guardrails stay.

Platform product
An owned internal capability other teams consume through a contract. It has users, jobs, a promise, refusals, and success evidence that is not vanity.

Promise
The product's public contract. For Northwind: application teams finish production jobs on a reviewed paved road, inside an explicit tenant boundary, without inheriting shared cluster authority.

Recovery evidence
Proof that tenant blast radius was restored in the model—not merely that a ticket closed or an API answered. Chapters 1–13 do not produce it. Chapter 14 produces **Evidence of restored isolation** and **Evidence of bounded platform-product recovery**.

Reconciliation
Comparing desired tenant intent, admitted contract version, recorded last known good, and actual restore, then making disagreement visible and owned.

Red capability
A runnable baseline result proving that the chapter's declared weakness is present. A successful baseline is not a healthy product.

Remaining guardrail
A default that survives a supported exit: digest pinning, workload identity, and no cluster-admin stay after a team leaves the scaffold.

Self-approval
An allowed plane change or restore whose `subject` or `approved_by` is a Chapter 8 plane identity. It is not `subject == approved_by`. `platform-team` may appear in both fields; `plane-reconciler` may not.

Shared control plane
`kubernetes-control-plane` consumed as a product. Shared cluster-admin is not shared-control-plane. It is one plane with no tenants.

Showback
Allocation of platform unit cost to tenants. A cheaper shared bill is not a platform-product SLI.

Supported exit
The Chapter 5 leaving act: leave the scaffold while remaining guardrails stay. An unofficial fork is not an exit.

Tenant
An isolation and ownership boundary, not a Kubernetes namespace label. Storefront and Fulfillment are tenants.

Tenant-workload non-metric
A real signal that belongs to a tenant path, such as `order_success_ratio`. It must not prove the platform product is healthy.

Time-to-first-environment
The platform-product SLI for `obtain-bounded-environment`. A created namespace is not that proof.

Unofficial fork
A path that skips the paved road without a supported exit. Conformance fails; support does not cover it.

Vanity metric
A gameable signal treated as success: portal launch, CSAT, ticket volume, or adoption percentage after deleting unofficial paths.

Workload identity
An attributable runtime subject with issuer, audience, and expiry. The platform productizes it; this book does not reteach federation.

References

The chapter text cites sources at the decision they support. This consolidated list is organized by chapter for review and maintenance. Product behavior and documentation change; verify version-specific details before applying an example to production.

This book's production decisions are Northwind's. Chapters that make no product or standards claim—product definition, intake, tenancy, catalog, paved road, environments, contracts, control plane, guardrails, measurement, quota, fleet, support, restore, and the conclusion—have no external vendor or framework entries. They inherit delivery and security contracts from the earlier books rather than citing a portal-product manual, operator textbook, or landing-zone guide.

Series books

- *Practical DevOps Engineering* v1.0 — one team's production delivery path: fast feedback, verifiable artifacts, reconciled infrastructure, bounded runtime, workload identity, observable outcomes, progressive delivery, compatible data change, safe asynchronous processing, **GitOps (Git-based operations)**, cost of one workload, incident coordination, and reconstruction from durable evidence.
- *Practical DevSecOps Engineering* v1.0 — security of that same path: assets and harms, threat and risk, attributable identity, privileged delegation, supply-chain trust, vulnerability treatment, secrets, data protection, policy, runtime confinement, detection, investigation, eradication, bounded restored trust, and operational governance.
- *Practical SRE Engineering* — planned fourth book. Portfolio **SLO (Service Level Objective)** programs, error-budget governance, on-call systems, regional-loss architecture, recurring game days, and reliability learning across a service portfolio.

Inherited contracts this book consumes

The Platform lab does not read the DevOps or DevSecOps working trees at runtime. It carries reduced, checksum-identified interface fixtures. Reader-facing chapters join those contracts by identifier, not by restating the earlier implementations.

- DevOps release expectations — artifact identity and promotion used by paved-road defaults and infrastructure contracts.
- DevOps workload identity — subject, issuer, audience, and expiry used by environment leases and the control plane.
- DevOps observability contract — `order_success_ratio` and `order_latency` retained as tenant-workload non-metrics.
- DevOps GitOps reconciliation — reviewed intent and `controller_may_rewrite_source: false` used by the plane and fleet.
- DevOps incident-recovery expectations — tenant-path recovery evidence that must not close a platform incident.
- DevOps restore contract — reconstruction roots and traffic-return gates used by isolated plane restore.
- DevSecOps authorization, exception, control, and evidence-map shapes — self-approval forbidden, exception IDs referenced rather than copied, independent producer required.

Chapter 7 — Infrastructure contracts

No external module catalog is cited. Tenant-visible parameters and hidden internals are Northwind's contract design. Terraform, Helm, and provider documentation remain the earlier books' concern where a single team applied infrastructure.

Chapter 8 — Shared control plane

No Kubernetes control-plane operations guide is cited as a product claim. `kubernetes-control-plane` is Northwind's shared product identifier. Last known good after a failed upgrade is a platform retention rule, not a quoted kube-apiserver runbook.

Chapter 14 — Isolated restore

No enterprise disaster-recovery or multi-region program is cited. The inherited DevOps restore contract supplies reconstruction identity. Regional-loss architecture and portfolio **RTO (Recovery Time Objective)** remain **SRE (Site Reliability Engineering)**.